

VLLM专题（三十一）——架构概述



大模型专题系列 专栏收录该内容

您已是超级会员，正在免费阅读会员专享内容

[查看更多超级会员权益](#)

本文档提供了vLLM架构的概述。

1. 入口点

vLLM 提供了多个与系统交互的入口点。下图展示了它们之间的关系。

1.1 LLM 类

LLM 类提供了用于进行离线推理的主要 Python 接口，即在不使用单独的模型推理服务器的情况下与模型进行交互。

以下是 LLM 类的使用示例：

```

from vllm import LLM, SamplingParams

# Define a list of input prompts
prompts = [
    "Hello, my name is",
    "The capital of France is",
    "The largest ocean is",
]

# Define sampling parameters
sampling_params = SamplingParams(temperature=0.8, top_p=0.95)

# Initialize the LLM engine with the OPT-125M model
llm = LLM(model="facebook/opt-125m")

# Generate outputs for the input prompts
outputs = llm.generate(prompts, sampling_params)

# Print the generated outputs
for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {
        prompt!r}, Generated text: {
        generated_text!r}")

```

更多 API 详细信息可以在 API 文档的离线推理部分找到。

LLM 类的代码可以在如下 `llm.py` 中找到：

```

# SPDX-License-Identifier: Apache-2.0

```

```
import itertools
import warnings
from collections.abc import Sequence
from contextlib import contextmanager
from typing import Any, Callable, ClassVar, Optional, Union, cast, overload

import cloudpickle
import torch.nn as nn
from tqdm import tqdm
from typing_extensions import TypeVar, deprecated

from vllm.beam_search import (BeamSearchInstance, BeamSearchOutput,
                             BeamSearchSequence, get_beam_search_score)
from vllm.config import CompilationConfig
from vllm.engine.arg_utils import (EngineArgs, HfOverrides, PoolerConfig,
                                   TaskOption)
from vllm.engine.llm_engine import LLMEngine
from vllm.entrypoints.chat_utils import (ChatCompletionMessageParam,
                                         ChatTemplateContentFormatOption,
                                         apply_hf_chat_template,
                                         apply_mistral_chat_template,
                                         parse_chat_messages,
                                         resolve_chat_template_content_format)
from vllm.entrypoints.score_utils import (_cosine_similarity,
                                         _validate_score_input_lens)
from vllm.inputs import PromptType, SingletonPrompt, TextPrompt, TokensPrompt
from vllm.inputs.parse import is_token_prompt, parse_and_batch_prompt
from vllm.logger import init_logger
from vllm.lora.request import LoRARequest
from vllm.model_executor.guided_decoding.guided_fields import (
    GuidedDecodingRequest, LLMGuidedOptions)
from vllm.outputs import (ClassificationRequestOutput, EmbeddingRequestOutput,
```

```

from vllm.outputs import (ClassificationRequestOutput, EmbeddingRequestOutput,
                           PoolingRequestOutput, RequestOutput,
                           ScoringRequestOutput)
from vllm.pooling_params import PoolingParams
from vllm.prompt_adapter.request import PromptAdapterRequest
from vllm.sampling_params import (BeamSearchParams, GuidedDecodingParams,
                                  RequestOutputKind, SamplingParams)
from vllm.transformers_utils.tokenizer import (AnyTokenizer, MistralTokenizer,
                                                get_cached_tokenizer)
from vllm.transformers_utils.tokenizer_group import TokenizerGroup
from vllm.usage.usage_lib import UsageContext
from vllm.utils import Counter, deprecate_args, deprecate_kwargs, is_list_of

logger = init_logger(__name__)

_R = TypeVar("_R", default=Any)

class LLM:
    """An LLM for generating texts from given prompts and sampling parameters.

    This class includes a tokenizer, a language model (possibly distributed
    across multiple GPUs), and GPU memory space allocated for intermediate
    states (aka KV cache). Given a batch of prompts and sampling parameters,
    this class generates texts from the model, using an intelligent batching
    mechanism and efficient memory management.

    Args:
        model: The name or path of a HuggingFace Transformers model.
        tokenizer: The name or path of a HuggingFace Transformers tokenizer.
        tokenizer_mode: The tokenizer mode. "auto" will use the fast tokenizer
            if available, and "slow" will always use the slow tokenizer.
    """

```

`skip_tokenizer_init`: If true, skip initialization of tokenizer and detokenizer. Expect valid `prompt_token_ids` and None for prompt from the input.

`trust_remote_code`: Trust remote code (e.g., from HuggingFace) when downloading the model and tokenizer.

`allowed_local_media_path`: Allowing API requests to read local images or videos from directories specified by the server file system. This is a security risk. Should only be enabled in trusted environments.

`tensor_parallel_size`: The number of GPUs to use for distributed execution with tensor parallelism.

`dtype`: The data type for the model weights and activations. Currently, we support ``float32``, ``float16``, and ``bfloat16``. If ``auto``, we use the ``torch_dtype`` attribute specified in the model config file. However, if the ``torch_dtype`` in the config is ``float32``, we will use ``float16`` instead.

`quantization`: The method used to quantize the model weights. Currently, we support "awq", "gptq", and "fp8" (experimental). If None, we first check the ``quantization_config`` attribute in the model config file. If that is None, we assume the model weights are not quantized and use ``dtype`` to determine the data type of the weights.

`revision`: The specific model version to use. It can be a branch name, a tag name, or a commit id.

`tokenizer_revision`: The specific tokenizer version to use. It can be a branch name, a tag name, or a commit id.

`seed`: The seed to initialize the random number generator for sampling.

`gpu_memory_utilization`: The ratio (between 0 and 1) of GPU memory to reserve for the model weights, activations, and KV cache. Higher values will increase the KV cache size and thus improve the model's throughput. However, if the value is too high, it may cause out-of-memory (OOM) errors.

`swap_space`: The size (GiB) of CPU memory per GPU to use as swap space. This can be used for temporarily storing the states of the requests when their ``best_of`` sampling parameters are larger than 1. If all requests will have ``best_of=1``, you can safely set this to 0. Noting that ``best_of`` is only supported in V0. Otherwise, too small values may cause out-of-memory (OOM) errors.

`cpu_offload_gb`: The size (GiB) of CPU memory to use for offloading the model weights. This virtually increases the GPU memory space you can use to hold the model weights, at the cost of CPU-GPU data transfer for every forward pass.

`enforce_eager`: Whether to enforce eager execution. If True, we will disable CUDA graph and always execute the model in eager mode. If False, we will use CUDA graph and eager execution in hybrid.

`max_seq_len_to_capture`: Maximum sequence len covered by CUDA graphs. When a sequence has context length larger than this, we fall back to eager mode. Additionally for encoder-decoder models, if the sequence length of the encoder input is larger than this, we fall back to the eager mode.

`disable_custom_all_reduce`: See `:class:`~vllm.config.ParallelConfig``

`disable_async_output_proc`: Disable async output processing. This may result in lower performance.

`hf_overrides`: If a dictionary, contains arguments to be forwarded to the HuggingFace config. If a callable, it is called to update the HuggingFace config.

`compilation_config`: Either an integer or a dictionary. If it is an integer, it is used as the level of compilation optimization. If it is a dictionary, it can specify the full compilation configuration.

`**kwargs`: Arguments for `:class:`~vllm.EngineArgs``. (See `:ref:`engine-args``)

Note:

This class is intended to be used for offline inference. For online

```

        This class is intended to be used for offline inference. For online
        serving, use the :class:`~vllm.AsyncLLMEngine` class instead.
    """

    DEPRECATE_LEGACY: ClassVar[bool] = True
    """A flag to toggle whether to deprecate the legacy generate/encode API."""

    DEPRECATE_INIT_POSARGS: ClassVar[bool] = True
    """
    A flag to toggle whether to deprecate positional arguments in
    :meth:`~LLM.__init__`.
    """

    @classmethod
    @contextmanager
    def deprecate_legacy_api(cls):
        cls.DEPRECATE_LEGACY = True

        yield

        cls.DEPRECATE_LEGACY = False

    @deprecate_args(
        start_index=2, # Ignore self and model
        is_deprecated=lambda: LLM.DEPRECATE_INIT_POSARGS,
        additional_message=(
            "All positional arguments other than `model` will be "
            "replaced with keyword arguments in an upcoming version."),
    )
    def __init__(
        self,
        model: str,

```

```

tokenizer: Optional[str] = None,
tokenizer_mode: str = "auto",
skip_tokenizer_init: bool = False,
trust_remote_code: bool = False,
allowed_local_media_path: str = "",
tensor_parallel_size: int = 1,
dtype: str = "auto",
quantization: Optional[str] = None,
revision: Optional[str] = None,
tokenizer_revision: Optional[str] = None,
seed: Optional[int] = None,
gpu_memory_utilization: float = 0.9,
swap_space: float = 4,
cpu_offload_gb: float = 0,
enforce_eager: Optional[bool] = None,
max_seq_len_to_capture: int = 8192,
disable_custom_all_reduce: bool = False,
disable_async_output_proc: bool = False,
hf_overrides: Optional[HfOverrides] = None,
mm_processor_kwargs: Optional[dict[str, Any]] = None,
# After positional args are removed, move this right below `model`
task: TaskOption = "auto",
override_pooler_config: Optional[PoolerConfig] = None,
compilation_config: Optional[Union[int, dict[str, Any]]] = None,
**kwargs,
) -> None:
    ...

    LLM constructor.

    Note: if enforce_eager is unset (enforce_eager is None)
    it defaults to False.
    ...

```



```

if "disable_log_stats" not in kwargs:
    kwargs["disable_log_stats"] = True

if "worker_cls" in kwargs:
    worker_cls = kwargs["worker_cls"]
    # if the worker_cls is not qualified string name,
    # we serialize it using cloudpickle to avoid pickling issues
    if isinstance(worker_cls, type):
        kwargs["worker_cls"] = cloudpickle.dumps(worker_cls)

if compilation_config is not None:
    if isinstance(compilation_config, (int, dict)):
        compilation_config_instance = CompilationConfig.from_cli(
            str(compilation_config))
    else:
        compilation_config_instance = compilation_config
else:
    compilation_config_instance = None

engine_args = EngineArgs(
    model=model,
    task=task,
    tokenizer=tokenizer,
    tokenizer_mode=tokenizer_mode,
    skip_tokenizer_init=skip_tokenizer_init,
    trust_remote_code=trust_remote_code,
    allowed_local_media_path=allowed_local_media_path,
    tensor_parallel_size=tensor_parallel_size,
    dtype=dtype,
    quantization=quantization,
    revision=revision

```

```

revision=revision,
tokenizer_revision=tokenizer_revision,
seed=seed,
gpu_memory_utilization=gpu_memory_utilization,
swap_space=swap_space,
cpu_offload_gb=cpu_offload_gb,
enforce_eager=enforce_eager,
max_seq_len_to_capture=max_seq_len_to_capture,
disable_custom_all_reduce=disable_custom_all_reduce,
disable_async_output_proc=disable_async_output_proc,
hf_overrides=hf_overrides,
mm_processor_kwargs=mm_processor_kwargs,
override_pooler_config=override_pooler_config,
compilation_config=compilation_config_instance,
**kwargs,
)

# Create the Engine (autoselects V0 vs V1)
self.llm_engine = LLMEngine.from_engine_args(
    engine_args=engine_args, usage_context=UsageContext.LLM_CLASS)
self.engine_class = type(self.llm_engine)

self.request_counter = Counter()
self.default_sampling_params: Union[dict[str, Any], None] = None

def get_tokenizer(self) -> AnyTokenizer:
    return self.llm_engine.get_tokenizer_group(TokenizerGroup).tokenizer

def set_tokenizer(self, tokenizer: AnyTokenizer) -> None:
    tokenizer_group = self.llm_engine.get_tokenizer_group(TokenizerGroup)

# While CachedTokenizer is dynamic, have no choice but

```

```

# compare class name. Misjudgment will arise from
# user-defined tokenizer started with 'Cached'
if tokenizer.__class__.__name__.startswith("Cached"):
    tokenizer_group.tokenizer = tokenizer
else:
    tokenizer_group.tokenizer = get_cached_tokenizer(tokenizer)

def get_default_sampling_params(self) -> SamplingParams:
    if self.default_sampling_params is None:
        self.default_sampling_params = (
            self.llm_engine.model_config.get_diff_sampling_param())
    if self.default_sampling_params:
        return SamplingParams.from_optional(**self.default_sampling_params)
    return SamplingParams()

@overload
def generate(
    self,
    prompts: Union[PromptType, Sequence[PromptType]],
    /,
    sampling_params: Optional[Union[SamplingParams,
                                    Sequence[SamplingParams]]] = None,
    *,
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
    guided_options_request: Optional[Union[LLMGuidedOptions,
                                            GuidedDecodingRequest]] = None,
) -> list[RequestOutput]:
    ...

@overload # LEGACY: single (prompt + optional token ids)

```

```

@deprecated("'prompt_token_ids' will become part of 'prompts'")
def generate(
    self,
    prompts: str,
    sampling_params: Optional[Union[SamplingParams,
                                    list[SamplingParams]]] = None,
    prompt_token_ids: Optional[list[int]] = None,
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
    guided_options_request: Optional[Union[LLMGuidedOptions,
                                           GuidedDecodingRequest]] = None,
) -> list[RequestOutput]:
    ...

```

```

@overload # LEGACY: multi (prompt + optional token ids)
@deprecated("'prompt_token_ids' will become part of 'prompts'")
def generate(
    self,
    prompts: list[str],
    sampling_params: Optional[Union[SamplingParams,
                                    list[SamplingParams]]] = None,
    prompt_token_ids: Optional[list[list[int]]] = None,
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
    guided_options_request: Optional[Union[LLMGuidedOptions,
                                           GuidedDecodingRequest]] = None,
) -> list[RequestOutput]:
    ...

```

```

@overload # LEGACY: single (token ids + optional prompt)

```

```

@overload # LEGACY: single (token ids + optional prompt)
@deprecated("'prompt_token_ids' will become part of 'prompts'")
def generate(
    self,
    prompts: Optional[str] = None,
    sampling_params: Optional[Union[SamplingParams,
                                    list[SamplingParams]]] = None,
    *,
    prompt_token_ids: list[int],
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
    guided_options_request: Optional[Union[LLMGuidedOptions,
                                           GuidedDecodingRequest]] = None,
) -> list[RequestOutput]:
    ...

@overload # LEGACY: multi (token ids + optional prompt)
@deprecated("'prompt_token_ids' will become part of 'prompts'")
def generate(
    self,
    prompts: Optional[list[str]] = None,
    sampling_params: Optional[Union[SamplingParams,
                                    list[SamplingParams]]] = None,
    *,
    prompt_token_ids: list[list[int]],
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
    guided_options_request: Optional[Union[LLMGuidedOptions,
                                           GuidedDecodingRequest]] = None,
) -> list[RequestOutput]:

```

```

...

@overload # LEGACY: single or multi token ids [pos-only]
@deprecated("'prompt_token_ids' will become part of 'prompts'")
def generate(
    self,
    prompts: None,
    sampling_params: None,
    prompt_token_ids: Union[list[int], list[list[int]]],
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
    guided_options_request: Optional[Union[LLMGuidedOptions,
                                           GuidedDecodingRequest]] = None,
) -> list[RequestOutput]:
    ...

@deprecate_kwargs(
    "prompt_token_ids",
    is_deprecated=lambda: LLM.DEPRECATE_LEGACY,
    additional_message="Please use the 'prompts' parameter instead.",
)
def generate(
    self,
    prompts: Union[Union[PromptType, Sequence[PromptType]],
                  Optional[Union[str, list[str]]]] = None,
    sampling_params: Optional[Union[SamplingParams,
                                    Sequence[SamplingParams]]] = None,
    prompt_token_ids: Optional[Union[list[int], list[list[int]]]] = None,
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,

```

```
guided_options_request: Optional[Union[LLMGuidedOptions,  
                                       GuidedDecodingRequest]] = None,  
priority: Optional[list[int]] = None,  
) -> list[RequestOutput]:
```

"""Generates the completions for the input prompts.

This class automatically batches the given prompts, considering the memory constraint. For the best performance, put all of your prompts into a single list and pass it to this method.

Args:

prompts: The prompts to the LLM. You may pass a sequence of prompts for batch inference. See :class:`~vllm.inputs.PromptType` for more details about the format of each prompts.

sampling_params: The sampling parameters for text generation. If None, we use the default sampling parameters.
When it is a single value, it is applied to every prompt.
When it is a list, the list must have the same length as the prompts and it is paired one by one with the prompt.

use_tqdm: Whether to use tqdm to display the progress bar.

lora_request: LoRA request to use for generation, if any.

prompt_adapter_request: Prompt Adapter request to use for generation, if any.

priority: The priority of the requests, if any.

Only applicable when priority scheduling policy is enabled.

Returns:

A list of ``RequestOutput`` objects containing the generated completions in the same order as the input prompts.

Note:

Using ``prompts`` and ``prompt token ids`` as keyword parameters is

```

using `prompt` and `prompt_token_ids` as keyword parameters is
considered legacy and may be deprecated in the future. You should
instead pass them via the ``inputs`` parameter.
"""
runner_type = self.llm_engine.model_config.runner_type
if runner_type not in ["generate", "transcription"]:
    messages = [
        "LLM.generate() is only supported for (conditional) generation "
        "models (XForCausalLM, XForConditionalGeneration).",
    ]

    supported_runner_types = self.llm_engine.model_config \
        .supported_runner_types
    if "generate" in supported_runner_types:
        messages.append(
            "Your model supports the 'generate' runner, but is "
            f"currently initialized for the '{
runner_type}' runner. "
            "Please initialize vLLM using `--task generate`.")

    raise ValueError(" ".join(messages))

if prompt_token_ids is not None:
    parsed_prompts = self._convert_v1_inputs(
        prompts=cast(Optional[Union[str, list[str]]], prompts),
        prompt_token_ids=prompt_token_ids,
    )
else:
    parsed_prompts = cast(Union[PromptType, Sequence[PromptType]],
        prompts)

if isinstance(guided_options_request, dict):
    if "guided_options_request" in kwargs:
        raise ValueError(

```



```

        if len(guided_options_request) > 1:
            raise ValueError(
                "You can only use one guided decoding but multiple is "
                f"specified: {
                    guided_options_request}")
        guided_options_request = GuidedDecodingRequest(
            **guided_options_request)

    if sampling_params is None:
        # Use default sampling params.
        sampling_params = self.get_default_sampling_params()

    self._validate_and_add_requests(
        prompts=parsed_prompts,
        params=sampling_params,
        lora_request=lora_request,
        prompt_adapter_request=prompt_adapter_request,
        guided_options=guided_options_request,
        priority=priority)

    outputs = self._run_engine(use_tqdm=use_tqdm)
    return self.engine_class.validate_outputs(outputs, RequestOutput)

def collective_rpc(self,
    method: Union[str, Callable[..., _R]],
    timeout: Optional[float] = None,
    args: tuple = (),
    kwargs: Optional[dict[str, Any]] = None) -> list[_R]:
    """
    Execute an RPC call on all workers.

    Args:

```

method: Name of the worker method to execute, or a callable that is serialized and sent to all workers to execute.

If the method is a callable, it should accept an additional `self` argument, in addition to the arguments passed in `args` and `kwargs`. The `self` argument will be the worker object.

timeout: Maximum time in seconds to wait for execution. Raises a :exc:`TimeoutError` on timeout. `None` means wait indefinitely.

args: Positional arguments to pass to the worker method.

kwargs: Keyword arguments to pass to the worker method.

Returns:

A list containing the results from each worker.

Note:

It is recommended to use this API to only pass control messages, and set up data-plane communication to pass data.

"""

```
executor = self.llm_engine.model_executor
return executor.collective_rpc(method, timeout, args, kwargs)
```

```
def apply_model(self, func: Callable[[nn.Module], _R]) -> list[_R]:
```

"""

Run a function directly on the model inside each worker, returning the result for each of them.

"""

```
executor = self.llm_engine.model_executor
return executor.apply_model(func)
```

```
def beam_search(
```

```
    self,
```

```
    prompts: list[Union[TokensPrompt, TextPrompt]],
```

```

        params: BeamSearchParams,
    ) -> list[BeamSearchOutput]:
        """
        Generate sequences using beam search.

        Args:
            prompts: A list of prompts. Each prompt can be a string or a list
                    of token IDs.
            params: The beam search parameters.

        TODO: how does beam search work together with length penalty, frequency
        penalty, and stopping criteria, etc.?
        """

        beam_width = params.beam_width
        max_tokens = params.max_tokens
        temperature = params.temperature
        ignore_eos = params.ignore_eos
        length_penalty = params.length_penalty

        def sort_beams_key(x: BeamSearchSequence) -> float:
            return get_beam_search_score(x.tokens, x.cum_logprob,
                                         tokenizer.eos_token_id,
                                         length_penalty)

        tokenizer = self.get_tokenizer()
        # generate 2 * beam_width candidates at each step
        # following the huggingface transformers implementation
        # at https://github.com/huggingface/transformers/blob/e15687fffe5c9d20598a19aeab7
        21ae0a7580f8a/src/transformers/generation/beam_search.py#L534 # noqa
        beam_search_params = SamplingParams(logprobs=2 * beam_width,

```

[illegible]

```

        use_tqdm=False)

    for (start, end), instance in zip(instance_start_and_end,
                                     instances):

        instance_new_beams = []
        for i in range(start, end):
            current_beam = all_beams[i]
            result = output[i]

            if result.outputs[0].logprobs is not None:
                # if `result.outputs[0].logprobs` is None, it means
                # the sequence is completed because of the max-model-len
                # or abortion. we don't need to add it to the new beams.
                logprobs = result.outputs[0].logprobs[0]
                for token_id, logprob_obj in logprobs.items():
                    new_beam = BeamSearchSequence(
                        tokens=current_beam.tokens + [token_id],
                        logprobs=current_beam.logprobs + [logprobs],
                        cum_logprob=current_beam.cum_logprob +
                        logprob_obj.logprob)

                    if token_id == tokenizer.eos_token_id and \
                       not ignore_eos:
                        instance.completed.append(new_beam)
                    else:
                        instance_new_beams.append(new_beam)
        sorted_beams = sorted(instance_new_beams,
                              key=sort_beams_key,
                              reverse=True)
        instance.beams = sorted_beams[:beam_width]

outputs = []

```

```

        for instance in instances:
            instance.completed.extend(instance.beams)
            sorted_completed = sorted(instance.completed,
                                      key=sort_beams_key,
                                      reverse=True)
            best_beams = sorted_completed[:beam_width]

            for beam in best_beams:
                beam.text = tokenizer.decode(beam.tokens)
                outputs.append(BeamSearchOutput(sequences=best_beams))

    return outputs

def chat(
    self,
    messages: Union[list[ChatCompletionMessageParam],
                    list[list[ChatCompletionMessageParam]]],
    sampling_params: Optional[Union[SamplingParams,
                                    list[SamplingParams]]] = None,
    use_tqdm: bool = True,
    lora_request: Optional[LoRARequest] = None,
    chat_template: Optional[str] = None,
    chat_template_content_format: ChatTemplateContentFormatOption = "auto",
    add_generation_prompt: bool = True,
    continue_final_message: bool = False,
    tools: Optional[list[dict[str, Any]]] = None,
    mm_processor_kwargs: Optional[dict[str, Any]] = None,
) -> list[RequestOutput]:
    """
    Generate responses for a chat conversation.

    The chat conversation is converted into a text prompt using the

```

The chat conversation is converted into a text prompt using the tokenizer and calls the :meth:`generate` method to generate the responses.

Multi-modal inputs can be passed in the same way you would pass them to the OpenAI API.

Args:

messages: A list of conversations or a single conversation.

- Each conversation is represented as a list of messages.
- Each message is a dictionary with 'role' and 'content' keys.

sampling_params: The sampling parameters for text generation.

If None, we use the default sampling parameters. When it is a single value, it is applied to every prompt. When it is a list, the list must have the same length as the prompts and it is paired one by one with the prompt.

use_tqdm: Whether to use tqdm to display the progress bar.

lora_request: LoRA request to use for generation, if any.

chat_template: The template to use for structuring the chat.

If not provided, the model's default chat template will be used.

chat_template_content_format: The format to render message content.

- "string" will render the content as a string.

Example: ``"Who are you?"``

- "openai" will render the content as a list of dictionaries, similar to OpenAI schema.

Example: ``[{"type": "text", "text": "Who are you?"}]``

add_generation_prompt: If True, adds a generation template to each message.

```

        continue_final_message: If True, continues the final message in
        the conversation instead of starting a new one. Cannot be
        ``True`` if ``add_generation_prompt`` is also ``True``.
    mm_processor_kwargs: Multimodal processor kwarg overrides for this
        chat request. Only used for offline requests.

Returns:
    A list of ``RequestOutput`` objects containing the generated
    responses in the same order as the input messages.
"""
list_of_messages: list[list[ChatCompletionMessageParam]]

# Handle multi and single conversations
if is_list_of(messages, list):
    # messages is list[list[...]]
    list_of_messages = cast(list[list[ChatCompletionMessageParam]],
                           messages)
else:
    # messages is list[...]
    list_of_messages = [
        cast(list[ChatCompletionMessageParam], messages)
    ]

tokenizer = self.get_tokenizer()
model_config = self.llm_engine.get_model_config()
resolved_content_format = resolve_chat_template_content_format(
    chat_template,
    chat_template_content_format,
    tokenizer,
)

prompts: list[Union[TokensPrompt, TextPrompt]] = []

```



```

for msgs in list_of_messages:
    # NOTE: _parse_chat_message_content_parts() currently doesn't
    # handle mm_processor_kwargs, since there is no implementation in
    # the chat message parsing for it.
    conversation, mm_data = parse_chat_messages(
        msgs,
        model_config,
        tokenizer,
        content_format=resolved_content_format,
    )

    prompt_data: Union[str, list[int]]
    if isinstance(tokenizer, MistralTokenizer):
        prompt_data = apply_mistral_chat_template(
            tokenizer,
            messages=msgs,
            chat_template=chat_template,
            add_generation_prompt=add_generation_prompt,
            continue_final_message=continue_final_message,
            tools=tools,
        )
    else:
        prompt_data = apply_hf_chat_template(
            tokenizer,
            conversation=conversation,
            chat_template=chat_template,
            add_generation_prompt=add_generation_prompt,
            continue_final_message=continue_final_message,
            tools=tools,
        )

```

```

        prompt: Union[TokensPrompt, TextPrompt]
        if is_list_of(prompt_data, int):
            prompt = TokensPrompt(prompt_token_ids=prompt_data)
        else:
            prompt = TextPrompt(prompt=prompt_data)

        if mm_data is not None:
            prompt["multi_modal_data"] = mm_data

        if mm_processor_kwargs is not None:
            prompt["mm_processor_kwargs"] = mm_processor_kwargs

        prompts.append(prompt)

    return self.generate(
        prompts,
        sampling_params=sampling_params,
        use_tqdm=use_tqdm,
        lora_request=lora_request,
    )

@overload
def encode(
    self,
    prompts: Union[PromptType, Sequence[PromptType]],
    /,
    pooling_params: Optional[Union[PoolingParams,
                                   Sequence[PoolingParams]]] = None,
    *,
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,

```

```

        prompt_adapter_request: Optional[PromptAdapterRequest] = None,
    ) -> list[PoolingRequestOutput]:
        ...

@overload # LEGACY: single (prompt + optional token ids)
@deprecated("'prompt_token_ids' will become part of 'prompts'")
def encode(
    self,
    prompts: str,
    pooling_params: Optional[Union[PoolingParams,
                                   Sequence[PoolingParams]]] = None,
    prompt_token_ids: Optional[list[int]] = None,
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
) -> list[PoolingRequestOutput]:
    ...

@overload # LEGACY: multi (prompt + optional token ids)
@deprecated("'prompt_token_ids' will become part of 'prompts'")
def encode(
    self,
    prompts: list[str],
    pooling_params: Optional[Union[PoolingParams,
                                   Sequence[PoolingParams]]] = None,
    prompt_token_ids: Optional[list[list[int]]] = None,
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
) -> list[PoolingRequestOutput]:
    ...

```

```

@overload # LEGACY: single (token ids + optional prompt)
@deprecated("'prompt_token_ids' will become part of 'prompts'")
def encode(
    self,
    prompts: Optional[str] = None,
    pooling_params: Optional[Union[PoolingParams,
                                   Sequence[PoolingParams]]] = None,
    *,
    prompt_token_ids: list[int],
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
) -> list[PoolingRequestOutput]:
    ...

@overload # LEGACY: multi (token ids + optional prompt)
@deprecated("'prompt_token_ids' will become part of 'prompts'")
def encode(
    self,
    prompts: Optional[list[str]] = None,
    pooling_params: Optional[Union[PoolingParams,
                                   Sequence[PoolingParams]]] = None,
    *,
    prompt_token_ids: list[list[int]],
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
) -> list[PoolingRequestOutput]:
    ...

@overload # LEGACY: single or multi token ids [pos-only]
@deprecated("'prompt_token_ids' will become part of 'prompts'")

```

```

@deprecated( "prompt_token_ids will become part of 'prompts' ")
def encode(
    self,
    prompts: None,
    pooling_params: None,
    prompt_token_ids: Union[list[int], list[list[int]]],
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
) -> list[PoolingRequestOutput]:
    ...

@deprecate_kwargs(
    "prompt_token_ids",
    is_deprecated=lambda: LLM.DEPRECATE_LEGACY,
    additional_message="Please use the 'prompts' parameter instead.",
)
def encode(
    self,
    prompts: Union[Union[PromptType, Sequence[PromptType]],
                    Optional[Union[str, list[str]]]] = None,
    pooling_params: Optional[Union[PoolingParams,
                                    Sequence[PoolingParams]]] = None,
    prompt_token_ids: Optional[Union[list[int], list[list[int]]]] = None,
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
) -> list[PoolingRequestOutput]:
    """Apply pooling to the hidden states corresponding to the input
    prompts.

    This class automatically batches the given prompts, considering

```

the memory constraint. For the best performance, put all of your prompts into a single list and pass it to this method.

Args:

`prompts`: The prompts to the LLM. You may pass a sequence of prompts for batch inference. See `:class:`~vllm.inputs.PromptType`` for more details about the format of each prompts.

`pooling_params`: The pooling parameters for pooling. If None, we use the default pooling parameters.

`use_tqdm`: Whether to use tqdm to display the progress bar.

`lora_request`: LoRA request to use for generation, if any.

`prompt_adapter_request`: Prompt Adapter request to use for generation, if any.

Returns:

A list of ```PoolingRequestOutput``` objects containing the pooled hidden states in the same order as the input prompts.

Note:

Using ```prompts``` and ```prompt_token_ids``` as keyword parameters is considered legacy and may be deprecated in the future. You should instead pass them via the ```inputs``` parameter.

"""

```
runner_type = self.llm_engine.model_config.runner_type
if runner_type != "pooling":
    messages = ["LLM.encode() is only supported for pooling models."]

    supported_runner_types = self.llm_engine.model_config \
        .supported_runner_types
    if "pooling" in supported_runner_types:
        messages.append(
            "Your model supports the 'pooling' runner, but is "
```

```

        f"currently initialized for the '{
runner_type}' runner. "
        "Please initialize vLLM using `--task embed`, "
        "`--task classify`, `--task score` etc.")

    raise ValueError(" ".join(messages))

if prompt_token_ids is not None:
    parsed_prompts = self._convert_v1_inputs(
        prompts=cast(Optional[Union[str, list[str]]], prompts),
        prompt_token_ids=prompt_token_ids,
    )
else:
    parsed_prompts = cast(Union[PromptType, Sequence[PromptType]],
                          prompts)

if pooling_params is None:
    # Use default pooling params.
    pooling_params = PoolingParams()

self._validate_and_add_requests(
    prompts=parsed_prompts,
    params=pooling_params,
    lora_request=lora_request,
    prompt_adapter_request=prompt_adapter_request,
)

outputs = self._run_engine(use_tqdm=use_tqdm)
return self.engine_class.validate_outputs(outputs,
                                          PoolingRequestOutput)

```

```
def embed/
```

```

def embed(
    self,
    prompts: Union[PromptType, Sequence[PromptType]],
    /,
    *,
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
) -> list[EmbeddingRequestOutput]:
    """
    Generate an embedding vector for each prompt.

    This class automatically batches the given prompts, considering
    the memory constraint. For the best performance, put all of your prompts
    into a single list and pass it to this method.

    Args:
        prompts: The prompts to the LLM. You may pass a sequence of prompts
            for batch inference. See :class:`~vllm.inputs.PromptType`
            for more details about the format of each prompts.
        use_tqdm: Whether to use tqdm to display the progress bar.
        lora_request: LoRA request to use for generation, if any.
        prompt_adapter_request: Prompt Adapter request to use for
            generation, if any.

    Returns:
        A list of ``EmbeddingRequestOutput`` objects containing the
        embedding vectors in the same order as the input prompts.
    """
    if self.llm_engine.model_config.task != "embed":
        raise ValueError(
            "Embedding API is only enabled for `--task embed`")

```



```

        items = self.encode(prompts,
                             use_tqdm=use_tqdm,
                             lora_request=lora_request,
                             prompt_adapter_request=prompt_adapter_request)

    return [EmbeddingRequestOutput.from_base(item) for item in items]

def classify(
    self,
    prompts: Union[PromptType, Sequence[PromptType]],
    /,
    *,
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
) -> list[ClassificationRequestOutput]:
    """
    Generate class logits for each prompt.

    This class automatically batches the given prompts, considering
    the memory constraint. For the best performance, put all of your prompts
    into a single list and pass it to this method.

    Args:
        prompts: The prompts to the LLM. You may pass a sequence of prompts
            for batch inference. See :class:`~vllm.inputs.PromptType`
            for more details about the format of each prompts.
        use_tqdm: Whether to use tqdm to display the progress bar.
        lora_request: LoRA request to use for generation, if any.
        prompt_adapter_request: Prompt Adapter request to use for
            generation, if any.

```

Returns:

A list of ``ClassificationRequestOutput`` objects containing the embedding vectors in the same order as the input prompts.

"""

```
if self.llm_engine.model_config.task != "classify":
    raise ValueError(
        "Classification API is only enabled for `--task classify`")
```

```
items = self.encode(prompts,
                    use_tqdm=use_tqdm,
                    lora_request=lora_request,
                    prompt_adapter_request=prompt_adapter_request)
```

```
return [ClassificationRequestOutput.from_base(item) for item in items]
```

```
def _embedding_score(
    self,
    tokenizer: AnyTokenizer,
    text_1: list[Union[str, TextPrompt, TokensPrompt]],
    text_2: list[Union[str, TextPrompt, TokensPrompt]],
    truncate_prompt_tokens: Optional[int] = None,
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
) -> list[ScoringRequestOutput]:
```

```
encoded_output: list[PoolingRequestOutput] = self.encode(
    text_1 + text_2,
    use_tqdm=use_tqdm,
    lora_request=lora_request,
    prompt_adapter_request=prompt_adapter_request)
```

```

prompt_adapter_request=prompt_adapter_request)

    encoded_output_1: list[PoolingRequestOutput] = encoded_output[
        0:len(text_1)]
    encoded_output_2: list[PoolingRequestOutput] = encoded_output[
        len(text_1):]

    if len(encoded_output_1) == 1:
        encoded_output_1 = encoded_output_1 * len(encoded_output_2)

    scores: list[PoolingRequestOutput] = []

    scores = _cosine_similarity(tokenizer=tokenizer,
                               embed_1=encoded_output_1,
                               embed_2=encoded_output_2)

    items = self.engine_class.validate_outputs(scores,
                                                PoolingRequestOutput)

    return [ScoringRequestOutput.from_base(item) for item in items]

def _cross_encoding_score(
    self,
    tokenizer: AnyTokenizer,
    text_1: list[str],
    text_2: list[str],
    truncate_prompt_tokens: Optional[int] = None,
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
) -> list[ScoringRequestOutput]:

    if isinstance(tokenizer, MistralTokenizer):

```

```

        raise ValueError(
            "Score API is only enabled for `--task embed or score`")

    if len(text_1) == 1:
        text_1 = text_1 * len(text_2)

    input_pairs = [(t1, t2) for t1, t2 in zip(text_1, text_2)]

    pooling_params = PoolingParams()

    tokenization_kwargs: dict[str, Any] = {
    }
    if truncate_prompt_tokens is not None:
        tokenization_kwargs["truncation"] = True
        tokenization_kwargs["max_length"] = truncate_prompt_tokens

    parsed_prompts = []

    for q, t in input_pairs:
        prompt_inputs = tokenizer(text=q,
                                text_pair=t,
                                **tokenization_kwargs)
        engine_prompt = TokensPrompt(
            prompt_token_ids=prompt_inputs["input_ids"],
            token_type_ids=prompt_inputs.get("token_type_ids"))
        parsed_prompts.append(engine_prompt)

    self._validate_and_add_requests(
        prompts=parsed_prompts,
        params=pooling_params,
        lora_request=lora_request,
        prompt_adapter_request=prompt_adapter_request,

```

```

    )

    outputs = self._run_engine(use_tqdm=use_tqdm)
    items = self.engine_class.validate_outputs(outputs,
                                              PoolingRequestOutput)

    return [ScoringRequestOutput.from_base(item) for item in items]

def score(
    self,
    text_1: Union[SingletonPrompt, Sequence[SingletonPrompt]],
    text_2: Union[SingletonPrompt, Sequence[SingletonPrompt]],
    /,
    *,
    truncate_prompt_tokens: Optional[int] = None,
    use_tqdm: bool = True,
    lora_request: Optional[Union[list[LoRARequest], LoRARequest]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
) -> list[ScoringRequestOutput]:
    """Generate similarity scores for all pairs ``<text,text_pair>``.

    The inputs can be ``1 -> 1``, ``1 -> N`` or ``N -> N``.
    In the ``1 - N`` case the ``text_1`` sentence will be replicated ``N``
    times to pair with the ``text_2`` sentences.
    The input pairs are used to build a list of prompts for the
    cross encoder model. This class automatically batches the prompts,
    considering the memory constraint. For the best performance, put all
    of your texts into a single list and pass it to this method.

    Args:
        text_1: can be a single prompt or a list of prompts, in which
            case it has to have the same length as the ``text_2`` list

```

```
text_2: The texts to pair with the query to form the input
to the LLM. See :class:`~vllm.inputs.PromptType` for
more details about the format of each prompts.
use_tqdm: Whether to use tqdm to display the progress bar.
lora_request: LoRA request to use for generation, if any.
prompt_adapter_request: Prompt Adapter request to use for
generation, if any.
```

Returns:

```
A list of ``ScoringRequestOutput`` objects containing the
generated scores in the same order as the input prompts.
```

```
"""
```

```
runner_type = self.llm_engine.model_config.runner_type
if runner_type != "pooling":
    messages = ["LLM.score() is only supported for pooling models."]

    supported_runner_types = self.llm_engine.model_config \
        .supported_runner_types
    if "pooling" in supported_runner_types:
        messages.append(
            "Your model supports the 'pooling' runner, but is "
            f"currently initialized for the '{
runner_type}' runner. "
            "Please initialize vLLM using `--task embed`, "
            "`--task classify`, `--task score` etc.")

    raise ValueError(" ".join(messages))

if self.llm_engine.model_config.task not in ("embed", "score"):
    raise ValueError(
        "Score API is only enabled for `--task embed or --task score`")
```

```

# the tokenizer for models such as
# "cross-encoder/ms-marco-MiniLM-L-6-v2" doesn't support passing
# lists of tokens to the `text` and `text_pair` kwargs
tokenizer = self.llm_engine.get_tokenizer()

def ensure_str(prompt: SingletonPrompt):
    if isinstance(prompt, dict):
        if "multi_modal_data" in prompt:
            raise ValueError("Multi-modal prompt is not "
                              "supported for scoring")
        elif "prompt_token_ids" in prompt:
            prompt = tokenizer.decode(
                cast(TokensPrompt, prompt)["prompt_token_ids"])
        elif "prompt" in prompt:
            prompt = cast(TextPrompt, prompt)["prompt"]
    assert type(prompt) is str
    return prompt

if isinstance(text_1, (str, dict)):
    # Convert a single prompt to a list.
    text_1 = [text_1]
    input_text_1: list[str] = [ensure_str(t) for t in text_1]

if isinstance(text_2, (str, dict)):
    # Convert a single prompt to a list.
    text_2 = [text_2]
    input_text_2: list[str] = [ensure_str(t) for t in text_2]

_validate_score_input_lens(input_text_1, input_text_2)

if self.llm_engine.model_config.is_cross_encoder:

```

```

        return self._cross_encoding_score(tokenizer, input_text_1,
                                           input_text_2,
                                           truncate_prompt_tokens, use_tqdm,
                                           lora_request,
                                           prompt_adapter_request)

    else:
        return self._embedding_score(
            tokenizer,
            input_text_1, # type: ignore[arg-type]
            input_text_2, # type: ignore[arg-type]
            truncate_prompt_tokens,
            use_tqdm,
            lora_request,
            prompt_adapter_request)

    def start_profile(self) -> None:
        self.llm_engine.start_profile()

    def stop_profile(self) -> None:
        self.llm_engine.stop_profile()

    def reset_prefix_cache(self) -> bool:
        return self.llm_engine.reset_prefix_cache()

    def sleep(self, level: int = 1):
        """
        Put the engine to sleep. The engine should not process any requests.
        The caller should guarantee that no requests are being processed
        during the sleep period, before `wake_up` is called.

        :param level: The sleep level. Level 1 sleep will offload the model
            weights and discard the kv cache. The content of kv cache is

```


forgotten. Level 1 sleep is good for sleeping and waking up the engine to run the same model again. The model weights are backed up in CPU memory. Please make sure there's enough CPU memory to store the model weights. Level 2 sleep will discard both the model weights and the kv cache. The content of both the model weights and kv cache is forgotten. Level 2 sleep is good for sleeping and waking up the engine to run a different model or update the model, where previous model weights are not needed. It reduces CPU memory pressure.

```
"""
```

```
self.reset_prefix_cache()
self.llm_engine.sleep(level=level)
```

```
def wake_up(self):
```

```
"""
```

```
Wake up the engine from sleep mode. See the :meth:`sleep` method
for more details."""
```

```
self.llm_engine.wake_up()
```

```
# LEGACY
```

```
def _convert_v1_inputs(
```

```
self,
```

```
prompts: Optional[Union[str, list[str]]],
```

```
prompt_token_ids: Optional[Union[list[int], list[list[int]]]],
```

```
):
```

```
# skip_tokenizer_init is now checked in engine
```

```
if prompts is not None:
```

```
    prompts = [p["content"] for p in parse_and_batch_prompt(prompts)]
```

```
if prompt_token_ids is not None:
```

```
    prompt_token_ids = [
```

```
        ["content"] for p in parse_and_batch_prompt(prompt_token_ids)
```

```

        p["content"] for p in parse_and_batch_prompt(prompt_token_ids)
    ]

    num_requests = None
    if prompts is not None:
        num_requests = len(prompts)
    if prompt_token_ids is not None:
        if (num_requests is not None
            and num_requests != len(prompt_token_ids)):
            raise ValueError("The lengths of prompts and prompt_token_ids "
                              "must be the same.")

        num_requests = len(prompt_token_ids)
    if num_requests is None:
        raise ValueError("Either prompts or prompt_token_ids must be "
                          "provided.")

    parsed_prompts: list[PromptType] = []
    for i in range(num_requests):
        item: PromptType

        if prompts is not None:
            item = TextPrompt(prompt=prompts[i])
        elif prompt_token_ids is not None:
            item = TokensPrompt(prompt_token_ids=prompt_token_ids[i])
        else:
            raise AssertionError

        parsed_prompts.append(item)

    return parsed_prompts

```

```

def _validate_and_add_requests(
    self,
    prompts: Union[PromptType, Sequence[PromptType]],
    params: Union[SamplingParams, Sequence[SamplingParams], PoolingParams,
                  Sequence[PoolingParams]],
    lora_request: Optional[Union[Sequence[LoRARequest], LoRARequest]],
    prompt_adapter_request: Optional[PromptAdapterRequest],
    guided_options: Optional[GuidedDecodingRequest] = None,
    priority: Optional[list[int]] = None,
) -> None:
    if guided_options is not None:
        warnings.warn(
            "guided_options_request is deprecated, use "
            "SamplingParams.guided_decoding instead",
            DeprecationWarning,
            stacklevel=2,
        )

    if isinstance(prompts, (str, dict)):
        # Convert a single prompt to a list.
        prompts = [prompts]

    num_requests = len(prompts)
    if isinstance(params, list) and len(params) != num_requests:
        raise ValueError("The lengths of prompts and params "
                          "must be the same.")
    if isinstance(lora_request,
                  list) and len(lora_request) != num_requests:
        raise ValueError("The lengths of prompts and lora_request "
                          "must be the same.")

    for sp in params if isinstance(params, list) else (params, ):

```

```

        if isinstance(sp, SamplingParams):
            self._add_guided_params(sp, guided_options)

        # We only care about the final output
        sp.output_kind = RequestOutputKind.FINAL_ONLY

    # Add requests to the engine.
    for i, prompt in enumerate(prompts):
        self._add_request(
            prompt,
            params[i] if isinstance(params, Sequence) else params,
            lora_request=lora_request[i] if isinstance(
                lora_request, Sequence) else lora_request,
            prompt_adapter_request=prompt_adapter_request,
            priority=priority[i] if priority else 0,
        )

    def _add_request(
        self,
        prompt: PromptType,
        params: Union[SamplingParams, PoolingParams],
        lora_request: Optional[LoRARequest] = None,
        prompt_adapter_request: Optional[PromptAdapterRequest] = None,
        priority: int = 0,
    ) -> None:
        request_id = str(next(self.request_counter))
        self.llm_engine.add_request(
            request_id,
            prompt,
            params,
            lora_request=lora_request,
            prompt_adapter_request=prompt_adapter_request

```

```

        prompt_adapter_request=prompt_adapter_request,
        priority=priority,
    )

    def _add_guided_params(
        self,
        params: SamplingParams,
        guided_options: Optional[GuidedDecodingRequest] = None):
        if guided_options is None:
            return params

        if params.guided_decoding is not None:
            raise ValueError("Cannot set both guided_options_request and "
                             "params.guided_decoding.")

        params.guided_decoding = GuidedDecodingParams(
            json=guided_options.guided_json,
            regex=guided_options.guided_regex,
            choice=guided_options.guided_choice,
            grammar=guided_options.guided_grammar,
            json_object=guided_options.guided_json_object,
            backend=guided_options.guided_decoding_backend,
            whitespace_pattern=guided_options.guided_whitespace_pattern)
        return params

    def _run_engine(
        self, *, use_tqdm: bool
    ) -> list[Union[RequestOutput, PoolingRequestOutput]]:
        # Initialize tqdm.
        if use_tqdm:
            num_requests = self.llm_engine.get_num_unfinished_requests()
            pbar = tqdm(

```

```

        total=num_requests,
        desc="Processed prompts",
        dynamic_ncols=True,
        postfix=(f"est. speed input: {
0:.2f} toks/s, "
                f"output: {
0:.2f} toks/s"),
    )

# Run the engine.
outputs: list[Union[RequestOutput, PoolingRequestOutput]] = []
total_in_toks = 0
total_out_toks = 0
while self.llm_engine.has_unfinished_requests():
    step_outputs = self.llm_engine.step()
    for output in step_outputs:
        if output.finished:
            outputs.append(output)
            if use_tqdm:
                if isinstance(output, RequestOutput):
                    # Calculate tokens only for RequestOutput
                    n = len(output.outputs)
                    assert output.prompt_token_ids is not None
                    total_in_toks += len(output.prompt_token_ids) * n
                    in_spd = total_in_toks / pbar.format_dict["elapsed"]
                    total_out_toks += sum(
                        len(stp.token_ids) for stp in output.outputs)
                    out_spd = (total_out_toks /
                              pbar.format_dict["elapsed"])
                    pbar.postfix = (
                        f"est. speed input: {
in_spd:.2f} toks/s, "

```

```

                f"output: {
out_spd:.2f} toks/s")
                pbar.update(n)
            else:
                pbar.update(1)

    if use_tqdm:
        pbar.close()
    # Sort the outputs by request ID.
    # This is necessary because some requests may be finished earlier than
    # its previous requests.
    return sorted(outputs, key=lambda x: int(x.request_id))

```

1.2 OpenAI 兼容 API 服务器

vLLM 的第二个主要接口是通过其 OpenAI 兼容的 API 服务器。可以使用 `vllm serve` 命令启动此服务器。

```
vllm serve <model>
```

`vllm` CLI 的代码可以在如下 `main.py` 中找到：

```

# SPDX-License-Identifier: Apache-2.0

# The CLI entrypoint to vLLM.
import os
import signal
import sys

import vllm.entrypoints.cli.benchmark.main
import vllm.entrypoints.cli.openai
import vllm.entrypoints.cli.serve

```

```

import vllm.version
from vllm.logger import init_logger
from vllm.utils import FlexibleArgumentParser

logger = init_logger(__name__)

CMD_MODULES = [
    vllm.entrypoints.cli.openai,
    vllm.entrypoints.cli.serve,
    vllm.entrypoints.cli.benchmark.main,
]

def register_signal_handlers():

    def signal_handler(sig, frame):
        sys.exit(0)

    signal.signal(signal.SIGINT, signal_handler)
    signal.signal(signal.SIGTSTP, signal_handler)

def env_setup():
    # The safest multiprocessing method is `spawn`, as the default `fork` method
    # is not compatible with some accelerators. The default method will be
    # changing in future versions of Python, so we should use it explicitly when
    # possible.
    #
    # We only set it here in the CLI entrypoint, because changing to `spawn`
    # could break some existing code using vLLM as a library. `spawn` will cause
    # unexpected behavior if the code is not protected by
    # if __name__ == '__main__':

```



```

# if __name__ == "__main__":
#
# References:
# - https://docs.python.org/3/library/multiprocessing.html#contexts-and-start-methods
# - https://pytorch.org/docs/stable/notes/multiprocessing.html#cuda-in-multiprocessing
# - https://pytorch.org/docs/stable/multiprocessing.html#sharing-cuda-tensors
# - https://docs.habana.ai/en/Latest/PyTorch/Getting\_Started\_with\_PyTorch\_and\_Gaudi/Getting\_Started\_with\_PyTorch.html?highlight=multiprocessing#torch-multiprocessing-for-data-loaders
if "VLLM_WORKER_MULTIPROC_METHOD" not in os.environ:
    logger.debug("Setting VLLM_WORKER_MULTIPROC_METHOD to 'spawn'")
    os.environ["VLLM_WORKER_MULTIPROC_METHOD"] = "spawn"

def main():
    env_setup()

    parser = FlexibleArgumentParser(description="vLLM CLI")
    parser.add_argument('-v',
                        '--version',
                        action='version',
                        version=vllm.version.__version__)
    subparsers = parser.add_subparsers(required=False, dest="subparser")
    cmds = {
    }

    for cmd_module in CMD_MODULES:
        new_cmds = cmd_module.cmd_init()
        for cmd in new_cmds:
            cmd.subparser_init(subparsers).set_defaults(
                dispatch_function=cmd.cmd)
            cmds[cmd.name] = cmd

```

```

args = parser.parse_args()
if args.subparser in cmds:
    cmds[args.subparser].validate(args)

if hasattr(args, "dispatch_function"):
    args.dispatch_function(args)
else:
    parser.print_help()

if __name__ == "__main__":
    main()

```

有时你可能会看到直接使用 API 服务器入口点，而不是通过 `vllm` CLI 命令。例如：

```
python -m vllm.entrypoints.openai.api_server --model <model>
```

该代码可以在 `api_server.py` 中找到。

```

# SPDX-License-Identifier: Apache-2.0

import asyncio
import atexit
import gc
import importlib
import inspect
import multiprocessing
import os
import re
import signal
import socket
import tempfile

```

```
import uuid
from argparse import Namespace
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager
from functools import partial
from http import HTTPStatus
from typing import Annotated, Optional, Union

import uvloop
from fastapi import APIRouter, Depends, FastAPI, Form, HTTPException, Request
from fastapi.exceptions import RequestValidationError
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse, Response, StreamingResponse
from starlette.datastructures import State
from starlette.routing import Mount
from typing_extensions import assert_never

import vllm.envs as envs
from vllm.config import ModelConfig
from vllm.engine.arg_utils import AsyncEngineArgs
from vllm.engine.async_llm_engine import AsyncLLMEngine # type: ignore
from vllm.engine.multiprocessing.client import MQLLMEngineClient
from vllm.engine.multiprocessing.engine import run_mp_engine
from vllm.engine.protocol import EngineClient
from vllm.entrypoints.chat_utils import load_chat_template
from vllm.entrypoints.launcher import serve_http
from vllm.entrypoints.logger import RequestLogger
from vllm.entrypoints.openai.cli_args import import (make_arg_parser,
                                                    validate_parsed_serve_args)

# yapf conflicts with isort for this block
# yapf: disable
from vllm.entrypoints.openai.protocol import ChatCompletionRequest
```

```
from vllm.entrypoints.openai.protocol import (ChatCompletionRequest,
                                              ChatCompletionResponse,
                                              CompletionRequest,
                                              CompletionResponse,
                                              DetokenizeRequest,
                                              DetokenizeResponse,
                                              EmbeddingChatRequest,
                                              EmbeddingCompletionRequest,
                                              EmbeddingRequest,
                                              EmbeddingResponse,
                                              EmbeddingResponseData,
                                              ErrorResponse,
                                              LoadLoRAAdapterRequest,
                                              PoolingChatRequest,
                                              PoolingCompletionRequest,
                                              PoolingRequest, PoolingResponse,
                                              RerankRequest, RerankResponse,
                                              ScoreRequest, ScoreResponse,
                                              TokenizeRequest,
                                              TokenizeResponse,
                                              TranscriptionRequest,
                                              TranscriptionResponse,
                                              UnloadLoRAAdapterRequest)

from vllm.entrypoints.openai.reasoning_parsers import ReasoningParserManager
# yapf: enable
from vllm.entrypoints.openai.serving_chat import OpenAIServingChat
from vllm.entrypoints.openai.serving_completion import OpenAIServingCompletion
from vllm.entrypoints.openai.serving_embedding import OpenAIServingEmbedding
from vllm.entrypoints.openai.serving_engine import OpenAIServing
from vllm.entrypoints.openai.serving_models import (BaseModelPath,
                                                    OpenAIServingModels)
from vllm.entrypoints.openai.serving_pooling import OpenAIServingPooling
```

```

from vllm.entrypoints.openai.serving_score import ServingScores
from vllm.entrypoints.openai.serving_tokenization import (
    OpenAIServingTokenization)
from vllm.entrypoints.openai.serving_transcription import (
    OpenAIServingTranscription)
from vllm.entrypoints.openai.tool_parsers import ToolParserManager
from vllm.entrypoints.utils import load_aware_call, with_cancellation
from vllm.logger import init_logger
from vllm.usage.usage_lib import UsageContext
from vllm.utils import (FlexibleArgumentParser, get_open_zmq_ipc_path,
                        is_valid_ipv6_address, set_ulimit)
from vllm.version import __version__ as VLLM_VERSION

TIMEOUT_KEEP_ALIVE = 5 # seconds

prometheus_multiproc_dir: tempfile.TemporaryDirectory

# Cannot use __name__ (https://github.com/vllm-project/vllm/pull/4765)
logger = init_logger('vllm.entrypoints.openai.api_server')

_running_tasks: set[asyncio.Task] = set()

@asynccontextmanager
async def lifespan(app: FastAPI):
    try:
        if app.state.log_stats:
            engine_client: EngineClient = app.state.engine_client

        async def _force_log():
            while True:
                await asyncio.sleep(10.)
    
```

```

        await engine_client.do_log_stats()

        task = asyncio.create_task(_force_log())
        _running_tasks.add(task)
        task.add_done_callback(_running_tasks.remove)
    else:
        task = None

    # Mark the startup heap as static so that it's ignored by GC.
    # Reduces pause times of oldest generation collections.
    gc.collect()
    gc.freeze()
    try:
        yield
    finally:
        if task is not None:
            task.cancel()
    finally:
        # Ensure app state including engine ref is gc'd
        del app.state

@asynccontextmanager
async def build_async_engine_client(
    args: Namespace) -> AsyncIterator[EngineClient]:

    # Context manager to handle engine_client lifecycle
    # Ensures everything is shutdown and cleaned up on error/exit
    engine_args = AsyncEngineArgs.from_cli_args(args)

    async with build_async_engine_client_from_engine_args(
        engine_args, args.disable_frontend_multiprocessing) as engine:

```

```

        engine_args, args.disable_frontend_multiprocessing) as engine:
    yield engine

@asynccontextmanager
async def build_async_engine_client_from_engine_args(
    engine_args: AsyncEngineArgs,
    disable_frontend_multiprocessing: bool = False,
) -> AsyncIterator[EngineClient]:
    """
    Create EngineClient, either:
        - in-process using the AsyncLLMEngine Directly
        - multiprocess using AsyncLLMEngine RPC

    Returns the Client or None if the creation failed.
    """

    # Create the EngineConfig (determines if we can use V1).
    usage_context = UsageContext.OPENAI_API_SERVER
    vllm_config = engine_args.create_engine_config(usage_context=usage_context)

    # V1 AsyncLLM.
    if envs.VLLM_USE_V1:
        if disable_frontend_multiprocessing:
            logger.warning(
                "V1 is enabled, but got --disable-frontend-multiprocessing. "
                "To disable frontend multiprocessing, set VLLM_USE_V1=0.")

    from vllm.v1.engine.async_llm import AsyncLLM
    async_llm: Optional[AsyncLLM] = None
    try:
        async_llm = AsyncLLM.from_vllm_config(

```

```

        vllm_config=vllm_config,
        usage_context=usage_context,
        disable_log_requests=engine_args.disable_log_requests,
        disable_log_stats=engine_args.disable_log_stats)
    yield async_llm
finally:
    if async_llm:
        async_llm.shutdown()

# V0 AsyncLLM.
elif (MQLLMEngineClient.is_unsupported_config(vllm_config)
      or disable_frontend_multiprocessing):

    engine_client: Optional[EngineClient] = None
    try:
        engine_client = AsyncLLMEngine.from_vllm_config(
            vllm_config=vllm_config,
            usage_context=usage_context,
            disable_log_requests=engine_args.disable_log_requests,
            disable_log_stats=engine_args.disable_log_stats)
    yield engine_client
    finally:
        if engine_client and hasattr(engine_client, "shutdown"):
            engine_client.shutdown()

# V0MQLLMEngine.
else:
    if "PROMETHEUS_MULTIPROC_DIR" not in os.environ:
        # Make TemporaryDirectory for prometheus multiprocessing
        # Note: global TemporaryDirectory will be automatically
        # cleaned up upon exit.
        global prometheus_multiproc_dir

```



```

prometheus_multiproc_dir = tempfile.TemporaryDirectory()
os.environ[
    "PROMETHEUS_MULTIPROC_DIR"] = prometheus_multiproc_dir.name
else:
    logger.warning(
        "Found PROMETHEUS_MULTIPROC_DIR was set by user. "
        "This directory must be wiped between vLLM runs or "
        "you will find inaccurate metrics. Unset the variable "
        "and vLLM will properly handle cleanup.")

# Select random path for IPC.
ipc_path = get_open_zmq_ipc_path()
logger.debug("Multiprocessing frontend to use %s for IPC Path.",
             ipc_path)

# Start RPCServer in separate process (holds the LLMEngine).
# the current process might have CUDA context,
# so we need to spawn a new process
context = multiprocessing.get_context("spawn")

# The Process can raise an exception during startup, which may
# not actually result in an exitcode being reported. As a result
# we use a shared variable to communicate the information.
engine_alive = multiprocessing.Value('b', True, lock=False)
engine_process = context.Process(
    target=run_mp_engine,
    args=(vllm_config, UsageContext.OPENAI_API_SERVER, ipc_path,
          engine_args.disable_log_stats,
          engine_args.disable_log_requests, engine_alive))
engine_process.start()
engine_pid = engine_process.pid
assert engine_pid is not None, "Engine process failed to start."

```

```

assert engine_pid is not None, "Engine process failed to start."
logger.info("Started engine process with PID %d", engine_pid)

def _cleanup_ipc_path():
    socket_path = ipc_path.replace("ipc://", "")
    if os.path.exists(socket_path):
        os.remove(socket_path)

# Ensure we clean up the local IPC socket file on exit.
atexit.register(_cleanup_ipc_path)

# Build RPCClient, which conforms to EngineClient Protocol.
build_client = partial(MQLLMEngineClient, ipc_path, vllm_config,
                       engine_pid)
mq_engine_client = await asyncio.get_running_loop().run_in_executor(
    None, build_client)
try:
    while True:
        try:
            await mq_engine_client.setup()
            break
        except TimeoutError:
            if (not engine_process.is_alive()
                or not engine_alive.value):
                raise RuntimeError(
                    "Engine process failed to start. See stack "
                    "trace for the root cause.") from None

    yield mq_engine_client # type: ignore[misc]
finally:
    # Ensure rpc server process was terminated
    engine_process.terminate()

```

```

        # Close all open connections to the backend
        mq_engine_client.close()

        # Wait for engine process to join
        engine_process.join(4)
        if engine_process.exitcode is None:
            # Kill if taking longer than 5 seconds to stop
            engine_process.kill()

        # Lazy import for prometheus multiprocessing.
        # We need to set PROMETHEUS_MULTIPROC_DIR environment variable
        # before prometheus_client is imported.
        # See https://prometheus.github.io/client\_python/multiprocess/
        from prometheus_client import multiprocess
        multiprocess.mark_process_dead(engine_process.pid)

    async def validate_json_request(raw_request: Request):
        content_type = raw_request.headers.get("content-type", "").lower()
        media_type = content_type.split(";", maxsplit=1)[0]
        if media_type != "application/json":
            raise HTTPException(
                status_code=HTTPStatus.UNSUPPORTED_MEDIA_TYPE,
                detail="Unsupported Media Type: Only 'application/json' is allowed"
            )

router = APIRouter()

def mount_metrics(app: FastAPI):

```

```

# Lazy import for prometheus multiprocessing.
# We need to set PROMETHEUS_MULTIPROC_DIR environment variable
# before prometheus_client is imported.
# See https://prometheus.github.io/client_python/multiprocess/
from prometheus_client import (CollectorRegistry, make_asgi_app,
                               multiprocessing)

prometheus_multiproc_dir_path = os.getenv("PROMETHEUS_MULTIPROC_DIR", None)
if prometheus_multiproc_dir_path is not None:
    logger.debug("vLLM to use %s as PROMETHEUS_MULTIPROC_DIR",
                 prometheus_multiproc_dir_path)
    registry = CollectorRegistry()
    multiprocessing.MultiProcessCollector(registry)

    # Add prometheus asgi middleware to route /metrics requests
    metrics_route = Mount("/metrics", make_asgi_app(registry=registry))
else:
    # Add prometheus asgi middleware to route /metrics requests
    metrics_route = Mount("/metrics", make_asgi_app())

# Workaround for 307 Redirect for /metrics
metrics_route.path_regex = re.compile("^/metrics(?P<path>.*)$")
app.routes.append(metrics_route)

def base(request: Request) -> OpenAIServing:
    # Reuse the existing instance
    return tokenization(request)

def models(request: Request) -> OpenAIServingModels:
    return request.app.state.openai_serving_models

```

```
    return request.app.state.openai_serving_models

def chat(request: Request) -> Optional[OpenAIServingChat]:
    return request.app.state.openai_serving_chat

def completion(request: Request) -> Optional[OpenAIServingCompletion]:
    return request.app.state.openai_serving_completion

def pooling(request: Request) -> Optional[OpenAIServingPooling]:
    return request.app.state.openai_serving_pooling

def embedding(request: Request) -> Optional[OpenAIServingEmbedding]:
    return request.app.state.openai_serving_embedding

def score(request: Request) -> Optional[ServingScores]:
    return request.app.state.openai_serving_scores

def rerank(request: Request) -> Optional[ServingScores]:
    return request.app.state.openai_serving_scores

def tokenization(request: Request) -> OpenAIServingTokenization:
    return request.app.state.openai_serving_tokenization

def transcription(request: Request) -> OpenAIServingTranscription:
```

```

        return request.app.state.openai_serving_transcription

def engine_client(request: Request) -> EngineClient:
    return request.app.state.engine_client

@router.get("/health")
async def health(raw_request: Request) -> Response:
    """Health check."""
    await engine_client(raw_request).check_health()
    return Response(status_code=200)

@router.get("/load")
async def get_server_load_metrics(request: Request):
    # This endpoint returns the current server load metrics.
    # It tracks requests utilizing the GPU from the following routes:
    # - /v1/chat/completions
    # - /v1/completions
    # - /v1/audio/transcriptions
    # - /v1/embeddings
    # - /pooling
    # - /score
    # - /v1/score
    # - /rerank
    # - /v1/rerank
    # - /v2/rerank
    return JSONResponse(
        content={
            'server_load': request.app.state.server_load_metrics})

```

```

@router.api_route("/ping", methods=["GET", "POST"])
async def ping(raw_request: Request) -> Response:
    """Ping check. Endpoint required for SageMaker"""
    return await health(raw_request)

@router.post("/tokenize", dependencies=[Depends(validate_json_request)])
@with_cancellation
async def tokenize(request: TokenizeRequest, raw_request: Request):
    handler = tokenization(raw_request)

    generator = await handler.create_tokenize(request, raw_request)
    if isinstance(generator, ErrorResponse):
        return JSONResponse(content=generator.model_dump(),
                             status_code=generator.code)
    elif isinstance(generator, TokenizeResponse):
        return JSONResponse(content=generator.model_dump())

    assert_never(generator)

@router.post("/detokenize", dependencies=[Depends(validate_json_request)])
@with_cancellation
async def detokenize(request: DetokenizeRequest, raw_request: Request):
    handler = tokenization(raw_request)

    generator = await handler.create_detokenize(request, raw_request)
    if isinstance(generator, ErrorResponse):
        return JSONResponse(content=generator.model_dump(),
                             status_code=generator.code)
    elif isinstance(generator, DetokenizeResponse):

```

```

elif isinstance(generator, DetokenizerResponse):
    return JSONResponse(content=generator.model_dump())

assert_never(generator)

@router.get("/v1/models")
async def show_available_models(raw_request: Request):
    handler = models(raw_request)

    models_ = await handler.show_available_models()
    return JSONResponse(content=models_.model_dump())

@router.get("/version")
async def show_version():
    ver = {
        "version": VLLM_VERSION}
    return JSONResponse(content=ver)

@router.post("/v1/chat/completions",
             dependencies=[Depends(validate_json_request)])
@with_cancellation
@load_aware_call
async def create_chat_completion(request: ChatCompletionRequest,
                                raw_request: Request):
    handler = chat(raw_request)
    if handler is None:
        return base(raw_request).create_error_response(
            message="The model does not support Chat Completions API")

```



```

generator = await handler.create_chat_completion(request, raw_request)

if isinstance(generator, ErrorResponse):
    return JSONResponse(content=generator.model_dump(),
                        status_code=generator.code)

elif isinstance(generator, ChatCompletionResponse):
    return JSONResponse(content=generator.model_dump())

return StreamingResponse(content=generator, media_type="text/event-stream")

@router.post("/v1/completions", dependencies=[Depends(validate_json_request)])
@with_cancellation
@load_aware_call
async def create_completion(request: CompletionRequest, raw_request: Request):
    handler = completion(raw_request)
    if handler is None:
        return base(raw_request).create_error_response(
            message="The model does not support Completions API")

    generator = await handler.create_completion(request, raw_request)
    if isinstance(generator, ErrorResponse):
        return JSONResponse(content=generator.model_dump(),
                            status_code=generator.code)
    elif isinstance(generator, CompletionResponse):
        return JSONResponse(content=generator.model_dump())

    return StreamingResponse(content=generator, media_type="text/event-stream")

@router.post("/v1/embeddings", dependencies=[Depends(validate_json_request)])

```

```

@with_cancellation
@load_aware_call
async def create_embedding(request: EmbeddingRequest, raw_request: Request):
    handler = embedding(raw_request)
    if handler is None:
        fallback_handler = pooling(raw_request)
        if fallback_handler is None:
            return base(raw_request).create_error_response(
                message="The model does not support Embeddings API")

    logger.warning(
        "Embeddings API will become exclusive to embedding models "
        "in a future release. To return the hidden states directly, "
        "use the Pooling API (`/pooling`) instead.")

    res = await fallback_handler.create_pooling(request, raw_request)

    generator: Union[ErrorResponse, EmbeddingResponse]
    if isinstance(res, PoolingResponse):
        generator = EmbeddingResponse(
            id=res.id,
            object=res.object,
            created=res.created,
            model=res.model,
            data=[
                EmbeddingResponseData(
                    index=d.index,
                    embedding=d.data, # type: ignore
                ) for d in res.data
            ],
            usage=res.usage,
        )

```

```

        else:
            generator = res
    else:
        generator = await handler.create_embedding(request, raw_request)

    if isinstance(generator, ErrorResponse):
        return JSONResponse(content=generator.model_dump(),
                            status_code=generator.code)
    elif isinstance(generator, EmbeddingResponse):
        return JSONResponse(content=generator.model_dump())

    assert_never(generator)

@router.post("/pooling", dependencies=[Depends(validate_json_request)])
@with_cancellation
@load_aware_call
async def create_pooling(request: PoolingRequest, raw_request: Request):
    handler = pooling(raw_request)
    if handler is None:
        return base(raw_request).create_error_response(
            message="The model does not support Pooling API")

    generator = await handler.create_pooling(request, raw_request)
    if isinstance(generator, ErrorResponse):
        return JSONResponse(content=generator.model_dump(),
                            status_code=generator.code)
    elif isinstance(generator, PoolingResponse):
        return JSONResponse(content=generator.model_dump())

    assert_never(generator)

```

```

@router.post("/score", dependencies=[Depends(validate_json_request)])
@with_cancellation
@load_aware_call
async def create_score(request: ScoreRequest, raw_request: Request):
    handler = score(raw_request)
    if handler is None:
        return base(raw_request).create_error_response(
            message="The model does not support Score API")

    generator = await handler.create_score(request, raw_request)
    if isinstance(generator, ErrorResponse):
        return JSONResponse(content=generator.model_dump(),
                            status_code=generator.code)
    elif isinstance(generator, ScoreResponse):
        return JSONResponse(content=generator.model_dump())

    assert_never(generator)

@router.post("/v1/score", dependencies=[Depends(validate_json_request)])
@with_cancellation
@load_aware_call
async def create_score_v1(request: ScoreRequest, raw_request: Request):
    logger.warning(
        "To indicate that Score API is not part of standard OpenAI API, we "
        "have moved it to `/score`. Please update your client accordingly.")

    return await create_score(request, raw_request)

```

```

@router.post("/v1/audio/transcriptions")
@with_cancellation
@load_aware_call
async def create_transcriptions(request: Annotated[TranscriptionRequest,
                                                Form()],
                               raw_request: Request):
    handler = transcription(raw_request)
    if handler is None:
        return base(raw_request).create_error_response(
            message="The model does not support Transcriptions API")

    audio_data = await request.file.read()
    generator = await handler.create_transcription(audio_data, request,
                                                  raw_request)

    if isinstance(generator, ErrorResponse):
        return JSONResponse(content=generator.model_dump(),
                           status_code=generator.code)

    elif isinstance(generator, TranscriptionResponse):
        return JSONResponse(content=generator.model_dump())

    return StreamingResponse(content=generator, media_type="text/event-stream")

@router.post("/rerank", dependencies=[Depends(validate_json_request)])
@with_cancellation
@load_aware_call
async def do_rerank(request: RerankRequest, raw_request: Request):
    handler = rerank(raw_request)
    if handler is None:
        return base(raw_request).create_error_response(

```

```

        message="The model does not support Rerank (Score) API")
    generator = await handler.do_rerank(request, raw_request)
    if isinstance(generator, ErrorResponse):
        return JSONResponse(content=generator.model_dump(),
                             status_code=generator.code)
    elif isinstance(generator, RerankResponse):
        return JSONResponse(content=generator.model_dump())

    assert_never(generator)

@router.post("/v1/rerank", dependencies=[Depends(validate_json_request)])
@with_cancellation
async def do_rerank_v1(request: RerankRequest, raw_request: Request):
    logger.warning_once(
        "To indicate that the rerank API is not part of the standard OpenAI"
        " API, we have located it at `/rerank`. Please update your client "
        "accordingly. (Note: Conforms to JinaAI rerank API)")

    return await do_rerank(request, raw_request)

@router.post("/v2/rerank", dependencies=[Depends(validate_json_request)])
@with_cancellation
async def do_rerank_v2(request: RerankRequest, raw_request: Request):
    return await do_rerank(request, raw_request)

TASK_HANDLERS: dict[str, dict[str, tuple]] = {

    "generate": {

```

```

        "messages": (ChatCompletionRequest, create_chat_completion),
        "default": (CompletionRequest, create_completion),
    },
    "embed": {
        "messages": (EmbeddingChatRequest, create_embedding),
        "default": (EmbeddingCompletionRequest, create_embedding),
    },
    "score": {
        "default": (RerankRequest, do_rerank)
    },
    "rerank": {
        "default": (RerankRequest, do_rerank)
    },
    "reward": {
        "messages": (PoolingChatRequest, create_pooling),
        "default": (PoolingCompletionRequest, create_pooling),
    },
    "classify": {
        "messages": (PoolingChatRequest, create_pooling),
        "default": (PoolingCompletionRequest, create_pooling),
    },
}

if envs.VLLM_SERVER_DEV_MODE:

    @router.post("/reset_prefix_cache")

```

```

async def reset_prefix_cache(raw_request: Request):
    """
    Reset the prefix cache. Note that we currently do not check if the
    prefix cache is successfully reset in the API server.
    """
    logger.info("Resetting prefix cache...")
    await engine_client(raw_request).reset_prefix_cache()
    return Response(status_code=200)

@router.post("/sleep")
async def sleep(raw_request: Request):
    # get POST params
    level = raw_request.query_params.get("level", "1")
    logger.info("sleep the engine with level %s", level)
    await engine_client(raw_request).sleep(int(level))
    # FIXME: in v0 with frontend multiprocessing, the sleep command
    # is sent but does not finish yet when we return a response.
    return Response(status_code=200)

@router.post("/wake_up")
async def wake_up(raw_request: Request):
    logger.info("wake up the engine")
    await engine_client(raw_request).wake_up()
    # FIXME: in v0 with frontend multiprocessing, the wake-up command
    # is sent but does not finish yet when we return a response.
    return Response(status_code=200)

@router.get("/is_sleeping")
async def is_sleeping(raw_request: Request):
    logger.info("check whether the engine is sleeping")
    is_sleeping = await engine_client(raw_request).is_sleeping()
    return JSONResponse(content={

```



```

        "is_sleeping": is_sleeping})

@router.post("/invocations", dependencies=[Depends(validate_json_request)])
async def invocations(raw_request: Request):
    """
    For SageMaker, routes requests to other handlers based on model `task`.
    """
    body = await raw_request.json()
    task = raw_request.app.state.task

    if task not in TASK_HANDLERS:
        raise HTTPException(
            status_code=400,
            detail=f"Unsupported task: '{task}' for '/invocations'. "
            f"Expected one of {set(TASK_HANDLERS.keys())}")

    handler_config = TASK_HANDLERS[task]
    if "messages" in body:
        request_model, handler = handler_config["messages"]
    else:
        request_model, handler = handler_config["default"]

    # this is required since we lose the FastAPI automatic casting
    request = request_model.model_validate(body)
    return await handler(request, raw_request)

if envs.VLLM_TORCH_PROFILER_DIR:
    from torch.profiler import

```

```

logger.warning(
    "Torch Profiler is enabled in the API server. This should ONLY be "
    "used for local development!")

@router.post("/start_profile")
async def start_profile(raw_request: Request):
    logger.info("Starting profiler...")
    await engine_client(raw_request).start_profile()
    logger.info("Profiler started.")
    return Response(status_code=200)

@router.post("/stop_profile")
async def stop_profile(raw_request: Request):
    logger.info("Stopping profiler...")
    await engine_client(raw_request).stop_profile()
    logger.info("Profiler stopped.")
    return Response(status_code=200)

if envs.VLLM_ALLOW_RUNTIME_LORA_UPDATING:
    logger.warning(
        "LoRA dynamic loading & unloading is enabled in the API server. "
        "This should ONLY be used for local development!")

    @router.post("/v1/load_lora_adapter",
                 dependencies=[Depends(validate_json_request)])
    async def load_lora_adapter(request: LoadLoRAAdapterRequest,
                                raw_request: Request):
        handler = models(raw_request)
        response = await handler.load_lora_adapter(request)
        if isinstance(response, ErrorResponse):
            return JSONResponse(content=response.model_dump(),

```

```

        status_code=response.code)

    return Response(status_code=200, content=response)

@router.post("/v1/unload_lora_adapter",
              dependencies=[Depends(validate_json_request)])
async def unload_lora_adapter(request: UnloadLoRAAdapterRequest,
                              raw_request: Request):
    handler = models(raw_request)
    response = await handler.unload_lora_adapter(request)
    if isinstance(response, ErrorResponse):
        return JSONResponse(content=response.model_dump(),
                            status_code=response.code)

    return Response(status_code=200, content=response)

def build_app(args: Namespace) -> FastAPI:
    if args.disable_fastapi_docs:
        app = FastAPI(openapi_url=None,
                      docs_url=None,
                      redoc_url=None,
                      lifespan=lifespan)
    else:
        app = FastAPI(lifespan=lifespan)
    app.include_router(router)
    app.root_path = args.root_path

    mount_metrics(app)

    app.add_middleware(
        CORSMiddleware,

```

```

        allow_origins=args.allowed_origins,
        allow_credentials=args.allow_credentials,
        allow_methods=args.allowed_methods,
        allow_headers=args.allowed_headers,
    )

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(_, exc):
    err = ErrorResponse(message=str(exc),
                        type="BadRequestError",
                        code=HTTPStatus.BAD_REQUEST)

    return JSONResponse(err.model_dump(),
                        status_code=HTTPStatus.BAD_REQUEST)

if token := envs.VLLM_API_KEY or args.api_key:

    @app.middleware("http")
    async def authentication(request: Request, call_next):
        if request.method == "OPTIONS":
            return await call_next(request)
        url_path = request.url.path
        if app.root_path and url_path.startswith(app.root_path):
            url_path = url_path[len(app.root_path):]
        if not url_path.startswith("/v1"):
            return await call_next(request)
        if request.headers.get("Authorization") != "Bearer " + token:
            return JSONResponse(content={
                "error": "Unauthorized"},
                                status_code=401)
        return await call_next(request)

if args.enable_request_id_headers:

```

```

if args.enable_request_id_headers:
    logger.warning(
        "CAUTION: Enabling X-Request-Id headers in the API Server. "
        "This can harm performance at high QPS.")

@app.middleware("http")
async def add_request_id(request: Request, call_next):
    request_id = request.headers.get(
        "X-Request-Id") or uuid.uuid4().hex
    response = await call_next(request)
    response.headers["X-Request-Id"] = request_id
    return response

for middleware in args.middleware:
    module_path, object_name = middleware.rsplit(".", 1)
    imported = getattr(importlib.import_module(module_path), object_name)
    if inspect.isclass(imported):
        app.add_middleware(imported) # type: ignore[arg-type]
    elif inspect.iscoroutinefunction(imported):
        app.middleware("http")(imported)
    else:
        raise ValueError(f"Invalid middleware {
            middleware}. "
                           f"Must be a function or a class.")

return app

async def init_app_state(
    engine_client: EngineClient,
    model_config: ModelConfig,
    state: State,

```

```

    args: Namespace,
) -> None:
    if args.served_model_name is not None:
        served_model_names = args.served_model_name
    else:
        served_model_names = [args.model]

    if args.disable_log_requests:
        request_logger = None
    else:
        request_logger = RequestLogger(max_log_len=args.max_log_len)

    base_model_paths = [
        BaseModelPath(name=name, model_path=args.model)
        for name in served_model_names
    ]

    state.engine_client = engine_client
    state.log_stats = not args.disable_log_stats

    resolved_chat_template = load_chat_template(args.chat_template)
    if resolved_chat_template is not None:
        logger.info("Using supplied chat template:\n%s",
                    resolved_chat_template)

    state.openai_serving_models = OpenAIServingModels(
        engine_client=engine_client,
        model_config=model_config,
        base_model_paths=base_model_paths,
        lora_modules=args.lora_modules,
        prompt_adapters=args.prompt_adapters,
    )

```

```

await state.openai_serving_models.init_static_loras()
state.openai_serving_chat = OpenAIServingChat(
    engine_client,
    model_config,
    state.openai_serving_models,
    args.response_role,
    request_logger=request_logger,
    chat_template=resolved_chat_template,
    chat_template_content_format=args.chat_template_content_format,
    return_tokens_as_token_ids=args.return_tokens_as_token_ids,
    enable_auto_tools=args.enable_auto_tool_choice,
    tool_parser=args.tool_call_parser,
    enable_reasoning=args.enable_reasoning,
    reasoning_parser=args.reasoning_parser,
    enable_prompt_tokens_details=args.enable_prompt_tokens_details,
) if model_config.runner_type == "generate" else None
state.openai_serving_completion = OpenAIServingCompletion(
    engine_client,
    model_config,
    state.openai_serving_models,
    request_logger=request_logger,
    return_tokens_as_token_ids=args.return_tokens_as_token_ids,
) if model_config.runner_type == "generate" else None
state.openai_serving_pooling = OpenAIServingPooling(
    engine_client,
    model_config,
    state.openai_serving_models,
    request_logger=request_logger,
    chat_template=resolved_chat_template,
    chat_template_content_format=args.chat_template_content_format,
) if model_config.runner_type == "pooling" else None
state.openai_serving_embedding = OpenAIServingEmbedding(

```

```

state.openai_serving_embedding = OpenAIServingEmbedding(
    engine_client,
    model_config,
    state.openai_serving_models,
    request_logger=request_logger,
    chat_template=resolved_chat_template,
    chat_template_content_format=args.chat_template_content_format,
) if model_config.task == "embed" else None
state.openai_serving_scores = ServingScores(
    engine_client,
    model_config,
    state.openai_serving_models,
    request_logger=request_logger) if model_config.task in (
    "score", "embed", "pooling") else None
state.jinaai_serving_reranking = ServingScores(
    engine_client,
    model_config,
    state.openai_serving_models,
    request_logger=request_logger
) if model_config.task == "score" else None
state.openai_serving_tokenization = OpenAIServingTokenization(
    engine_client,
    model_config,
    state.openai_serving_models,
    request_logger=request_logger,
    chat_template=resolved_chat_template,
    chat_template_content_format=args.chat_template_content_format,
)
state.openai_serving_transcription = OpenAIServingTranscription(
    engine_client,
    model_config,
    state.openai_serving_models,

```



```

        request_logger=request_logger,
    ) if model_config.runner_type == "transcription" else None
    state.task = model_config.task

    state.enable_server_load_tracking = args.enable_server_load_tracking
    state.server_load_metrics = 0

def create_server_socket(addr: tuple[str, int]) -> socket.socket:
    family = socket.AF_INET
    if is_valid_ipv6_address(addr[0]):
        family = socket.AF_INET6

    sock = socket.socket(family=family, type=socket.SOCK_STREAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEPORT, 1)
    sock.bind(addr)

    return sock

async def run_server(args, **uvicorn_kwargs) -> None:
    logger.info("vLLM API server version %s", VLLM_VERSION)
    logger.info("args: %s", args)

    if args.tool_parser_plugin and len(args.tool_parser_plugin) > 3:
        ToolParserManager.import_tool_parser(args.tool_parser_plugin)

    valid_tool_parsers = ToolParserManager.tool_parsers.keys()
    if args.enable_auto_tool_choice \
        and args.tool_call_parser not in valid_tool_parsers:
        raise KeyError(f"invalid tool call parser: {

```

```

        args.tool_call_parser} "
            f"(chose from {
        { {
            ','.join(valid_tool_parsers)} }})")

valid_reasoning_parsers = ReasoningParserManager.reasoning_parsers.keys()
if args.enable_reasoning \
    and args.reasoning_parser not in valid_reasoning_parsers:
    raise KeyError(
        f"invalid reasoning parser: {
        args.reasoning_parser} "
        f"(chose from {
        { {
            ','.join(valid_reasoning_parsers)} }})")

# workaround to make sure that we bind the port before the engine is set up.
# This avoids race conditions with ray.
# see https://github.com/vllm-project/vllm/issues/8204
sock_addr = (args.host or "", args.port)
sock = create_server_socket(sock_addr)

# workaround to avoid footguns where uvicorn drops requests with too
# many concurrent requests active
set_ulimit()

def signal_handler(*_) -> None:
    # Interrupt server on sigterm while initializing
    raise KeyboardInterrupt("terminated")

signal.signal(signal.SIGTERM, signal_handler)

async with build_async_engine_client(args) as engine_client:

```

```

async with build_async_engine_client(args) as engine_client:
    app = build_app(args)

    model_config = await engine_client.get_model_config()
    await init_app_state(engine_client, model_config, app.state, args)

    def _listen_addr(a: str) -> str:
        if is_valid_ipv6_address(a):
            return '[' + a + ']'
        return a or "0.0.0.0"

    is_ssl = args.ssl_keyfile and args.ssl_certfile
    logger.info("Starting vLLM API server on http%s://%s:%d",
                "s" if is_ssl else "", _listen_addr(sock_addr[0]),
                sock_addr[1])

    shutdown_task = await serve_http(
        app,
        sock=sock,
        enable_ssl_refresh=args.enable_ssl_refresh,
        host=args.host,
        port=args.port,
        log_level=args.uvicorn_log_level,
        timeout_keep_alive=TIMEOUT_KEEP_ALIVE,
        ssl_keyfile=args.ssl_keyfile,
        ssl_certfile=args.ssl_certfile,
        ssl_ca_certs=args.ssl_ca_certs,
        ssl_cert_reqs=args.ssl_cert_reqs,
        **uvicorn_kwargs,
    )

```

NB: Await server shutdown only after the backend context is exited

```
await shutdown_task

sock.close()

if __name__ == "__main__":
    # NOTE(simon):
    # This section should be in sync with vllm/entrypoints/cli/main.py for CLI
    # entrypoints.
    parser = FlexibleArgumentParser(
        description="vLLM OpenAI-Compatible RESTful API server.")
    parser = make_arg_parser(parser)
    args = parser.parse_args()
    validate_parsed_serve_args(args)

    uvloop.run(run_server(args))
```

有关 API 服务器的更多详细信息，请参阅 **OpenAI 兼容服务器** 文档。

2. LLM 引擎

`LLMEngine` 和 `AsyncLLMEngine` 类是 vLLM 系统功能的核心，负责处理模型推理和异步请求处理。

2.1 LLM 引擎

`LLMEngine` 类是 vLLM 引擎的核心组件。它负责接收来自客户端的请求并生成模型的输出。`LLMEngine` 包括输入处理、模型执行（可能分布在多个主机和/或 GPU 上）、调度和输出处理。

- 输入处理：使用指定的分词器处理输入文本的分词。
- 调度：选择在每个步骤中处理哪些请求。
- 模型执行：管理语言模型的执行，包括跨多个 GPU 的分布式执行。
- 输出处理：处理模型生成的输出，将语言模型的 token ID 解码为人类可读的文本。

LLMEngine 的代码可以在如下 `llm_engine.py` 中找到:

```
# SPDX-License-Identifier: Apache-2.0

import copy
import time
from collections import Counter as collectionsCounter
from collections import deque
from contextlib import contextmanager
from dataclasses import dataclass
from functools import partial
from typing import (TYPE_CHECKING, Callable, ClassVar, Deque, Dict, Iterable,
                    List, Mapping, NamedTuple, Optional)
from typing import Sequence as GenericSequence
from typing import Set, Type, Union, cast, overload

import torch
from typing_extensions import TypeVar, deprecated

import vllm.envs as envs
from vllm.config import (DecodingConfig, LoRAConfig, ModelConfig,
                        ObservabilityConfig, ParallelConfig, SchedulerConfig,
                        VllmConfig)
from vllm.core.scheduler import ScheduledSequenceGroup, SchedulerOutputs
```

```
from vllm.engine.arg_utils import EngineArgs
from vllm.engine.metrics_types import StatLoggerBase, Stats
from vllm.engine.output_processor.interfaces import (
    SequenceGroupOutputProcessor)
from vllm.engine.output_processor.stop_checker import StopChecker
from vllm.engine.output_processor.util import create_output_by_sequence_group
from vllm.entrypoints.openai.logits_processors import (
    get_logits_processors as get_openai_logits_processors)
from vllm.executor.executor_base import ExecutorBase
from vllm.inputs import (INPUT_REGISTRY, InputRegistry, ProcessorInputs,
                        PromptType, SingletonInputsAdapter)
from vllm.inputs.parse import is_encoder_decoder_inputs, is_token_prompt
from vllm.inputs.preprocess import InputPreprocessor
from vllm.logger import init_logger
from vllm.logits_process import get_bad_words_logits_processors
from vllm.lora.request import LoRARequest
from vllm.model_executor.guided_decoding import (
    get_local_guided_decoding_logits_processor)
from vllm.model_executor.layers.sampler import SamplerOutput
from vllm.multimodal import MULTIMODAL_REGISTRY, MultiModalRegistry
from vllm.outputs import (PoolingRequestOutput, RequestOutput,
                        RequestOutputFactory)
from vllm.pooling_params import PoolingParams
from vllm.prompt_adapter.request import PromptAdapterRequest
from vllm.sampling_params import RequestOutputKind, SamplingParams
from vllm.sequence import (ExecuteModelRequest, ParallelSampleSequenceGroup,
                        PoolingSequenceGroupOutput, Sequence, SequenceGroup,
                        SequenceGroupBase, SequenceGroupMetadata,
                        SequenceGroupOutput, SequenceStatus)
from vllm.tracing import (SpanAttributes, SpanKind, extract_trace_context,
                        init_tracer)
from vllm.transformers_utils.detokenizer import Detokenizer
```

```

from vllm.transformers_utils.tokenizer import AnyTokenizer
from vllm.transformers_utils.tokenizer_group import (
    BaseTokenizerGroup, init_tokenizer_from_configs)
from vllm.usage.usage_lib import (UsageContext, is_usage_stats_enabled,
                                   usage_message)
from vllm.utils import (Counter, Device, deprecate_kwargs,
                        resolve_obj_by_qualname, weak_bind)
from vllm.version import __version__ as VLLM_VERSION
from vllm.worker.model_runner_base import InputProcessingError

logger = init_logger(__name__)
_LOCAL_LOGGING_INTERVAL_SEC = 5

_G = TypeVar("_G", bound=BaseTokenizerGroup, default=BaseTokenizerGroup)
_O = TypeVar("_O", RequestOutput, PoolingRequestOutput)

@dataclass
class SchedulerOutputState:
    """Caches the scheduler outputs for a virtual engine. Used for Multi-Step"""
    seq_group_metadata_list: Optional[List[SequenceGroupMetadata]] = None
    scheduler_outputs: Optional[SchedulerOutputs] = None
    allow_async_output_proc: bool = False
    last_output: Optional[SamplerOutput] = None

class OutputData(NamedTuple):
    outputs: List[SamplerOutput]
    seq_group_metadata_list: List[SequenceGroupMetadata]
    scheduler_outputs: SchedulerOutputs
    is_async: bool
    is_last_step: bool

```

```

# Indicates if this output is from the first step of the
# multi-step. When multi-step is disabled, this is always
# set to True.
# is_first_step_output is invalid when `outputs` has
# outputs from multiple steps.
is_first_step_output: Optional[bool]
skip: List[int]

```

```

class SchedulerContext:

```

```

    def __init__(self, multi_step_stream_outputs: bool = False):
        self.output_queue: Deque[OutputData] = deque()
        self.request_outputs: List[Union[RequestOutput,
                                         PoolingRequestOutput]] = []

        self.seq_group_metadata_list: Optional[
            List[SequenceGroupMetadata]] = None
        self.scheduler_outputs: Optional[SchedulerOutputs] = None

        self.multi_step_stream_outputs: bool = multi_step_stream_outputs

    def append_output(self, outputs: List[SamplerOutput],
                     seq_group_metadata_list: List[SequenceGroupMetadata],
                     scheduler_outputs: SchedulerOutputs, is_async: bool,
                     is_last_step: bool,
                     is_first_step_output: Optional[bool]):
        self.output_queue.append(
            OutputData(outputs=outputs,
                       seq_group_metadata_list=seq_group_metadata_list,
                       scheduler_outputs=scheduler_outputs,
                       is_async=is_async,
                       is_last_step=is_last_step,

```



```
is_last_step=is_last_step,  
is_first_step_output=is_first_step_output,  
skip=[]))
```

```
class LLMEngine:
```

```
    """An LLM engine that receives requests and generates texts.
```

This is the main class for the vLLM engine. It receives requests from clients and generates texts from the LLM. It includes a tokenizer, a language model (possibly distributed across multiple GPUs), and GPU memory space allocated for intermediate states (aka KV cache). This class utilizes iteration-level scheduling and efficient memory management to maximize the serving throughput.

The `:class:`~vllm.LLM`` class wraps this class for offline batched inference and the `:class:`AsyncLLMEngine`` class wraps this class for online serving.

The config arguments are derived from `:class:`~vllm.EngineArgs``. (See [:ref:`engine-args`](#))

Args:

`model_config`: The configuration related to the LLM model.

`cache_config`: The configuration related to the KV cache memory management.

`parallel_config`: The configuration related to distributed execution.

`scheduler_config`: The configuration related to the request scheduler.

`device_config`: The configuration related to the device.

`lora_config` (Optional): The configuration related to serving multi-LoRA.

`speculative_config` (Optional): The configuration related to speculative decoding.

`executor_class`: The model executor class for managing distributed

```

        execution.
        prompt_adapter_config (Optional): The configuration related to serving
            prompt adapters.
        log_stats: Whether to log statistics.
        usage_context: Specified entry point, used for usage info collection.
    """

    DO_VALIDATE_OUTPUT: ClassVar[bool] = False
    """A flag to toggle whether to validate the type of request output."""

    @classmethod
    @contextmanager
    def enable_output_validation(cls):
        cls.DO_VALIDATE_OUTPUT = True

        yield

        cls.DO_VALIDATE_OUTPUT = False

    @classmethod
    def validate_output(
        cls,
        output: object,
        output_type: Type[_O],
    ) -> _O:
        do_validate = cls.DO_VALIDATE_OUTPUT

        if ((TYPE_CHECKING or do_validate)
            and not isinstance(output, output_type)):
            raise TypeError(f"Expected output of type {
                output_type}, "
                            f"but found type {

```

```

        type(output))}")

    return cast(_O, output)

@classmethod
def validate_outputs(
    cls,
    outputs: GenericSequence[object],
    output_type: Type[_O],
) -> List[_O]:
    do_validate = cls.DO_VALIDATE_OUTPUT

    outputs_: List[_O]
    if TYPE_CHECKING or do_validate:
        outputs_ = []
        for output in outputs:
            if not isinstance(output, output_type):
                raise TypeError(f"Expected output of type {
                    output_type}, "
                                f"but found type {
                                    type(output)}")

            outputs_.append(output)
    else:
        outputs_ = outputs

    return outputs_

tokenizer: Optional[BaseTokenizerGroup]

def __init__(
    self

```

```

self,
vllm_config: VllmConfig,
executor_class: Type[ExecutorBase],
log_stats: bool,
usage_context: UsageContext = UsageContext.ENGINE_CONTEXT,
stat_loggers: Optional[Dict[str, StatLoggerBase]] = None,
input_registry: InputRegistry = INPUT_REGISTRY,
mm_registry: MultiModalRegistry = MULTIMODAL_REGISTRY,
use_cached_outputs: bool = False,
) -> None:
    if envs.VLLM_USE_V1:
        raise ValueError(
            "Using V0 LLMEngine, but envs.VLLM_USE_V1=True. "
            "This should not happen. As a workaround, try using "
            "LLMEngine.from_vllm_config(...) or explicitly set "
            "VLLM_USE_V1=0 or 1 and report this issue on Github.")

    self.vllm_config = vllm_config
    self.model_config = vllm_config.model_config
    self.cache_config = vllm_config.cache_config
    self.lora_config = vllm_config.lora_config
    self.parallel_config = vllm_config.parallel_config
    self.scheduler_config = vllm_config.scheduler_config
    self.device_config = vllm_config.device_config
    self.speculative_config = vllm_config.speculative_config # noqa
    self.load_config = vllm_config.load_config
    self.decoding_config = vllm_config.decoding_config or DecodingConfig( # noqa
    )
    self.prompt_adapter_config = vllm_config.prompt_adapter_config # noqa
    self.observability_config = vllm_config.observability_config or ObservabilityConf
ig( # noqa
    )

```

```

logger.info(
    "Initializing a V0 LLM engine (v%s) with config: %s, "
    "use_cached_outputs=%s, ",
    VLLM_VERSION,
    vllm_config,
    use_cached_outputs,
)

self.log_stats = log_stats
self.use_cached_outputs = use_cached_outputs

if not self.model_config.skip_tokenizer_init:
    self.tokenizer = self._init_tokenizer()
    self.detokenizer = Detokenizer(self.tokenizer)
    tokenizer_group = self.get_tokenizer_group()
else:
    self.tokenizer = None
    self.detokenizer = None
    tokenizer_group = None

# Ensure that the function doesn't contain a reference to self,
# to avoid engine GC issues
def get_tokenizer_for_seq(sequence: Sequence) -> AnyTokenizer:
    assert tokenizer_group, ("tokenizer_group cannot be None, "
                            "make sure skip_tokenizer_init is False")
    return tokenizer_group.get_lora_tokenizer(sequence.lora_request)

self.seq_counter = Counter()
self.generation_config_fields = (
    self.model_config.try_get_generation_config())

```

```

self.input_preprocessor = InputPreprocessor(self.model_config,
                                             self.tokenizer,
                                             mm_registry)

self.input_registry = input_registry
self.input_processor = input_registry.create_input_processor(
    self.model_config)

self.model_executor = executor_class(vllm_config=vllm_config, )

if self.model_config.runner_type != "pooling":
    self._initialize_kv_caches()

# If usage stat is enabled, collect relevant info.
if is_usage_stats_enabled():
    from vllm.model_executor.model_loader import (
        get_architecture_class_name)
    usage_message.report_usage(
        get_architecture_class_name(self.model_config),
        usage_context,
        extra_kvs={

            # Common configuration
            "dtype":
                str(self.model_config.dtype),
            "tensor_parallel_size":
                self.parallel_config.tensor_parallel_size,
            "block_size":
                self.cache_config.block_size,
            "gpu_memory_utilization":
                self.cache_config.gpu_memory_utilization,

```

```

        # Quantization
        "quantization":
            self.model_config.quantization,
        "kv_cache_dtype":
            str(self.cache_config.cache_dtype),

        # Feature flags
        "enable_lora":
            bool(self.lora_config),
        "enable_prompt_adapter":
            bool(self.prompt_adapter_config),
        "enable_prefix_caching":
            self.cache_config.enable_prefix_caching,
        "enforce_eager":
            self.model_config.enforce_eager,
        "disable_custom_all_reduce":
            self.parallel_config.disable_custom_all_reduce,
    })

    if self.tokenizer:
        # Ping the tokenizer to ensure liveness if it runs in a
        # different process.
        self.tokenizer.ping()

    self.cached_scheduler_outputs = [
        SchedulerOutputState()
        for _ in range(self.parallel_config.pipeline_parallel_size)
    ]

    self.scheduler_contexts = [
        SchedulerContext(multi_step_stream_outputs=self.scheduler_config.

```

```

        multi_step_stream_outputs)
    for _ in range(self.parallel_config.pipeline_parallel_size)
]

if self.model_config.use_async_output_proc:
    process_model_outputs = weak_bind(self._process_model_outputs)

    self.async_callbacks = [
        partial(process_model_outputs,
                ctx=self.scheduler_contexts[v_id])
        for v_id in range(self.parallel_config.pipeline_parallel_size)
    ]
else:
    self.async_callbacks = []

# Currently used by AsyncLLMEngine to ensure quick append
# of request outputs to asyncio queues
self.process_request_outputs_callback: Optional[Callable] = None

# Create the scheduler.
# NOTE: the cache_config here have been updated with the numbers of
# GPU and CPU blocks, which are profiled in the distributed executor.
if isinstance(self.vllm_config.scheduler_config.scheduler_cls, str):
    Scheduler = resolve_obj_by_qualname(
        self.vllm_config.scheduler_config.scheduler_cls)
else:
    Scheduler = self.vllm_config.scheduler_config.scheduler_cls
self.scheduler = [
    Scheduler(
        self.scheduler_config, self.cache_config, self.lora_config,
        self.parallel_config.pipeline_parallel_size,
        self.async_callbacks[v_id]

```



```

self.tracer = None
if self.observability_config.otlp_traces_endpoint:
    self.tracer = init_tracer(
        "vllm.llm_engine",
        self.observability_config.otlp_traces_endpoint)

# Create sequence output processor, e.g. for beam search or
# speculative decoding.
self.output_processor = (
    SequenceGroupOutputProcessor.create_output_processor(
        self.scheduler_config,
        self.detokenizer,
        self.scheduler,
        self.seq_counter,
        get_tokenizer_for_seq,
        stop_checker=StopChecker(
            self.scheduler_config.max_model_len,
            get_tokenizer_for_seq,
        ),
    ))

self.seq_id_to_seq_group: Dict[str, SequenceGroupBase] = {
}

# Flag to set when an input fails to process and the engine should run
# the next step without re-scheduling.
self._skip_scheduling_next_step = False

def _initialize_kv_caches(self) -> None:
    """Initialize the KV cache in the worker(s).

```

The workers will determine the number of blocks in both the GPU cache and the swap CPU cache.

```
"""
start = time.time()
num_gpu_blocks, num_cpu_blocks = (
    self.model_executor.determine_num_available_blocks())

if self.cache_config.num_gpu_blocks_override is not None:
    num_gpu_blocks_override = self.cache_config.num_gpu_blocks_override
    logger.info(
        "Overriding num_gpu_blocks=%d with "
        "num_gpu_blocks_override=%d", num_gpu_blocks,
        num_gpu_blocks_override)
    num_gpu_blocks = num_gpu_blocks_override

self.cache_config.num_gpu_blocks = num_gpu_blocks
self.cache_config.num_cpu_blocks = num_cpu_blocks

self.model_executor.initialize_cache(num_gpu_blocks, num_cpu_blocks)
elapsed = time.time() - start
logger.info(("init engine (profile, create kv cache, "
            "warmup model) took %.2f seconds"), elapsed)

@classmethod
def _get_executor_cls(cls,
                      engine_config: VllmConfig) -> Type[ExecutorBase]:
    # distributed_executor_backend must be set in VllmConfig.__post_init__
    distributed_executor_backend = (
        engine_config.parallel_config.distributed_executor_backend)
    # Initialize the cluster and specify the executor class.
    if isinstance(distributed_executor_backend, type):
        if not issubclass(distributed_executor_backend, ExecutorBase):
```

```

        raise TypeError(
            "distributed_executor_backend must be a subclass of "
            f"ExecutorBase. Got {
            distributed_executor_backend}."
        )
        executor_class = distributed_executor_backend
    elif distributed_executor_backend == "ray":
        from vllm.executor.ray_distributed_executor import (
            RayDistributedExecutor)
        executor_class = RayDistributedExecutor
    elif distributed_executor_backend == "mp":
        from vllm.executor.mp_distributed_executor import (
            MultiprocessingDistributedExecutor)
        assert not envs.VLLM_USE_RAY_SPMD_WORKER, (
            "multiprocessing distributed executor backend does not "
            "support VLLM_USE_RAY_SPMD_WORKER=1")
        executor_class = MultiprocessingDistributedExecutor
    elif distributed_executor_backend == "uni":
        # JAX-style, single-process, multi-device executor.
        from vllm.executor.uniproc_executor import UniProcExecutor
        executor_class = UniProcExecutor
    elif distributed_executor_backend == "external_launcher":
        # executor with external launcher
        from vllm.executor.uniproc_executor import ( # noqa
            ExecutorWithExternalLauncher)
        executor_class = ExecutorWithExternalLauncher
    else:
        raise ValueError("unrecognized distributed_executor_backend: "
            f"{
            distributed_executor_backend}")
    return executor_class

```

@classmethod

```

@classmethod
def from_vllm_config(
    cls,
    vllm_config: VllmConfig,
    usage_context: UsageContext = UsageContext.ENGINE_CONTEXT,
    stat_loggers: Optional[Dict[str, StatLoggerBase]] = None,
    disable_log_stats: bool = False,
) -> "LLMEngine":
    return cls(
        vllm_config=vllm_config,
        executor_class=cls._get_executor_cls(vllm_config),
        log_stats=(not disable_log_stats),
        usage_context=usage_context,
        stat_loggers=stat_loggers,
    )

@classmethod
def from_engine_args(
    cls,
    engine_args: EngineArgs,
    usage_context: UsageContext = UsageContext.ENGINE_CONTEXT,
    stat_loggers: Optional[Dict[str, StatLoggerBase]] = None,
) -> "LLMEngine":
    """Creates an LLM engine from the engine arguments."""
    # Create the engine configs.
    vllm_config = engine_args.create_engine_config(usage_context)

    engine_cls = cls
    if envs.VLLM_USE_V1:
        from vllm.v1.engine.llm_engine import LLMEngine as V1LLMEngine
        engine_cls = V1LLMEngine

```

```

        return engine_cls.from_vllm_config(
            vllm_config=vllm_config,
            usage_context=usage_context,
            stat_loggers=stat_loggers,
            disable_log_stats=engine_args.disable_log_stats,
        )

    def __reduce__(self):
        # This is to ensure that the LLMEngine is not referenced in
        # the closure used to initialize Ray worker actors
        raise RuntimeError("LLMEngine should not be pickled!")

    def __del__(self):
        # Shutdown model executor when engine is garbage collected
        # Use getattr since __init__ can fail before the field is set
        if model_executor := getattr(self, "model_executor", None):
            model_executor.shutdown()

    def get_tokenizer_group(
        self,
        group_type: Type[_G] = BaseTokenizerGroup,
    ) -> _G:
        tokenizer_group = self.tokenizer

        if tokenizer_group is None:
            raise ValueError("Unable to get tokenizer because "
                              "skip_tokenizer_init is True")
        if not isinstance(tokenizer_group, group_type):
            raise TypeError("Invalid type of tokenizer group. "
                              f"Expected type: {
                                group_type}, but "
                              f"found type: {

```

```

        type(tokenizer_group)}")

    return tokenizer_group

def get_tokenizer(
    self,
    lora_request: Optional[LoRARequest] = None,
) -> AnyTokenizer:
    return self.get_tokenizer_group().get_lora_tokenizer(lora_request)

def _init_tokenizer(self) -> BaseTokenizerGroup:
    return init_tokenizer_from_configs(
        model_config=self.model_config,
        scheduler_config=self.scheduler_config,
        parallel_config=self.parallel_config,
        lora_config=self.lora_config)

def _verify_args(self) -> None:
    self.model_config.verify_with_parallel_config(self.parallel_config)
    self.cache_config.verify_with_parallel_config(self.parallel_config)
    if self.lora_config:
        self.lora_config.verify_with_model_config(self.model_config)
        self.lora_config.verify_with_scheduler_config(
            self.scheduler_config)
    if self.prompt_adapter_config:
        self.prompt_adapter_config.verify_with_model_config(
            self.model_config)

def _add_processed_request(
    self,
    request_id: str,
    processed_inputs: ProcessorInputs,

```

```

        processed_inputs=processed_inputs,
        params: Union[SamplingParams, PoolingParams],
        arrival_time: float,
        lora_request: Optional[LoRARequest],
        prompt_adapter_request: Optional[PromptAdapterRequest],
        trace_headers: Optional[Mapping[str, str]] = None,
        priority: int = 0,
    ) -> Optional[SequenceGroup]:
        """Add a processed request to the engine's request pool.
        return the created sequence group.
        """
        if isinstance(params, SamplingParams) and params.n > 1:
            ParallelSampleSequenceGroup.add_request(
                request_id,
                self,
                params,
                processed_inputs=processed_inputs,
                arrival_time=arrival_time,
                lora_request=lora_request,
                trace_headers=trace_headers,
                prompt_adapter_request=prompt_adapter_request,
                priority=priority,
            )
            return None

        self._validate_model_inputs(processed_inputs, lora_request)
        # Create the sequences.
        block_size = self.cache_config.block_size
        seq_id = next(self.seq_counter)
        eos_token_id = self.input_preprocessor.get_eos_token_id(lora_request)

        if is_encoder_decoder_inputs(processed_inputs):

```



```
        decoder_inputs = processed_inputs["decoder"]
        encoder_inputs = processed_inputs["encoder"]
    else:
        decoder_inputs = processed_inputs
        encoder_inputs = None

    seq = Sequence(seq_id, decoder_inputs, block_size, eos_token_id,
                  lora_request, prompt_adapter_request)

    encoder_seq = (None if encoder_inputs is None else Sequence(
        seq_id, encoder_inputs, block_size, eos_token_id, lora_request,
        prompt_adapter_request))

    # Create a SequenceGroup based on SamplingParams or PoolingParams
    if isinstance(params, SamplingParams):
        seq_group = self._create_sequence_group_with_sampling(
            request_id,
            seq,
            params,
            arrival_time=arrival_time,
            lora_request=lora_request,
            trace_headers=trace_headers,
            prompt_adapter_request=prompt_adapter_request,
            encoder_seq=encoder_seq,
            priority=priority)
    elif isinstance(params, PoolingParams):
        seq_group = self._create_sequence_group_with_pooling(
            request_id,
            seq,
            params,
            arrival_time=arrival_time,
            lora_request=lora_request,
```

```

        prompt_adapter_request=prompt_adapter_request,
        encoder_seq=encoder_seq,
        priority=priority)
    else:
        raise ValueError(
            "Either SamplingParams or PoolingParams must be provided.")

    # Add the sequence group to the scheduler with least unfinished seqs.
    costs = [
        scheduler.get_num_unfinished_seq_groups()
        for scheduler in self.scheduler
    ]
    min_cost_scheduler = self.scheduler[costs.index(min(costs))]
    min_cost_scheduler.add_seq_group(seq_group)

    return seq_group

def stop_remote_worker_execution_loop(self) -> None:
    self.model_executor.stop_remote_worker_execution_loop()

@overload
def add_request(
    self,
    request_id: str,
    prompt: PromptType,
    params: Union[SamplingParams, PoolingParams],
    arrival_time: Optional[float] = None,
    lora_request: Optional[LoRARequest] = None,
    trace_headers: Optional[Mapping[str, str]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
    priority: int = 0,
) -> None:

```

```

...

@overload
@deprecated("'inputs' will be renamed to 'prompt'")
def add_request(
    self,
    request_id: str,
    *,
    inputs: PromptType,
    params: Union[SamplingParams, PoolingParams],
    arrival_time: Optional[float] = None,
    lora_request: Optional[LoRARequest] = None,
    trace_headers: Optional[Mapping[str, str]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
    priority: int = 0,
) -> None:
    ...

@deprecate_kwargs(
    "inputs",
    additional_message="Please use the 'prompt' parameter instead.",
)
def add_request(
    self,
    request_id: str,
    prompt: Optional[PromptType] = None,
    params: Optional[Union[SamplingParams, PoolingParams]] = None,
    arrival_time: Optional[float] = None,
    lora_request: Optional[LoRARequest] = None,
    trace_headers: Optional[Mapping[str, str]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
    priority: int = 0,
) -> None:
    ...

```

```

        priority: int = 0,
        *,
        inputs: Optional[PromptType] = None, # DEPRECATED
    ) -> None:

```

"""Add a request to the engine's request pool.

The request is added to the request pool and will be processed by the scheduler as `engine.step()` is called. The exact scheduling policy is determined by the scheduler.

Args:

`request_id`: The unique ID of the request.

`prompt`: The prompt to the LLM. See `:class:`~vllm.inputs.PromptType`` for more details about the format of each input.

`params`: Parameters for sampling or pooling.
 `:class:`~vllm.SamplingParams`` for text generation.
 `:class:`~vllm.PoolingParams`` for pooling.

`arrival_time`: The arrival time of the request. If None, we use the current monotonic time.

`lora_request`: The LoRA request to add.

`trace_headers`: OpenTelemetry trace headers.

`prompt_adapter_request`: The prompt adapter request to add.

`priority`: The priority of the request.
 Only applicable with priority scheduling.

Details:

- Set `arrival_time` to the current time if it is None.
- Set `prompt_token_ids` to the encoded prompt if it is None.
- Create `n` number of `:class:`~vllm.Sequence`` objects.
- Create a `:class:`~vllm.SequenceGroup`` object from the list of `:class:`~vllm.Sequence``.
- Add the `:class:`~vllm.SequenceGroup`` object to the scheduler.

Example:

```
>>> # initialize engine
>>> engine = LLMEngine.from_engine_args(engine_args)
>>> # set request arguments
>>> example_prompt = "Who is the president of the United States?"
>>> sampling_params = SamplingParams(temperature=0.0)
>>> request_id = 0
>>>
>>> # add the request to the engine
>>> engine.add_request(
>>>     str(request_id),
>>>     example_prompt,
>>>     SamplingParams(temperature=0.0))
>>> # continue the request processing
>>> ...
"""
if inputs is not None:
    prompt = inputs
assert prompt is not None and params is not None

if lora_request is not None and not self.lora_config:
    raise ValueError(f"Got lora_request {
        lora_request} but LoRA is "
                    "not enabled!")

if priority != 0 and not self.scheduler_config.policy == "priority":
    raise ValueError(f"Got priority {
        priority} but "
                    "Priority scheduling is not enabled.")

if isinstance(params, SamplingParams) \
```

```

        and (params.guided_decoding or params.logits_processors) \
        and self.scheduler_config.num_scheduler_steps > 1:
            raise ValueError(
                "Guided decoding and logits processors are not supported "
                "in multi-step decoding")

    if arrival_time is None:
        arrival_time = time.time()

    if self.tokenizer is not None:
        self._validate_token_prompt(
            prompt,
            tokenizer=self.get_tokenizer(lora_request=lora_request))

    preprocessed_inputs = self.input_preprocessor.preprocess(
        prompt,
        request_id=request_id,
        lora_request=lora_request,
        prompt_adapter_request=prompt_adapter_request,
    )
    processed_inputs = self.input_processor(preprocessed_inputs)

    self._add_processed_request(
        request_id=request_id,
        processed_inputs=processed_inputs,
        params=params,
        arrival_time=arrival_time,
        lora_request=lora_request,
        prompt_adapter_request=prompt_adapter_request,
        trace_headers=trace_headers,
        priority=priority,

```

```

def _validate_token_prompt(self, prompt: PromptType,
                           tokenizer: AnyTokenizer):
    # Guard against out-of-vocab tokens.
    # For some tokenizers, tokenizer.decode will happily return empty text
    # for token ids that are out of vocab, and we don't detect token ids
    # that are greater than the max token id before running the model.
    # However, these token ids will later crash a cuda kernel at runtime
    # with an index out of bounds error. This will crash the entire engine.
    # This needs to happen before multimodal input pre-processing, which
    # may add dummy <image> tokens that aren't part of the tokenizer's
    # vocabulary.
    if is_token_prompt(prompt):
        prompt_ids = prompt["prompt_token_ids"]
        if len(prompt_ids) == 0:
            # Empty prompt check is handled later
            return
        max_input_id = max(prompt_ids)
        if max_input_id > tokenizer.max_token_id:
            raise ValueError(
                "Token id {} is out of vocabulary".format(max_input_id))

def _create_sequence_group_with_sampling(
    self,
    request_id: str,
    seq: Sequence,
    sampling_params: SamplingParams,
    arrival_time: float,
    lora_request: Optional[LoRARequest],
    trace_headers: Optional[Mapping[str, str]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,

```

```

encoder_seq: Optional[Sequence] = None,
priority: int = 0,
) -> SequenceGroup:
    """Creates a SequenceGroup with SamplingParams."""
    max_logprobs = self.get_model_config().max_logprobs
    if (sampling_params.logprobs
        and sampling_params.logprobs > max_logprobs) or (
        sampling_params.prompt_logprobs
        and sampling_params.prompt_logprobs > max_logprobs):
        raise ValueError(f"Cannot request more than "
                        f"{
max_logprobs} logprobs.")

    sampling_params = self._build_logits_processors(
        sampling_params, lora_request)

    # Defensive copy of SamplingParams, which are used by the sampler,
    # this doesn't deep-copy LogitsProcessor objects
    sampling_params = sampling_params.clone()

    sampling_params.update_from_generation_config(
        self.generation_config_fields, seq.eos_token_id)

    # Create the sequence group.
    draft_size = 1
    if self.vllm_config.speculative_config is not None:
        draft_size = \
            self.vllm_config.speculative_config.num_speculative_tokens + 1
    seq_group = SequenceGroup(
        request_id=request_id,
        seqs=[seq],
        arrival_time=arrival_time,

```



```

        sampling_params=sampling_params,
        lora_request=lora_request,
        trace_headers=trace_headers,
        prompt_adapter_request=prompt_adapter_request,
        encoder_seq=encoder_seq,
        priority=priority,
        draft_size=draft_size)

    return seq_group

def _create_sequence_group_with_pooling(
    self,
    request_id: str,
    seq: Sequence,
    pooling_params: PoolingParams,
    arrival_time: float,
    lora_request: Optional[LoRARequest],
    prompt_adapter_request: Optional[PromptAdapterRequest],
    encoder_seq: Optional[Sequence] = None,
    priority: int = 0,
) -> SequenceGroup:
    """Creates a SequenceGroup with PoolingParams."""
    # Defensive copy of PoolingParams, which are used by the pooler
    pooling_params = pooling_params.clone()
    # Create the sequence group.
    seq_group = SequenceGroup(
        request_id=request_id,
        seqs=[seq],
        arrival_time=arrival_time,
        lora_request=lora_request,
        pooling_params=pooling_params,
        prompt_adapter_request=prompt_adapter_request,
    )

```

```

        prompt_adapter_request=prompt_adapter_request,
        encoder_seq=encoder_seq,
        priority=priority)
    return seq_group

def abort_request(self, request_id: Union[str, Iterable[str]]) -> None:
    """Aborts a request(s) with the given ID.

    Args:
        request_id: The ID(s) of the request to abort.

    Details:
        - Refer to the
          :meth:`~vllm.core.scheduler.Scheduler.abort_seq_group`
          from class :class:`~vllm.core.scheduler.Scheduler`.

    Example:
        >>> # initialize engine and add a request with request_id
        >>> request_id = str(0)
        >>> # abort the request
        >>> engine.abort_request(request_id)
    """
    for scheduler in self.scheduler:
        scheduler.abort_seq_group(
            request_id, seq_id_to_seq_group=self.seq_id_to_seq_group)

def get_model_config(self) -> ModelConfig:
    """Gets the model configuration."""
    return self.model_config

def get_parallel_config(self) -> ParallelConfig:
    """Gets the parallel configuration."""

```

```

        return self.parallel_config

    def get_decoding_config(self) -> DecodingConfig:
        """Gets the decoding configuration."""
        return self.decoding_config

    def get_scheduler_config(self) -> SchedulerConfig:
        """Gets the scheduler configuration."""
        return self.scheduler_config

    def get_lora_config(self) -> LoRAConfig:
        """Gets the LoRA configuration."""
        return self.lora_config

    def get_num_unfinished_requests(self) -> int:
        """Gets the number of unfinished requests."""
        return sum(scheduler.get_num_unfinished_seq_groups()
                    for scheduler in self.scheduler)

    def has_unfinished_requests(self) -> bool:
        """Returns True if there are unfinished requests."""
        return any(scheduler.has_unfinished_seqs()
                    for scheduler in self.scheduler)

    def has_unfinished_requests_for_virtual_engine(
        self, virtual_engine: int) -> bool:
        """
        Returns True if there are unfinished requests for the virtual engine.
        """
        return self.scheduler[virtual_engine].has_unfinished_seqs()

    def reset_prefix_cache(self) -> bool:

```

```

        """Reset prefix cache for all devices."""

        success = True
        for scheduler in self.scheduler:
            success = success and scheduler.reset_prefix_cache()
        return success

    @staticmethod
    def _process_sequence_group_outputs(
        seq_group: SequenceGroup,
        outputs: List[PoolingSequenceGroupOutput],
    ) -> None:
        seq_group.pooled_data = outputs[0].data

        for seq in seq_group.get_seqs():
            seq.status = SequenceStatus.FINISHED_STOPPED

        return

    def _update_num_computed_tokens_for_multi_step_prefill(
        self, seq_group: SequenceGroup,
        seq_group_meta: SequenceGroupMetadata,
        is_first_step_output: Optional[bool]):
        """
        This function updates num_computed_tokens for prompt sequences
        when Multi-Step is enabled.

        seq_group: SequenceGroup to update the num_computed_tokens for.
        seq_group_meta: Metadata of the given SequenceGroup.
        is_first_step_output: Optional[bool] -
            When available, is_first_step_output indicates if the appended
            output token is the output of the first step in multi-step

```

```
        output token is the output of the first-step in multi-step.
        A value of None indicates that outputs from all steps in
        in multi-step are submitted in a single burst.
        """
```

```
    assert self.scheduler_config.is_multi_step
```

```
    if not seq_group_meta.is_prompt:
        # num_computed_token updates for multi-step decodes happen after
        # the tokens are appended to the sequence.
        return
```

```
    do_update: bool = False
    if self.scheduler_config.chunked_prefill_enabled:
        # In multi-step + chunked-prefill case, the prompt sequences
        # that are scheduled are fully processed in the first step.
        do_update = is_first_step_output is None or is_first_step_output
    else:
        # Normal multi-step decoding case. In this case prompt-sequences
        # are actually single-stepped. Always update in this case.
        assert seq_group.state.num_steps == 1
        do_update = True
```

```
    if do_update:
        seq_group.update_num_computed_tokens(
            seq_group_meta.token_chunk_size)
```

```
def _process_model_outputs(self,
                           ctx: SchedulerContext,
                           request_id: Optional[str] = None) -> None:
    """Apply the model output to the sequences in the scheduled seq groups
    and return responses.
```

```

ctx: The virtual engine context to work on
request_id: If provided, then only this request is going to be processed
"""

now = time.time()

if len(ctx.output_queue) == 0:
    return None

# Get pending async postprocessor
if request_id:
    # When we process only one request, no pop is required
    # (since later we will process all of the rest)
    (outputs, seq_group_metadata_list, scheduler_outputs, is_async,
     is_last_step, is_first_step_output, skip) = ctx.output_queue[0]
else:
    (outputs, seq_group_metadata_list, scheduler_outputs, is_async,
     is_last_step, is_first_step_output,
     skip) = ctx.output_queue.popleft()

# Sanity check
assert len(seq_group_metadata_list) == len(
    scheduler_outputs.scheduled_seq_groups)

has_multiple_outputs: bool = len(outputs) > 1
outputs_by_sequence_group: List[List[SequenceGroupOutput]]
if has_multiple_outputs:
    assert self.scheduler_config.is_multi_step or \
           self.speculative_config
    # Organize outputs by [step][sequence group] instead of
    # [sequence group][step].

```

```

if self.scheduler_config.is_multi_step:
    outputs_by_sequence_group = create_output_by_sequence_group(
        outputs, len(seq_group_metadata_list))
elif self.speculative_config:
    # Decodes are multi-steps while prefills are not, outputting at
    # most 1 token. Separate them so that we can trigger chunk
    # processing without having to pad or copy over prompts K times
    # to match decodes structure (costly with prompt_logprobs).
    num_prefills = sum(sg.is_prompt
                        for sg in seq_group_metadata_list)
    prefills, decodes = outputs[:num_prefills], outputs[
        num_prefills:]
    outputs_by_sequence_group = create_output_by_sequence_group(
        decodes,
        num_seq_groups=len(seq_group_metadata_list) - num_prefills)
    outputs_by_sequence_group = [p.outputs for p in prefills
                                ] + outputs_by_sequence_group
    # We have outputs for multiple steps submitted in a single burst,
    # so invalidate is_first_step_output.
    is_first_step_output = None
else:
    outputs_by_sequence_group = outputs

# Determine the requests we need to operate on
if request_id:
    indices = []
    for i, seq_group_meta in enumerate(seq_group_metadata_list):
        if seq_group_meta.request_id == request_id:
            assert i not in skip # Cannot be called twice
            indices.append(i)
            break

```

```

        # If the request_id was not found, then it means that
        # this is a new request that has no pending async
        # postprocessor
        if not indices:
            return
    else:
        indices = range(len(seq_group_metadata_list)) # type: ignore

    finished_before: List[int] = []
    finished_now: List[int] = []
    for i in indices:
        if i in skip:
            continue

        seq_group_meta = seq_group_metadata_list[i]
        scheduled_seq_group = scheduler_outputs.scheduled_seq_groups[i]

        seq_group: SequenceGroup = scheduled_seq_group.seq_group

        if seq_group.is_finished():
            finished_before.append(i)
            continue

        output: List[SequenceGroupOutput]
        if has_multiple_outputs:
            output = outputs_by_sequence_group[i]
        else:
            output = [outputs_by_sequence_group[0][i]]

        if not is_async:
            if self.scheduler_config.is_multi_step:

```



```

        # Updates happen only if the sequence is prefill
        self._update_num_computed_tokens_for_multi_step_prefill(
            seq_group, seq_group_meta, is_first_step_output)
    else:
        seq_group.update_num_computed_tokens(
            seq_group_meta.token_chunk_size or 0)

    if outputs:
        for o in outputs:
            if (isinstance(o, SamplerOutput)
                and seq_group.metrics is not None):
                if seq_group.metrics.model_forward_time is not None:
                    seq_group.metrics.model_forward_time += (
                        o.model_forward_time or 0)
                else:
                    seq_group.metrics.model_forward_time = (
                        o.model_forward_time)
                if seq_group.metrics.model_execute_time is not None:
                    seq_group.metrics.model_execute_time += (
                        o.model_execute_time or 0)
                else:
                    seq_group.metrics.model_execute_time = (
                        o.model_execute_time)

    if self.model_config.runner_type == "pooling":
        self._process_sequence_group_outputs(seq_group, output)
    else:
        self.output_processor.process_prompt_logprob(seq_group, output)
        if seq_group_meta.do_sample:
            self.output_processor.process_outputs(
                seq_group, output, is_async)

```

```

        if seq_group.is_finished():
            finished_now.append(i)

# Generate outputs for the requests that finished this iteration
    for i in finished_now:
        scheduled_seq_group = scheduler_outputs.scheduled_seq_groups[i]

        seq_group = scheduled_seq_group.seq_group
        seq_group.maybe_set_first_token_time(now)
        if not seq_group.is_prefill():
            seq_group.set_last_token_time(now)
        request_output = RequestOutputFactory.create(
            seq_group,
            self.seq_id_to_seq_group,
            use_cache=self.use_cached_outputs)
        if request_output:
            ctx.request_outputs.append(request_output)

# When we process a single request, we skip it for the next time,
# and invoke the request output callback (if there was final output)
    if request_id:
        assert len(indices) == 1
        skip.append(indices[0])

        if (finished_now
            and self.process_request_outputs_callback is not None):
            self.process_request_outputs_callback(ctx.request_outputs)
            ctx.request_outputs.clear()
        return

# Free currently finished requests
    if finished_now:

```

```

        if finished_now:
            for scheduler in self.scheduler:
                scheduler.free_finished_seq_groups()

        # For multi-step without streaming, don't create outputs each iteration
        if not is_last_step and not ctx.multi_step_stream_outputs:
            # Immediately process request outputs here (if callback is given)
            if (finished_now
                and self.process_request_outputs_callback is not None):
                self.process_request_outputs_callback(ctx.request_outputs)
                ctx.request_outputs.clear()
            return

        # Create the outputs
        for i in indices:
            if i in skip or i in finished_before or i in finished_now:
                continue # Avoids double processing

            scheduled_seq_group = scheduler_outputs.scheduled_seq_groups[i]

            seq_group = scheduled_seq_group.seq_group
            seq_group.maybe_set_first_token_time(now)
            if not seq_group.is_prefill():
                seq_group.set_last_token_time(now)
            request_output = RequestOutputFactory.create(
                seq_group,
                self.seq_id_to_seq_group,
                use_cache=self.use_cached_outputs)
            if request_output:
                ctx.request_outputs.append(request_output)

        # For multi-step with streaming, create outputs each iteration

```

```

if not is_last_step and ctx.multi_step_stream_outputs:
    # Immediately process request outputs here (if callback is given)
    if self.process_request_outputs_callback is not None:
        self.process_request_outputs_callback(ctx.request_outputs)
        ctx.request_outputs.clear()
    return

for seq_group in scheduler_outputs.ignored_seq_groups:
    params = seq_group.sampling_params
    if params is not None and params.output_kind == (
        RequestOutputKind.DELTA) and not seq_group.is_finished():
        continue

    request_output = RequestOutputFactory.create(
        seq_group,
        self.seq_id_to_seq_group,
        use_cache=self.use_cached_outputs,
    )
    if request_output:
        ctx.request_outputs.append(request_output)

    # Immediately process request outputs here (if callback is given)
    if (ctx.request_outputs
        and self.process_request_outputs_callback is not None):
        self.process_request_outputs_callback(ctx.request_outputs)
        ctx.request_outputs.clear()

    # For async case, we need to record the stats here.
    # For non-async case, the stats are done in the
    # LLMEngine/AsyncLLMEngine directly
    if is_async:
        # Log stats.

```

```

        self.do_log_stats(scheduler_outputs, outputs, finished_before,
                          skip)

        # Tracing
        self.do_tracing(scheduler_outputs, finished_before)

    return None

def _advance_to_next_step(
    self, output: List[SamplerOutput],
    seq_group_metadata_list: List[SequenceGroupMetadata],
    scheduled_seq_groups: List[ScheduledSequenceGroup]) -> None:
    """Given model output from a single run, append the tokens to the
    sequences. This is normally done inside output processor, but it is
    required if the worker is to perform async forward pass to next step.
    """
    for seq_group_metadata, sequence_group_outputs, scheduled_seq_group in \
        zip(seq_group_metadata_list, output, scheduled_seq_groups):
        seq_group = scheduled_seq_group.seq_group

        if seq_group.is_finished():
            continue

        if self.scheduler_config.is_multi_step:
            # Updates happen only if the sequence is prefill
            self._update_num_computed_tokens_for_multi_step_prefill(
                seq_group, seq_group_metadata,
                seq_group.state.num_steps == 1)
        else:
            token_chunk_size = (seq_group_metadata.token_chunk_size
                                if seq_group_metadata.token_chunk_size
                                is not None else 0)

```

```

        seq_group.update_num_computed_tokens(token_chunk_size)

    if seq_group_metadata.do_sample:
        assert len(sequence_group_outputs.samples) == 1, (
            "Async output processor expects a single sample"
            " (i.e sampling_params.n == 1)")
        sample = sequence_group_outputs.samples[0]

        assert len(seq_group.seqs) == 1
        seq = seq_group.seqs[0]

        if self.scheduler_config.is_multi_step:
            is_prefill_append = seq.data.get_num_uncomputed_tokens(
            ) == 0
            seq.append_token_id(sample.output_token, sample.logprobs)
            if not is_prefill_append:
                seq_group.update_num_computed_tokens(1)
        else:
            seq.append_token_id(sample.output_token, sample.logprobs)

def step(self) -> List[Union[RequestOutput, PoolingRequestOutput]]:
    """Performs one decoding iteration and returns newly generated results.

    .. figure:: https://i.imgur.com/sv2HssD.png
       :alt: Overview of the step function
       :align: center

       Overview of the step function.

    Details:
    - Step 1: Schedules the sequences to be executed in the next

```

iteration and the token blocks to be swapped in/out/copy.

- Depending on the scheduling policy, sequences may be `preempted/reordered`.
 - A Sequence Group (SG) refer to a group of sequences that are generated from the same prompt.
-
- Step 2: Calls the distributed executor to execute the model.
 - Step 3: Processes the model output. This mainly includes:
 - Decodes the relevant outputs.
 - Updates the scheduled sequence groups with model outputs based on its `sampling parameters` (`use\_beam\_search` or not).
 - Frees the finished sequence groups.
-
- Finally, it creates and returns the newly generated results.

Example:

```
>>> # Please see the example/ folder for more detailed examples.
>>>
>>> # initialize engine and request arguments
>>> engine = LLMEngine.from_engine_args(engine_args)
>>> example_inputs = [(0, "What is LLM?",
>>>     SamplingParams(temperature=0.0))]
>>>
>>> # Start the engine with an event loop
>>> while True:
>>>     if example_inputs:
>>>         req_id, prompt, sampling_params = example_inputs.pop(0)
>>>         engine.add_request(str(req_id),prompt,sampling_params)
>>>
>>>     # continue the request processing
```

```

>>> request_outputs = engine.step()
>>> for request_output in request_outputs:
>>>     if request_output.finished:
>>>         # return or show the request output
>>>
>>> if not (engine.has_unfinished_requests() or example_inputs):
>>>     break
"""
if self.parallel_config.pipeline_parallel_size > 1:
    raise NotImplementedError(
        "Pipeline parallelism is only supported through AsyncLLMEngine "
        "as performance will be severely degraded otherwise.")

# For llm_engine, there is no pipeline parallel support, so the engine
# used is always 0.
virtual_engine = 0

# These are cached outputs from previous iterations. None if on first
# iteration
cached_outputs = self.cached_scheduler_outputs[virtual_engine]
seq_group_metadata_list = cached_outputs.seq_group_metadata_list
scheduler_outputs = cached_outputs.scheduler_outputs
allow_async_output_proc = cached_outputs.allow_async_output_proc

ctx = self.scheduler_contexts[virtual_engine]

# Clear outputs for each new scheduler iteration
ctx.request_outputs.clear()

# Skip the scheduler if there are any remaining steps in the seq groups.
# This ensures that the scheduler is only called again when the current
# batch has completed.

```



```

# The scheduler is also skipped if a single request caused the last
# engine step to fail, and the previous schedule needs to be rerun.
if not self._has_remaining_steps(
    seq_group_metadata_list
) and not self._skip_scheduling_next_step:
    # Schedule iteration
    (seq_group_metadata_list, scheduler_outputs,
     allow_async_output_proc
    ) = self.scheduler[virtual_engine].schedule()

    ctx.seq_group_metadata_list = seq_group_metadata_list
    ctx.scheduler_outputs = scheduler_outputs

    finished_requests_ids = self.scheduler[
        virtual_engine].get_and_reset_finished_requests_ids()
    # When n>1, elements in self.seq_id_to_seq_group should be deleted
    # here, otherwise memory leaks.
    for finished_request_id in finished_requests_ids:
        if finished_request_id in self.seq_id_to_seq_group:
            del self.seq_id_to_seq_group[finished_request_id]

    # Maybe switch from async mode to sync mode
    if not allow_async_output_proc and len(ctx.output_queue) > 0:
        self._process_model_outputs(ctx=ctx)

    if (self.scheduler_config.is_multi_step
        and scheduler_outputs.num_lookahead_slots > 0):
        # cache the scheduler outputs for the next iteration if we have
        # lookahead slots
        self._cache_scheduler_outputs_for_multi_step(
            virtual_engine, seq_group_metadata_list, scheduler_outputs,

```

```

        allow_async_output_proc)

    else:
        finished_requests_ids = list()

    assert seq_group_metadata_list is not None
    assert scheduler_outputs is not None

    if not scheduler_outputs.is_empty():

        # Check if we have a cached last_output from the previous iteration.
        # For supporting PP this is probably the best way to pass the
        # sampled_token_ids, as a separate broadcast over all the PP stages
        # will cause one virtual engine's microbatch to block the pipeline.
        last_sampled_token_ids = \
            self._get_last_sampled_token_ids(virtual_engine)

        execute_model_req = ExecuteModelRequest(
            seq_group_metadata_list=seq_group_metadata_list,
            blocks_to_swap_in=scheduler_outputs.blocks_to_swap_in,
            blocks_to_swap_out=scheduler_outputs.blocks_to_swap_out,
            blocks_to_copy=scheduler_outputs.blocks_to_copy,
            num_lookahead_slots=scheduler_outputs.num_lookahead_slots,
            running_queue_size=scheduler_outputs.running_queue_size,
            finished_requests_ids=finished_requests_ids,
            # We use ExecuteModelRequest to pass the last sampled_token_ids
            # to each of the non-last PP stages for in-place prepare_input.
            last_sampled_token_ids=last_sampled_token_ids)

    if allow_async_output_proc:
        execute_model_req.async_callback = self.async_callbacks[
            virtual_engine]

```

```

try:
    outputs = self.model_executor.execute_model(
        execute_model_req=execute_model_req)
    self._skip_scheduling_next_step = False
except InputProcessingError as e:
    # The input for this request cannot be processed, so we must
    # abort it. If there are remaining requests in the batch that
    # have been scheduled, they will be retried on the next step.
    invalid_request_id = e.request_id
    self._abort_and_cache_schedule(
        request_id=invalid_request_id,
        virtual_engine=virtual_engine,
        seq_group_metadata_list=seq_group_metadata_list,
        scheduler_outputs=scheduler_outputs,
        allow_async_output_proc=allow_async_output_proc)
    # Raise so the caller is notified that this request failed
    raise

    # We need to do this here so that last step's sampled_token_ids can
    # be passed to the next iteration for PP.
    if self.scheduler_config.is_multi_step:
        self._update_cached_scheduler_output(virtual_engine, outputs)
else:
    # Nothing scheduled => If there is pending async postprocessor,
    # then finish it here.
    if len(ctx.output_queue) > 0:
        self._process_model_outputs(ctx=ctx)
    # No outputs in this case
    outputs = []

# Finish the current step for all the sequence groups.
    if self.scheduler_config.is_multi_step:

```

```

        for seq_group in seq_group_metadata_list:
            seq_group.finish_step()

    if not self._has_remaining_steps(seq_group_metadata_list):
        # clear the cache if we have finished all the steps.
        if self.scheduler_config.is_multi_step:
            self.cached_scheduler_outputs[0] = SchedulerOutputState()

        # is_first_step_output is True only when the num_steps of all
        # the sequences are 1. When the num_steps > 1,
        # multi_step_model_runner does the first-step output append.
        is_first_step_output: bool = False if not seq_group_metadata_list \
            else seq_group_metadata_list[0].state.num_steps == 1

        # Add results to the output_queue
        ctx.append_output(outputs=outputs,
                          seq_group_metadata_list=seq_group_metadata_list,
                          scheduler_outputs=scheduler_outputs,
                          is_async=allow_async_output_proc,
                          is_last_step=True,
                          is_first_step_output=is_first_step_output)

    if outputs and allow_async_output_proc:
        assert len(outputs) == 1, (
            "Async postprocessor expects only a single output set")

        self._advance_to_next_step(
            outputs[0], seq_group_metadata_list,
            scheduler_outputs.scheduled_seq_groups)

    # Check if need to run the usual non-async path
    if not allow_async_output_proc:

```

```

        if not allow_async_output_proc:
            self._process_model_outputs(ctx=ctx)

            # Log stats.
            self.do_log_stats(scheduler_outputs, outputs)

            # Tracing
            self.do_tracing(scheduler_outputs)
        else:
            # Multi-step case
            return ctx.request_outputs

    if not self.has_unfinished_requests():
        # Drain async postprocessor (if exists)
        if len(ctx.output_queue) > 0:
            self._process_model_outputs(ctx=ctx)
            assert len(ctx.output_queue) == 0

        # Stop the execute model loop in parallel workers until there are
        # more requests to process. This avoids waiting indefinitely in
        # torch.distributed ops which may otherwise timeout, and unblocks
        # the RPC thread in the workers so that they can process any other
        # queued control plane messages, such as add/remove Lora adapters.
        logger.debug("Stopping remote worker execution loop.")
        self.model_executor.stop_remote_worker_execution_loop()

    return ctx.request_outputs

def _abort_and_cache_schedule(
    self, request_id: str, virtual_engine: int,
    seq_group_metadata_list: List[SequenceGroupMetadata],
    scheduler_outputs: SchedulerOutputs,

```

```

        allow_async_output_proc: bool) -> None:
        """Aborts a single request, and caches the scheduler outputs minus that
        request. This allows the next step to continue processing the remaining
        requests without having to re-run the scheduler."""

        # Abort the request and remove its sequence group from the current
        # schedule
        self.abort_request(request_id)
        for i, metadata in enumerate(seq_group_metadata_list):
            if metadata.request_id == request_id:
                del seq_group_metadata_list[i]
                break
        for i, group in enumerate(scheduler_outputs.scheduled_seq_groups):
            if group.seq_group.request_id == request_id:
                del scheduler_outputs.scheduled_seq_groups[i]
                break

        # If there are still other sequence groups left in the schedule, cache
        # them and flag the engine to reuse the schedule.
        if len(seq_group_metadata_list) > 0:
            self._skip_scheduling_next_step = True
            # Reuse multi-step caching logic
            self._cache_scheduler_outputs_for_multi_step(
                virtual_engine=virtual_engine,
                scheduler_outputs=scheduler_outputs,
                seq_group_metadata_list=seq_group_metadata_list,
                allow_async_output_proc=allow_async_output_proc)

    def _has_remaining_steps(
        self, seq_group_metadata_list: Optional[List[SequenceGroupMetadata]]
    ) -> bool:
        if (not self.scheduler.config.is_multi_step

```

```

        or not seq_group_metadata_list):
            return False

        # TODO(will) this is a sanity check for now to make sure that all the
        # seqs are on the same steps. Eventually we will want to do some sort of
        # dynamic scheduling when doing multi-step decoding.
        ref_remaining_steps = seq_group_metadata_list[0].state.remaining_steps
        if any([
            seq_group.state.remaining_steps != ref_remaining_steps
            for seq_group in seq_group_metadata_list[1:]
        ]):
            raise AssertionError("All running sequence groups should "
                                "have the same remaining steps.")

        return ref_remaining_steps > 0

def _cache_scheduler_outputs_for_multi_step(
    self, virtual_engine: int,
    seq_group_metadata_list: Optional[List[SequenceGroupMetadata]],
    scheduler_outputs: SchedulerOutputs,
    allow_async_output_proc: bool) -> None:
    co = self.cached_scheduler_outputs[virtual_engine]

    co.seq_group_metadata_list = seq_group_metadata_list
    co.scheduler_outputs = scheduler_outputs
    co.allow_async_output_proc = allow_async_output_proc
    co.last_output = None

def _update_cached_scheduler_output(
    self, virtual_engine: int,
    output: List[Optional[SamplerOutput]]) -> None:
    if (self.parallel_config.pipeline_parallel_size > 1 and len(output) > 0

```

```

        if (self.parallel_config.pipeline_parallel_size > 1 and len(output) > 0
            and output[0] is not None):
            last_output = output[-1]
            assert last_output is not None
            assert last_output.sampled_token_ids_cpu is not None
            assert last_output.sampled_token_ids is None
            assert last_output.sampled_token_probs is None
            self.cached_scheduler_outputs[
                virtual_engine].last_output = last_output

def _get_last_sampled_token_ids(
    self, virtual_engine: int) -> Optional[torch.Tensor]:
    cached_last_output = self.cached_scheduler_outputs[
        virtual_engine].last_output
    if (self.scheduler_config.is_multi_step
        and self.parallel_config.pipeline_parallel_size > 1
        and cached_last_output is not None
        and cached_last_output.sampled_token_ids_cpu is not None):
        return cached_last_output.sampled_token_ids_cpu
    return None

def add_logger(self, logger_name: str, logger: StatLoggerBase) -> None:
    if not self.log_stats:
        raise RuntimeError(
            "Stat logging is disabled. Set `disable_log_stats=False` "
            "argument to enable.")
    if logger_name in self.stat_loggers:
        raise KeyError(f"Logger with name {
            logger_name} already exists.")
    self.stat_loggers[logger_name] = logger

def remove_logger(self, logger_name: str) -> None:

```



```

if not self.log_stats:
    raise RuntimeError(
        "Stat logging is disabled. Set `disable_log_stats=False` "
        "argument to enable.")
if logger_name not in self.stat_loggers:
    raise KeyError(f"Logger with name {
        logger_name} does not exist.")
del self.stat_loggers[logger_name]

def do_log_stats(self,
    scheduler_outputs: Optional[SchedulerOutputs] = None,
    model_output: Optional[List[SamplerOutput]] = None,
    finished_before: Optional[List[int]] = None,
    skip: Optional[List[int]] = None) -> None:
    """Forced log when no requests active."""
    if self.log_stats:
        stats = self._get_stats(scheduler_outputs, model_output,
                                finished_before, skip)
        for logger in self.stat_loggers.values():
            logger.log(stats)

def _get_stats(self,
    scheduler_outputs: Optional[SchedulerOutputs],
    model_output: Optional[List[SamplerOutput]] = None,
    finished_before: Optional[List[int]] = None,
    skip: Optional[List[int]] = None) -> Stats:
    """Get Stats to be Logged to Prometheus.

    Args:
        scheduler_outputs: Optional, used to populate metrics related to
            the scheduled batch,
        model_output: Optional, used to emit speculative decoding metrics

```

```

        which are created by the workers.
        finished_before: Optional, indices of sequences that were finished
        before. These sequences will be ignored.
        skip: Optional, indices of sequences that were preempted. These
        sequences will be ignored.
    """
    now = time.time()

    # System State
    # Scheduler State
    num_running_sys = sum(
        len(scheduler.running) for scheduler in self.scheduler)
    num_swapped_sys = sum(
        len(scheduler.swapped) for scheduler in self.scheduler)
    num_waiting_sys = sum(
        len(scheduler.waiting) for scheduler in self.scheduler)

    # KV Cache Usage in %
    num_total_gpu = self.cache_config.num_gpu_blocks
    gpu_cache_usage_sys = 0.
    if num_total_gpu: # Guard against both None and 0
        num_free_gpu = sum(
            scheduler.block_manager.get_num_free_gpu_blocks()
            for scheduler in self.scheduler)
        gpu_cache_usage_sys = 1.0 - (num_free_gpu / num_total_gpu)

    num_total_cpu = self.cache_config.num_cpu_blocks
    cpu_cache_usage_sys = 0.
    if num_total_cpu: # Guard against both None and 0
        num_free_cpu = sum(
            scheduler.block_manager.get_num_free_cpu_blocks()
            for scheduler in self.scheduler)

```

```

        for scheduler in self.schedulers:
            cpu_cache_usage_sys = 1.0 - (num_free_cpu / num_total_cpu)

# Prefix Cache Hit Rate. Note that we always use
# the cache hit rate of the first virtual engine.
            cpu_prefix_cache_hit_rate = self.scheduler[
                0].get_prefix_cache_hit_rate(Device.CPU)
            gpu_prefix_cache_hit_rate = self.scheduler[
                0].get_prefix_cache_hit_rate(Device.GPU)

# Iteration stats
            num_prompt_tokens_iter = 0
            num_generation_tokens_iter = 0
            num_tokens_iter = 0
            time_to_first_tokens_iter: List[float] = []
            time_per_output_tokens_iter: List[float] = []
            num_preemption_iter = (0 if scheduler_outputs is None else
                                   scheduler_outputs.preempted)

# Request stats
            # Latency
            time_e2e_requests: List[float] = []
            time_queue_requests: List[float] = []
            time_inference_requests: List[float] = []
            time_prefill_requests: List[float] = []
            time_decode_requests: List[float] = []
            time_in_queue_requests: List[float] = []
            model_forward_time_requests: List[float] = []
            model_execute_time_requests: List[float] = []

            # Metadata
            num_prompt_tokens_requests: List[int] = []
            num_generation_tokens_requests: List[int] = []

```

```

n_requests: List[int] = []
max_num_generation_tokens_requests: List[int] = []
max_tokens_requests: List[int] = []
finished_reason_requests: List[str] = []

# LoRA requests
running_lora_adapters = dict(
    collectionsCounter([
        running_request.lora_request.lora_name
        for scheduler in self.scheduler
        for running_request in scheduler.running
        if running_request.lora_request
    ]))
waiting_lora_adapters = dict(
    collectionsCounter([
        waiting_request.lora_request.lora_name
        for scheduler in self.scheduler
        for waiting_request in scheduler.waiting
        if waiting_request.lora_request
    ]))
max_lora_stat = "0"
if self.lora_config:
    max_lora_stat = str(self.lora_config.max_loras)

# NOTE: This loop assumes prefill seq_groups are before
# decode seq_groups in scheduled_seq_groups.
if scheduler_outputs is not None:
    # For async postprocessor, already finished sequences need to be
    # not counted (to avoid double counting)
    actual_num_batched_tokens = scheduler_outputs.num_batched_tokens # type: ignore

```

ore

```

num_generation_tokens_from_prefill_groups = 0
# NOTE: if scheduler_outputs.num_prefill_groups > 0 and
# the len of scheduler_outputs.scheduled_seq_groups is !=
# scheduler_outputs.num_prefill_groups, this means that
# chunked prefills have been detected.

for idx, scheduled_seq_group in enumerate(
    scheduler_outputs.scheduled_seq_groups):
    # Skip double logging when using async output proc
    if finished_before and idx in finished_before:
        actual_num_batched_tokens -= 1
        continue

    # Currently, skip == preempted sequences, so we need to skip
    # their log stats
    if skip and idx in skip:
        continue

    group_was_prefill = idx < scheduler_outputs.num_prefill_groups
    seq_group = scheduled_seq_group.seq_group

    # NOTE: a seq_group that completed all of its prefill tokens
    # in the last iteration will have seq_group.is_prefill() = False
    # with group_was_prefill = True
    if group_was_prefill:
        # Number of prompt tokens.
        num_prompt_tokens_iter += (
            scheduled_seq_group.token_chunk_size)

    # If the seq_group just finished the prefill state
    # get TTFT.
    if not seq_group.is_prefill():

```

```

    if not seq_group.is_prefill():
        latency = seq_group.get_last_token_latency()
        time_to_first_tokens_iter.append(latency)

        # One generation token per finished prefill.
        num_generation_tokens_from_prefill_groups += (
            seq_group.num_seqs())
    else:
        # TPOTs.
        latency = seq_group.get_last_token_latency()
        time_per_output_tokens_iter.append(latency)
        if seq_group.state.current_step == 0:
            # For async_output_proc, the do_log_stats()
            # is called following init_multi_step(), which
            # sets the current_step to zero.
            actual_num_batched_tokens += \
                seq_group.state.num_steps - 1
        else:
            actual_num_batched_tokens += \
                seq_group.state.current_step - 1

# Because of chunked prefill, we can have a single sequence
# group that does multiple prompt_runs. To prevent logging
# the same metadata more than once per request, we standardize
# on logging request level information for finished requests,
# which can only happen once.
    if seq_group.is_finished():
        # Latency timings
        time_e2e_requests.append(now -
                                seq_group.metrics.arrival_time)
        if (seq_group.metrics.first_scheduled_time is not None and
            seq_group.metrics.first_token_time is not None):

```

```
time_queue_requests.append(
    seq_group.metrics.first_scheduled_time -
    seq_group.metrics.arrival_time)
time_prefill_requests.append(
    seq_group.metrics.first_token_time -
    seq_group.metrics.first_scheduled_time)
time_decode_requests.append(
    now - seq_group.metrics.first_token_time)
time_inference_requests.append(
    now - seq_group.metrics.first_scheduled_time)
if seq_group.metrics.time_in_queue is not None:
    time_in_queue_requests.append(
        seq_group.metrics.time_in_queue)
if seq_group.metrics.model_forward_time is not None:
    model_forward_time_requests.append(
        seq_group.metrics.model_forward_time)
if seq_group.metrics.model_execute_time is not None:
    model_execute_time_requests.append(
        seq_group.metrics.model_execute_time * 1000)
# Metadata
num_prompt_tokens_requests.append(
    len(seq_group.prompt_token_ids))
num_generation_tokens_requests.extend([
    seq.get_output_len()
    for seq in seq_group.get_finished_seqs()
])
max_num_generation_tokens_requests.append(
    max(seq.get_output_len()
        for seq in seq_group.get_seqs()))
if seq_group.sampling_params is not None:
    n_requests.append(seq_group.sampling_params.n)
    max_tokens_requests.append(
```

```

        seq_group.sampling_params.max_tokens)
    finished_reason_requests.extend([
        SequenceStatus.get_finished_reason(seq.status)
        for seq in seq_group.get_finished_seqs()
    ])

    # Number of generation tokens.
    # num_batched_tokens equals the number of prompt_tokens plus the
    # number of decode_tokens in a single iteration. So,
    # num_generation_tokens = num_batched_tokens - num_prompt_tokens
    # + num_generation_tokens_from_prefill_groups (since we generate
    # one token on prefills on iters where the prefill finishes).
    num_generation_tokens_iter = (
        actual_num_batched_tokens - num_prompt_tokens_iter +
        num_generation_tokens_from_prefill_groups)
    num_tokens_iter = (num_generation_tokens_iter +
        num_prompt_tokens_iter)

    # Spec decode, if enabled, emits specialized metrics from the worker in
    # sampler output.
    if model_output and isinstance(model_output[0], SamplerOutput) and (
        model_output[0].spec_decode_worker_metrics is not None):
        spec_decode_metrics = model_output[0].spec_decode_worker_metrics
    else:
        spec_decode_metrics = None

    return Stats(
        now=now,
        # System stats
        # Scheduler State
        num_running_sys=num_running_sys,
        num_swapped_sys=num_swapped_sys,
        num_waiting_sys=num_waiting_sys

```



```
num_waiting_sys=num_waiting_sys,
# KV Cache Usage in %
gpu_cache_usage_sys=gpu_cache_usage_sys,
cpu_cache_usage_sys=cpu_cache_usage_sys,
# Prefix Cache Hit Rate
cpu_prefix_cache_hit_rate=cpu_prefix_cache_hit_rate,
gpu_prefix_cache_hit_rate=gpu_prefix_cache_hit_rate,

# Iteration stats
num_prompt_tokens_iter=num_prompt_tokens_iter,
num_generation_tokens_iter=num_generation_tokens_iter,
num_tokens_iter=num_tokens_iter,
time_to_first_tokens_iter=time_to_first_tokens_iter,
time_per_output_tokens_iter=time_per_output_tokens_iter,
spec_decode_metrics=spec_decode_metrics,
num_preemption_iter=num_preemption_iter,

# Request stats
# Latency
time_e2e_requests=time_e2e_requests,
time_queue_requests=time_queue_requests,
time_inference_requests=time_inference_requests,
time_prefill_requests=time_prefill_requests,
time_decode_requests=time_decode_requests,
time_in_queue_requests=time_in_queue_requests,
model_forward_time_requests=model_forward_time_requests,
model_execute_time_requests=model_execute_time_requests,
# Metadata
num_prompt_tokens_requests=num_prompt_tokens_requests,
num_generation_tokens_requests=num_generation_tokens_requests,
max_num_generation_tokens_requests=
max_num_generation_tokens_requests,
```

```

        n_requests=n_requests,
        max_tokens_requests=max_tokens_requests,
        finished_reason_requests=finished_reason_requests,
        max_lora=str(max_lora_stat),
        waiting_lora_adapters=list(waiting_lora_adapters.keys()),
        running_lora_adapters=list(running_lora_adapters.keys()))

def add_lora(self, lora_request: LoRARequest) -> bool:
    return self.model_executor.add_lora(lora_request)

def remove_lora(self, lora_id: int) -> bool:
    return self.model_executor.remove_lora(lora_id)

def list_loras(self) -> Set[int]:
    return self.model_executor.list_loras()

def pin_lora(self, lora_id: int) -> bool:
    return self.model_executor.pin_lora(lora_id)

def add_prompt_adapter(
    self, prompt_adapter_request: PromptAdapterRequest) -> bool:
    return self.model_executor.add_prompt_adapter(prompt_adapter_request)

def remove_prompt_adapter(self, prompt_adapter_id: int) -> bool:
    return self.model_executor.remove_prompt_adapter(prompt_adapter_id)

def list_prompt_adapters(self) -> List[int]:
    return self.model_executor.list_prompt_adapters()

def start_profile(self) -> None:
    self.model_executor.start_profile()

```

```

def stop_profile(self) -> None:
    self.model_executor.stop_profile()

def sleep(self, level: int = 1) -> None:
    assert self.vllm_config.model_config.enable_sleep_mode, (
        "Sleep mode is not enabled in the model config")
    self.model_executor.sleep(level=level)

def wake_up(self) -> None:
    assert self.vllm_config.model_config.enable_sleep_mode, (
        "Sleep mode is not enabled in the model config")
    self.model_executor.wake_up()

def is_sleeping(self) -> bool:
    return self.model_executor.is_sleeping

def check_health(self) -> None:
    if self.tokenizer:
        self.tokenizer.check_health()
    self.model_executor.check_health()

def is_tracing_enabled(self) -> bool:
    return self.tracer is not None

def do_tracing(self,
               scheduler_outputs: SchedulerOutputs,
               finished_before: Optional[List[int]] = None) -> None:
    if self.tracer is None:
        return

    for idx, scheduled_seq_group in enumerate(
        scheduler_outputs.scheduled_seq_groups):

```



```
seq_span.set_attribute(SpanAttributes.GEN_AI_REQUEST_MAX_TOKENS,
                        seq_group.sampling_params.max_tokens)
seq_span.set_attribute(SpanAttributes.GEN_AI_REQUEST_N,
                        seq_group.sampling_params.n)
seq_span.set_attribute(SpanAttributes.GEN_AI_USAGE_NUM_SEQUENCES,
                        seq_group.num_seqs())
seq_span.set_attribute(SpanAttributes.GEN_AI_USAGE_PROMPT_TOKENS,
                        len(seq_group.prompt_token_ids))
seq_span.set_attribute(
    SpanAttributes.GEN_AI_USAGE_COMPLETION_TOKENS,
    sum([
        seq.get_output_len()
        for seq in seq_group.get_finished_seqs()
    ]))
seq_span.set_attribute(SpanAttributes.GEN_AI_LATENCY_TIME_IN_QUEUE,
                        metrics.time_in_queue)
seq_span.set_attribute(
    SpanAttributes.GEN_AI_LATENCY_TIME_TO_FIRST_TOKEN, ttft)
seq_span.set_attribute(SpanAttributes.GEN_AI_LATENCY_E2E, e2e_time)
if metrics.scheduler_time is not None:
    seq_span.set_attribute(
        SpanAttributes.GEN_AI_LATENCY_TIME_IN_SCHEDULER,
        metrics.scheduler_time)
if metrics.model_forward_time is not None:
    seq_span.set_attribute(
        SpanAttributes.GEN_AI_LATENCY_TIME_IN_MODEL_FORWARD,
        metrics.model_forward_time / 1000.0)
if metrics.model_execute_time is not None:
    seq_span.set_attribute(
        SpanAttributes.GEN_AI_LATENCY_TIME_IN_MODEL_EXECUTE,
        metrics.model_execute_time)
```

```

def _validate_model_inputs(self, inputs: ProcessorInputs,
                           lora_request: Optional[LoRARequest]):
    if is_encoder_decoder_inputs(inputs):
        # For encoder-decoder multimodal models, the max_prompt_len
        # restricts the decoder prompt length
        prompt_inputs = inputs["decoder" if self.model_config.
                               is_multimodal_model else "encoder"]
    else:
        prompt_inputs = inputs

    prompt_ids = SingletonInputsAdapter(prompt_inputs).prompt_token_ids

    if prompt_ids is None or len(prompt_ids) == 0:
        raise ValueError("Prompt cannot be empty")

    if self.model_config.is_multimodal_model:
        max_prompt_len = self.model_config.max_model_len

        if len(prompt_ids) > max_prompt_len:
            raise ValueError(
                f"The prompt (total length {
                len(prompt_ids)}) is too long "
                f"to fit into the model (context length {
                max_prompt_len}). "
                "Make sure that `max_model_len` is no smaller than the "
                "number of text tokens plus multimodal tokens. For image "
                "inputs, the number of image tokens depends on the number "
                "of images, and possibly their aspect ratios as well.")

    # TODO: Find out how many placeholder tokens are there so we can
    # check that chunked prefill does not truncate them
    # max_batch_len = self.scheduler.config.max_num_batched_tokens

```

```

def _build_logits_processors(
    self, sampling_params: SamplingParams,
    lora_request: Optional[LoRARequest]) -> SamplingParams:
    """Constructs logits processors based on the guided_decoding,
    logits_bias, and allowed_token_ids fields in sampling_params. Deletes
    those fields and adds the constructed logits processors to the
    logits_processors field. Returns the modified sampling params."""

    logits_processors = []

    if sampling_params.guided_decoding is not None:
        # Defensively copy sampling params since guided decoding logits
        # processors can have different state for each request
        sampling_params = copy.copy(sampling_params)
        guided_decoding = sampling_params.guided_decoding

        logger.debug(
            "Building guided decoding logits processor in "
            "LLMEngine. Params: %s", guided_decoding)

        tokenizer = self.get_tokenizer(lora_request=lora_request)
        guided_decoding.backend = guided_decoding.backend or \
            self.decoding_config.guided_decoding_backend

        logger.debug("Reasoning backend: %s",
            self.decoding_config.reasoning_backend)

        processor = get_local_guided_decoding_logits_processor(
            guided_params=guided_decoding,
            tokenizer=tokenizer,

```

```

        model_config=self.model_config,
        reasoning_backend=self.decoding_config.reasoning_backend,
    )
    if processor:
        logits_processors.append(processor)

    # Unset so this doesn't get passed down to the model
    sampling_params.guided_decoding = None

    if (sampling_params.logit_bias or sampling_params.allowed_token_ids):
        tokenizer = self.get_tokenizer(lora_request=lora_request)

        processors = get_openai_logits_processors(
            logit_bias=sampling_params.logit_bias,
            allowed_token_ids=sampling_params.allowed_token_ids,
            tokenizer=tokenizer)
        logits_processors.extend(processors)

    # Unset so these don't get passed down to the model
    sampling_params.logit_bias = None
    sampling_params.allowed_token_ids = None

    if len(sampling_params.bad_words) > 0:
        tokenizer = self.get_tokenizer(lora_request)
        processors = get_bad_words_logits_processors(
            bad_words=sampling_params.bad_words, tokenizer=tokenizer)
        logits_processors.extend(processors)

    if logits_processors:
        if sampling_params.logits_processors is None:
            sampling_params.logits_processors = logits_processors
        else:

```



```

        sampling_params.logits_processors.extend(logits_processors)

    return sampling_params

if envs.is_set("VLLM_USE_V1") and envs.VLLM_USE_V1:
    from vllm.v1.engine.llm_engine import LLMEngine as V1LLMEngine
    LLMEngine = V1LLMEngine  # type: ignore

```

2.2 AsyncLLMEngine

`AsyncLLMEngine` 类是 `LLMEngine` 类的异步封装。它使用 `asyncio` 创建一个后台循环，持续处理传入的请求。`AsyncLLMEngine` 专为在线服务设计，能够处理多个并发请求并将输出流式传输给客户端。

OpenAI 兼容的 API 服务器使用了 `AsyncLLMEngine`。此外，还有一个演示用的 API 服务器作为更简单的示例，代码如下 `api_server.py` 所示：

```

# SPDX-License-Identifier: Apache-2.0
"""
NOTE: This API server is used only for demonstrating usage of AsyncEngine
and simple performance benchmarks. It is not intended for production use.
For production use, we recommend using our OpenAI compatible server.
We are also not going to accept PRs modifying this file, please
change `vllm/entrypoints/openai/api_server.py` instead.
"""

import asyncio
import json
import ssl

from argparse import Namespace
from collections.abc import AsyncGenerator
from typing import Any, Optional

```

```

from typing import Any, Optional

from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse, Response, StreamingResponse

from vllm.engine.arg_utils import AsyncEngineArgs
from vllm.engine.async_llm_engine import AsyncLLMEngine
from vllm.entrypoints.launcher import serve_http
from vllm.entrypoints.utils import with_cancellation
from vllm.logger import init_logger
from vllm.sampling_params import SamplingParams
from vllm.usage.usage_lib import UsageContext
from vllm.utils import FlexibleArgumentParser, random_uuid, set_ulimit
from vllm.version import __version__ as VLLM_VERSION

logger = init_logger("vllm.entrypoints.api_server")

TIMEOUT_KEEP_ALIVE = 5  # seconds.
app = FastAPI()
engine = None

@app.get("/health")
async def health() -> Response:
    """Health check."""
    return Response(status_code=200)

@app.post("/generate")
async def generate(request: Request) -> Response:
    """Generate completion for the request.

```

```
The request should be a JSON object with the following fields:
- prompt: the prompt to use for the generation.
- stream: whether to stream the results or not.
- other fields: the sampling parameters (See `SamplingParams` for details).
"""
```

```
request_dict = await request.json()
return await _generate(request_dict, raw_request=request)
```

```
@with_cancellation
```

```
async def _generate(request_dict: dict, raw_request: Request) -> Response:
```

```
    prompt = request_dict.pop("prompt")
```

```
    stream = request_dict.pop("stream", False)
```

```
    sampling_params = SamplingParams(**request_dict)
```

```
    request_id = random_uuid()
```

```
    assert engine is not None
```

```
    results_generator = engine.generate(prompt, sampling_params, request_id)
```

```
# Streaming case
```

```
async def stream_results() -> AsyncGenerator[bytes, None]:
```

```
    async for request_output in results_generator:
```

```
        prompt = request_output.prompt
```

```
        assert prompt is not None
```

```
        text_outputs = [
```

```
            prompt + output.text for output in request_output.outputs
```

```
        ]
```

```
        ret = {
```

```
            "text": text_outputs}
```

```
        yield (json.dumps(ret) + "\n").encode("utf-8")
```

```
    if stream:
```

```

        return StreamingResponse(stream_results())

    # Non-streaming case
    final_output = None
    try:
        async for request_output in results_generator:
            final_output = request_output
    except asyncio.CancelledError:
        return Response(status_code=499)

    assert final_output is not None
    prompt = final_output.prompt
    assert prompt is not None
    text_outputs = [prompt + output.text for output in final_output.outputs]
    ret = {
        "text": text_outputs
    }
    return JSONResponse(ret)

def build_app(args: Namespace) -> FastAPI:
    global app

    app.root_path = args.root_path
    return app

async def init_app(
    args: Namespace,
    llm_engine: Optional[AsyncLLMEngine] = None,
) -> FastAPI:
    app = build_app(args)

```

```

global engine

engine_args = AsyncEngineArgs.from_cli_args(args)
engine = (llm_engine
          if llm_engine is not None else AsyncLLMEngine.from_engine_args(
              engine_args, usage_context=UsageContext.API_SERVER))

return app

async def run_server(args: Namespace,
                    llm_engine: Optional[AsyncLLMEngine] = None,
                    **uvicorn_kwargs: Any) -> None:
    logger.info("vLLM API server version %s", VLLM_VERSION)
    logger.info("args: %s", args)

    set_ulimit()

    app = await init_app(args, llm_engine)
    assert engine is not None

    shutdown_task = await serve_http(
        app,
        sock=None,
        enable_ssl_refresh=args.enable_ssl_refresh,
        host=args.host,
        port=args.port,
        log_level=args.log_level,
        timeout_keep_alive=TIMEOUT_KEEP_ALIVE,
        ssl_keyfile=args.ssl_keyfile,
        ssl_certfile=args.ssl_certfile,

```

```

        ssl_ca_certs=args.ssl_ca_certs,
        ssl_cert_reqs=args.ssl_cert_reqs,
        **uvicorn_kwangs,
    )

    await shutdown_task

if __name__ == "__main__":
    parser = FlexibleArgumentParser()
    parser.add_argument("--host", type=str, default=None)
    parser.add_argument("--port", type=parser.check_port, default=8000)
    parser.add_argument("--ssl-keyfile", type=str, default=None)
    parser.add_argument("--ssl-certfile", type=str, default=None)
    parser.add_argument("--ssl-ca-certs",
                        type=str,
                        default=None,
                        help="The CA certificates file")
    parser.add_argument(
        "--enable-ssl-refresh",
        action="store_true",
        default=False,
        help="Refresh SSL Context when SSL certificate files change")
    parser.add_argument(
        "--ssl-cert-reqs",
        type=int,
        default=int(ssl.CERT_NONE),
        help="Whether client certificate is required (see stdlib ssl module's)"
    )
    parser.add_argument(
        "--root-path",
        type=str,

```

```

        default=None,
        help="FastAPI root_path when app is behind a path based routing proxy")
    parser.add_argument("--log-level", type=str, default="debug")
    parser = AsyncEngineArgs.add_cli_args(parser)
    args = parser.parse_args()

    asyncio.run(run_server(args))

```

`AsyncLLMEngine` 的代码为如下 `async_llm_engine.py` :

```

# SPDX-License-Identifier: Apache-2.0

import asyncio
import copy
import time
import weakref
from functools import partial
from typing import (Any, AsyncGenerator, Callable, Coroutine, Dict, Iterable,
                    List, Mapping, Optional, Set, Tuple, Type, Union, overload)
from weakref import ReferenceType

from typing_extensions import deprecated

import vllm.envs as envs
from vllm.config import (DecodingConfig, LoRAConfig, ModelConfig,
                         ParallelConfig, SchedulerConfig, VllmConfig)
from vllm.core.scheduler import SchedulerOutputs
from vllm.engine.arg_utils import AsyncEngineArgs
from vllm.engine.async_timeout import asyncio_timeout
from vllm.engine.llm_engine import LLMEngine, SchedulerOutputState
from vllm.engine.metrics_types import StatLoggerBase
from vllm.engine.protocol import EngineClient

```

```

from vllm.executor.executor_base import ExecutorBase
from vllm.inputs import PromptType
from vllm.inputs.preprocess import InputPreprocessor
from vllm.logger import init_logger
from vllm.lora.request import LoRARequest
from vllm.model_executor.guided_decoding import (
    get_guided_decoding_logits_processor)
from vllm.model_executor.layers.sampler import SamplerOutput
from vllm.outputs import PoolingRequestOutput, RequestOutput
from vllm.pooling_params import PoolingParams
from vllm.prompt_adapter.request import PromptAdapterRequest
from vllm.sampling_params import SamplingParams
from vllm.sequence import ExecuteModelRequest
from vllm.transformers_utils.tokenizer import AnyTokenizer
from vllm.usage.usage_lib import UsageContext
from vllm.utils import deprecate_kwargs, weak_bind

logger = init_logger(__name__)
ENGINE_ITERATION_TIMEOUT_S = envs.VLLM_ENGINE_ITERATION_TIMEOUT_S

class AsyncEngineDeadError(RuntimeError):
    pass

def _log_task_completion(task: asyncio.Task,
                        error_callback: Callable[[Exception], None]) -> None:
    """This function is only intended for the `engine.run_engine_loop()` task.

    In particular, that task runs a `while True` loop that can only exit if
    there is an exception.
    """

```



```

exception = None
try:
    return_value = task.result()
    raise AssertionError(
        f"The engine background task should never finish without an "
        f"exception. {
            return_value}")
except asyncio.exceptions.CancelledError:
    # We assume that if the task is cancelled, we are gracefully shutting
    # down. This should only happen on program exit.
    logger.info("Engine is gracefully shutting down.")
except Exception as e:
    exception = e
    logger.error("Engine background task failed", exc_info=e)
    error_callback(exception)
    raise AsyncEngineDeadError(
        "Task finished unexpectedly. This should never happen! "
        "Please open an issue on GitHub. See stack trace above for the "
        "actual cause.") from e

STOP_ITERATION = Exception() # Sentinel

class AsyncStream:
    """A stream of RequestOutputs or PoolingRequestOutputs for a request
    that can be iterated over asynchronously via an async generator."""

    def __init__(self, request_id: str, cancel: Callable[[str], None]) -> None:
        self.request_id = request_id

```

```

self._cancel = cancel
self._queue: asyncio.Queue = asyncio.Queue()
self._finished = False

def put(self, item: Union[RequestOutput, PoolingRequestOutput,
                          Exception]) -> None:
    if not self._finished:
        self._queue.put_nowait(item)

def finish(
    self,
    exception: Optional[Union[BaseException, Type[BaseException]]] = None,
) -> None:
    if not self._finished:
        self._finished = True
        self._queue.put_nowait(
            exception if self._is_raisable(exception) else STOP_ITERATION)

@property
def finished(self) -> bool:
    return self._finished

async def generator(
    self
) -> AsyncGenerator[Union[RequestOutput, PoolingRequestOutput], None]:
    try:
        while True:
            result = await self._queue.get()
            if self._is_raisable(result):
                if result == STOP_ITERATION:
                    return
                raise result
    
```

```

        yield result
    except GeneratorExit:
        self._cancel(self.request_id)
        raise asyncio.CancelledError from None

    @staticmethod
    def _is_raisable(value: Any):
        return isinstance(value, BaseException) or \
            (isinstance(value, type) and \
             isinstance(value, BaseException))

class RequestTracker:
    """Synchronous abstraction for tracking requests."""

    def __init__(self) -> None:
        self._request_streams: Dict[str, AsyncStream] = {
            }
        self._aborted_requests: asyncio.Queue[str] = asyncio.Queue()
        self._new_requests: asyncio.Queue[Tuple[AsyncStream,
                                                  dict]] = asyncio.Queue()
        self.new_requests_event = asyncio.Event()

    def __contains__(self, item):
        return item in self._request_streams

    def __len__(self) -> int:
        return len(self._request_streams)

    def propagate_exception(self,
                           exc: Exception,
                           request_id: Optional[str] = None) -> None:

```

```

        request_id: Optional[str] = None) -> None:
    """Propagate an exception to request streams
    (all if request_id is None)."""
    if request_id is not None:
        self.abort_request(request_id, exception=exc)
    else:
        # NB: tuple() used here because self.abort_request pops the stream
        # out of self._request_streams, so we can't iterate on it directly
        for rid in tuple(self._request_streams.keys()):
            self.abort_request(rid, exception=exc)

def process_request_output(self,
                           request_output: Union[RequestOutput,
                                                  PoolingRequestOutput],
                           *,
                           verbose: bool = False) -> None:
    """Process a request output from the engine."""
    request_id = request_output.request_id
    finished = request_output.finished

    if finished:
        stream = self._request_streams.pop(request_id, None)
    else:
        stream = self._request_streams.get(request_id)
        # Guard against a KeyError which can occur if the request was aborted
        # while the output was generated
    if stream is not None:
        stream.put(request_output)
        if finished:
            stream.finish()

    if verbose and finished:

```

```

        logger.info("Finished request %s.", request_id)

    def process_exception(self,
                          request_id: str,
                          exception: BaseException,
                          *,
                          verbose: bool = False) -> None:
        """Propagate an exception from the engine."""
        if verbose:
            logger.info("Finished request %s.", request_id)
        self.abort_request(request_id, exception=exception)

    def add_request(self,
                   request_id: str,
                   *,
                   verbose: bool = False,
                   **engine_add_request_kwargs) -> AsyncStream:
        """Add a request to be sent to the engine on the next background
        loop iteration."""
        if request_id in self._request_streams:
            raise KeyError(f"Request {
                request_id} already exists.")

        abort_request = partial(self.abort_request, verbose=verbose)
        stream = AsyncStream(request_id, abort_request)
        self._new_requests.put_nowait((stream, {

            "request_id": request_id,
            **engine_add_request_kwargs
        })))

    self.new_requests_event.set()

```

```

        if verbose:
            logger.info("Added request %s.", request_id)

        return stream

def abort_request(self,
                  request_id: str,
                  *,
                  exception: Optional[Union[BaseException,
                                           Type[BaseException]]] = None,
                  verbose: bool = False) -> None:
    """Abort a request during next background loop iteration."""
    if verbose:
        logger.info("Aborted request %s.", request_id)

    self._aborted_requests.put_nowait(request_id)

    stream = self._request_streams.pop(request_id, None)
    if stream is not None:
        stream.finish(exception=exception)

def get_new_and_aborted_requests(self) -> Tuple[List[Dict], Set[str]]:
    """Get the new requests and finished requests to be
    sent to the engine."""
    new_requests: List[Dict] = []
    finished_requests: Set[str] = set()

    while not self._aborted_requests.empty():
        request_id = self._aborted_requests.get_nowait()
        finished_requests.add(request_id)

```

```

        while not self._new_requests.empty():
            stream, new_request = self._new_requests.get_nowait()
            request_id = stream.request_id
            if request_id in finished_requests:
                # The request has already been aborted.
                stream.finish(asyncio.CancelledError)
                finished_requests.discard(request_id)
            else:
                self._request_streams[request_id] = stream
                new_requests.append(new_request)

        return new_requests, finished_requests

    async def wait_for_new_requests(self):
        if not self.has_new_requests():
            await self.new_requests_event.wait()
        self.new_requests_event.clear()

    def has_new_requests(self):
        return not self._new_requests.empty()

class _AsyncLLMEngine(LLMEngine):
    """Extension of LLMEngine to add async methods."""

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

    async def step_async(
        self, virtual_engine: int
    ) -> List[Union[RequestOutput, PoolingRequestOutput]]:

```

```
"""Performs one decoding iteration and returns newly generated results.  
The workers are ran asynchronously if possible.
```

```
  
This function performs one decoding iteration of the engine. It first  
schedules the sequences to be executed in the next iteration and the  
token blocks to be swapped in/out/copy. Then, it executes the model  
and updates the scheduler with the model outputs. Finally, it decodes  
the sequences and returns the newly generated results.
```

```
"""
```

```
# these are cached outputs from previous iterations. None if on first  
# iteration
```

```
cached_outputs = self.cached_scheduler_outputs[virtual_engine]  
seq_group_metadata_list = cached_outputs.seq_group_metadata_list  
scheduler_outputs = cached_outputs.scheduler_outputs  
allow_async_output_proc = cached_outputs.allow_async_output_proc
```

```
ctx = self.scheduler_contexts[virtual_engine]
```

```
# Clear outputs for each new scheduler iteration  
ctx.request_outputs.clear()
```

```
# skip the scheduler if there are any remaining steps in the seq groups.  
# This ensures that the scheduler is only called again when the current  
# batch has completed.
```

```
if not self._has_remaining_steps(seq_group_metadata_list):
```

```
    # Schedule iteration
```

```
    (seq_group_metadata_list, scheduler_outputs,  
     allow_async_output_proc  
    ) = self.scheduler[virtual_engine].schedule()
```

```
    ctx.seq_group_metadata_list = seq_group_metadata_list
```



```

ctx.scheduler_outputs = scheduler_outputs

finished_requests_ids = self.scheduler[
    virtual_engine].get_and_reset_finished_requests_ids()

# Maybe switch from async mode to sync mode
if not allow_async_output_proc and len(ctx.output_queue) > 0:
    self._process_model_outputs(ctx=ctx)

if (self.scheduler_config.is_multi_step
    and scheduler_outputs.num_lookahead_slots > 0):
    # cache the scheduler outputs for the next iteration if we have
    # Lookahead slots
    self._cache_scheduler_outputs_for_multi_step(
        virtual_engine, seq_group_metadata_list, scheduler_outputs,
        allow_async_output_proc)
else:
    finished_requests_ids = list()

assert seq_group_metadata_list is not None
assert scheduler_outputs is not None

if not scheduler_outputs.is_empty():

    # Check if we have a cached last_output from the previous iteration.
    # For supporting PP this is probably the best way to pass the
    # sampled_token_ids, as a separate broadcast over all the PP stages
    # will cause one virtual engine's microbatch to block the pipeline.
    last_sampled_token_ids = \
        self._get_last_sampled_token_ids(virtual_engine)

execute_model_req = ExecuteModelRequest(

```

```

execute_model_req = ExecuteModelRequest(
    seq_group_metadata_list=seq_group_metadata_list,
    blocks_to_swap_in=scheduler_outputs.blocks_to_swap_in,
    blocks_to_swap_out=scheduler_outputs.blocks_to_swap_out,
    blocks_to_copy=scheduler_outputs.blocks_to_copy,
    virtual_engine=virtual_engine,
    num_lookahead_slots=scheduler_outputs.num_lookahead_slots,
    running_queue_size=scheduler_outputs.running_queue_size,
    finished_requests_ids=finished_requests_ids,
    # We use ExecuteModelRequest to pass the last sampled_token_ids
    # to each of the non-last PP stages for in-place prepare_input.
    last_sampled_token_ids=last_sampled_token_ids)

if allow_async_output_proc:
    execute_model_req.async_callback = self.async_callbacks[
        virtual_engine]

# Execute the model.
outputs = await self.model_executor.execute_model_async(
    execute_model_req)

# we need to do this here so that last step's sampled_token_ids can
# be passed to the next iteration for PP.
if self.scheduler_config.is_multi_step:
    self._update_cached_scheduler_output(virtual_engine, outputs)
else:
    if len(ctx.output_queue) > 0:
        self._process_model_outputs(ctx=ctx)
    outputs = []

# Finish the current step for all the sequence groups.
if self.scheduler_config.is_multi_step:

```

```

        for seq_group in seq_group_metadata_list:
            seq_group.finish_step()

    if not self._has_remaining_steps(seq_group_metadata_list):
        # Clear the cache if we have finished all the steps
        if self.scheduler_config.is_multi_step:
            self.cached_scheduler_outputs[
                virtual_engine] = SchedulerOutputState()

        # is_first_step_output is True only when the num_steps of all
        # the sequences are 1. When the num_steps > 1,
        # multi_step_model_runner does the first-step output append.
        is_first_step_output: bool = False if not seq_group_metadata_list \
            else seq_group_metadata_list[0].state.num_steps == 1

        ctx.append_output(outputs=outputs,
                          seq_group_metadata_list=seq_group_metadata_list,
                          scheduler_outputs=scheduler_outputs,
                          is_async=allow_async_output_proc,
                          is_last_step=True,
                          is_first_step_output=is_first_step_output)

    if outputs and allow_async_output_proc:
        assert len(
            outputs
        ) == 1, "Async postprocessor expects only a single output set"
        self._advance_to_next_step(
            outputs[0], seq_group_metadata_list,
            scheduler_outputs.scheduled_seq_groups)

    if not allow_async_output_proc:
        self._process_model_outputs(ctx=ctx)

```

```

        # Log stats.
        self.do_log_stats(scheduler_outputs, outputs)

        # Tracing
        self.do_tracing(scheduler_outputs)

    else:
        # Multi-step case
        return ctx.request_outputs

    if not self.has_unfinished_requests():
        # Drain async postprocessor (if exists)
        if len(ctx.output_queue) > 0:
            self._process_model_outputs(ctx=ctx)
            assert len(ctx.output_queue) == 0

    return ctx.request_outputs

async def stop_remote_worker_execution_loop_async(self) -> None:
    """Stop the remote worker execution loop."""
    await self.model_executor.stop_remote_worker_execution_loop_async()

async def get_tokenizer_async(self,
                              lora_request: Optional[LoRARequest] = None
                              ) -> AnyTokenizer:
    return await (
        self.get_tokenizer_group().get_lora_tokenizer_async(lora_request))

@overload
@deprecated("'inputs' will be renamed to 'prompt'")
async def add_request_async(

```

```

    async def add_request_async(
        self,
        request_id: str,
        *,
        inputs: PromptType,
        params: Union[SamplingParams, PoolingParams],
        arrival_time: Optional[float] = None,
        lora_request: Optional[LoRARequest] = None,
        trace_headers: Optional[Mapping[str, str]] = None,
        prompt_adapter_request: Optional[PromptAdapterRequest] = None,
        priority: int = 0,
    ) -> None:
        ...

    @overload
    async def add_request_async(
        self,
        request_id: str,
        prompt: PromptType,
        params: Union[SamplingParams, PoolingParams],
        arrival_time: Optional[float] = None,
        lora_request: Optional[LoRARequest] = None,
        trace_headers: Optional[Mapping[str, str]] = None,
        prompt_adapter_request: Optional[PromptAdapterRequest] = None,
        priority: int = 0,
    ) -> None:
        ...

    @deprecate_kwargs(
        "inputs",
        additional_message="Please use the 'prompt' parameter instead.",
    )

```

```

async def add_request_async(
    self,
    request_id: str,
    prompt: Optional[PromptType] = None,
    params: Optional[Union[SamplingParams, PoolingParams]] = None,
    arrival_time: Optional[float] = None,
    lora_request: Optional[LoRARequest] = None,
    trace_headers: Optional[Mapping[str, str]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
    priority: int = 0,
    *,
    inputs: Optional[PromptType] = None, # DEPRECATED
) -> None:
    """Async version of :meth:`add_request`."""
    if inputs is not None:
        prompt = inputs
    assert prompt is not None and params is not None

    if lora_request is not None and not self.lora_config:
        raise ValueError(f"Got lora_request {
            lora_request} but LoRA is "
                        "not enabled!")
    if priority != 0 and not self.scheduler_config.policy == "priority":
        raise ValueError(f"Got priority {
            priority} but "
                        "Priority scheduling is not enabled.")
    if arrival_time is None:
        arrival_time = time.time()

    if self.tokenizer is not None:
        tokenizer = await self.get_tokenizer_async(lora_request)
        self._validate_token_prompt(prompt, tokenizer=tokenizer)

```

```

preprocessed_inputs = await self.input_preprocessor.preprocess_async(
    prompt,
    request_id=request_id,
    lora_request=lora_request,
    prompt_adapter_request=prompt_adapter_request,
)
processed_inputs = self.input_processor(preprocessed_inputs)

if isinstance(params, SamplingParams) and \
    params.guided_decoding is not None:
    # Guided decoding has an async implementation for building logits
    # processors in a separate threadpool.
    # We want to invoke that here instead of using the blocking
    # implementation in the LLMEngine
    params = await build_guided_decoding_logits_processor_async(
        sampling_params=params,
        tokenizer=await self.get_tokenizer_async(lora_request),
        default_guided_backend=self.decoding_config.
        guided_decoding_backend,
        reasoning_backend=self.decoding_config.reasoning_backend,
        model_config=self.model_config)

self._add_processed_request(
    request_id=request_id,
    processed_inputs=processed_inputs,
    params=params,
    arrival_time=arrival_time,
    lora_request=lora_request,
    prompt_adapter_request=prompt_adapter_request,
    trace_headers=trace_headers,
    priority=priority,

```

```

        priority, priority,
    )

    async def check_health_async(self) -> None:
        if self.tokenizer:
            self.tokenizer.check_health()
        self.model_executor.check_health()

    async def build_guided_decoding_logits_processor_async(
        self, sampling_params: SamplingParams, tokenizer: AnyTokenizer,
        default_guided_backend: str, reasoning_backend: Optional[str],
        model_config: ModelConfig) -> SamplingParams:
        """Constructs logits processors based on the guided_decoding,
        logits_bias, and allowed_token_ids fields in sampling_params. Deletes
        those fields and adds the constructed logits processors to the
        logits_processors field. Modifies sampling params in-place and returns
        the modified sampling params."""
        if sampling_params.guided_decoding is None:
            return sampling_params

        # Defensively copy sampling params since guided decoding logits
        # processors can have different state for each request
        sampling_params = copy.copy(sampling_params)
        guided_decoding = sampling_params.guided_decoding

        logger.info(
            "Building guided decoding logits processor. "
            "guided_decoding: %s%s", guided_decoding,
            f", reasoning_backend: {
                reasoning_backend}"
            if reasoning_backend is not None else ""
        )

```



```
guided_decoding.backend = guided_decoding.backend or default_guided_backend
```

```
processor = await get_guided_decoding_logits_processor(  
    guided_params=guided_decoding,  
    tokenizer=tokenizer,  
    reasoning_backend=reasoning_backend,  
    model_config=model_config)
```

```
if processor:  
    if sampling_params.logits_processors is None:  
        sampling_params.logits_processors = []  
    sampling_params.logits_processors.append(processor)
```

```
# Unset guided decoding params after constructing the lp from them  
sampling_params.guided_decoding = None
```

```
return sampling_params
```

```
class AsyncLLMEngine(EngineClient):
```

```
    """An asynchronous wrapper for :class:`LLMEngine`.
```

```
This class is used to wrap the :class:`LLMEngine` class to make it  
asynchronous. It uses asyncio to create a background loop that keeps  
processing incoming requests. The :class:`LLMEngine` is kicked by the  
generate method when there are requests in the waiting queue. The generate  
method yields the outputs from the :class:`LLMEngine` to the caller.
```

```
Args:
```

```
    log_requests: Whether to log the requests.
```

```
    start_engine_loop: If True, the background task to run the engine
```

```

        will be automatically started in the generate call.
    *args: Arguments for :class:`LLMEngine`.
    **kwargs: Arguments for :class:`LLMEngine`.
    """

    _engine_class: Type[_AsyncLLMEngine] = _AsyncLLMEngine

    def __init__(self,
                 *args,
                 log_requests: bool = True,
                 start_engine_loop: bool = True,
                 **kwargs) -> None:
        if envs.VLLM_USE_V1:
            raise ValueError(
                "Using V0 AsyncLLMEngine, but envs.VLLM_USE_V1=True. "
                "This should not happen. As a workaround, try using "
                "AsyncLLMEngine.from_vllm_config(...) or explicitly set "
                "VLLM_USE_V1=0 or 1 and report this issue on Github.")

        self.log_requests = log_requests
        self.engine = self._engine_class(*args, **kwargs)

        # This ensures quick processing of request outputs
        # so the append to asyncio queues is not delayed,
        # especially for multi-step.
        self.use_process_request_outputs_callback = (
            self.engine.model_config.use_async_output_proc)

        if self.use_process_request_outputs_callback:
            self.engine.process_request_outputs_callback = \
                weak_bind(self.process_request_outputs)

```

```

self.background_loop: Optional[asyncio.Future] = None
# We need to keep a reference to unshielded
# task as well to prevent it from being garbage
# collected
self._background_loop_unshielded: Optional[asyncio.Task] = None
self.start_engine_loop = start_engine_loop
self._errored_with: Optional[BaseException] = None

# Lazy initialized fields
self._request_tracker: RequestTracker

def __del__(self):
    if rt := getattr(self, "request_tracker", None):
        # Wake up engine loop so that it will exit cleanly
        rt.new_requests_event.set()

    @classmethod
    def _get_executor_cls(cls,
                        engine_config: VllmConfig) -> Type[ExecutorBase]:
        return LLMEngine._get_executor_cls(engine_config)

    @classmethod
    def from_vllm_config(
        cls,
        vllm_config: VllmConfig,
        start_engine_loop: bool = True,
        usage_context: UsageContext = UsageContext.ENGINE_CONTEXT,
        stat_loggers: Optional[dict[str, StatLoggerBase]] = None,
        disable_log_requests: bool = False,
        disable_log_stats: bool = False,
    ) -> "AsyncLLMEngine":
        """
        Create an AsyncLLMEngine from a VllmConfig.
        """

```

```

        """Create an AsyncLLMEngine from the EngineArgs."""

        return cls(
            vllm_config=vllm_config,
            executor_class=cls._get_executor_cls(vllm_config),
            start_engine_loop=start_engine_loop,
            log_requests=not disable_log_requests,
            log_stats=not disable_log_stats,
            usage_context=usage_context,
            stat_loggers=stat_loggers,
        )

    @classmethod
    def from_engine_args(
        cls,
        engine_args: AsyncEngineArgs,
        start_engine_loop: bool = True,
        usage_context: UsageContext = UsageContext.ENGINE_CONTEXT,
        stat_loggers: Optional[Dict[str, StatLoggerBase]] = None,
    ) -> "AsyncLLMEngine":
        """Creates an async LLM engine from the engine arguments."""

        vllm_config = engine_args.create_engine_config(usage_context)

        async_engine_cls = cls
        if envs.VLLM_USE_V1:
            from vllm.v1.engine.async_llm import AsyncLLM as V1AsyncLLMEngine
            async_engine_cls = V1AsyncLLMEngine

        return async_engine_cls.from_vllm_config(
            vllm_config=vllm_config,
            start_engine_loop=start_engine_loop,

```

```

        usage_context=usage_context,
        stat_loggers=stat_loggers,
        disable_log_stats=engine_args.disable_log_stats,
        disable_log_requests=engine_args.disable_log_requests,
    )

    @property
    def is_running(self) -> bool:
        return (self.background_loop is not None
                and self._background_loop_unshielded is not None
                and not self._background_loop_unshielded.done())

    @property
    def is_stopped(self) -> bool:
        return self.errorred or (self.background_loop is not None and
                                self._background_loop_unshielded is not None
                                and self._background_loop_unshielded.done())

    @property
    def errorred(self) -> bool:
        return self._errorred_with is not None

    @property
    def dead_error(self) -> BaseException:
        return AsyncEngineDeadError(
            "Background loop is not running. If it was running, "
            "inspect the output to find the stacktrace of the "
            "error that caused the background loop to stop "
            "(AsyncEngineDeadError).")

    def set_errorred(self, exc: Exception) -> None:
        self._errorred_with = exc

```

```

def _error_callback(self, exc: Exception) -> None:
    self.set_errored(exc)
    self._request_tracker.propagate_exception(exc)

async def get_input_preprocessor(self) -> InputPreprocessor:
    return self.engine.input_preprocessor

async def get_tokenizer(
    self,
    lora_request: Optional[LoRARequest] = None,
) -> AnyTokenizer:
    return await self.engine.get_tokenizer_async(lora_request)

def start_background_loop(self) -> None:
    """Start the background loop."""
    if self.errored:
        raise AsyncEngineDeadError(
            "Background loop has errored already.") from self._errored_with
    if self.is_running:
        raise RuntimeError("Background loop is already running.")
    # Initialize the RequestTracker here so it uses the right event loop.
    self._request_tracker = RequestTracker()

    self._background_loop_unshielded = asyncio.get_event_loop(
    ).create_task(self.run_engine_loop(weakref.ref(self)))
    self._background_loop_unshielded.add_done_callback(
        partial(_log_task_completion, error_callback=self._error_callback))
    self.background_loop = asyncio.shield(self._background_loop_unshielded)

def shutdown_background_loop(self) -> None:
    """

```

Shut down the background loop.

This method needs to be called during cleanup to remove references to `self` and properly GC the resources held by the async LLM engine (e.g., the executors as well as their resources).

```
"""
```

```
if self._background_loop_unshielded is not None:
    self._background_loop_unshielded.cancel()
    self._background_loop_unshielded = None
self.background_loop = None
```

```
async def engine_step(self, virtual_engine: int) -> bool:
    """Kick the engine to process the waiting requests.
```

```
Returns True if there are in-progress requests."""
```

```
new_requests, aborted_requests = (
    self._request_tracker.get_new_and_aborted_requests())
```

```
for new_request in new_requests:
    # Add the request into the vLLM engine's waiting queue.
    try:
        await self.engine.add_request_async(**new_request)
    except ValueError as e:
        # TODO: use a vLLM specific error for failed validation
        self._request_tracker.process_exception(
            new_request["request_id"],
            e,
            verbose=self.log_requests,
        )
```

```

if aborted_requests:
    await self._engine_abort(aborted_requests)

request_outputs = await self.engine.step_async(virtual_engine)

# Put the outputs into the corresponding streams.
# If used as a callback, then already invoked inside
# LLMEngine's _process_model_outputs
if not self.use_process_request_outputs_callback:
    all_finished = self.process_request_outputs(request_outputs)
else:
    # For callback case, we only need to detect when all
    # requests are finished
    all_finished = all(request_output.finished
                        for request_output in request_outputs)

return not all_finished

def process_request_outputs(self, request_outputs) -> bool:
    # Put the outputs into the corresponding streams.
    all_finished = True
    for request_output in request_outputs:
        self._request_tracker.process_request_output(
            request_output, verbose=self.log_requests)
        all_finished = all_finished and request_output.finished

    return all_finished

async def _engine_abort(self, request_ids: Iterable[str]):
    self.engine.abort_request(request_ids)

```



```

@staticmethod
async def run_engine_loop(engine_ref: ReferenceType):
    """We use a weakref to the engine so that the running loop
    doesn't prevent the engine being garbage collected."""
    engine: Optional[AsyncLLMEngine] = engine_ref()
    if not engine:
        return

    pipeline_parallel_size = \
        engine.engine.parallel_config.pipeline_parallel_size
    has_requests_in_progress = [False] * pipeline_parallel_size
    while True:
        if not any(has_requests_in_progress):
            logger.debug("Waiting for new requests...")
            # Stop the execute model loop in parallel workers until there
            # are more requests to process. This avoids waiting
            # indefinitely in torch.distributed ops which may otherwise
            # timeout, and unblocks the RPC thread in the workers so that
            # they can process any other queued control plane messages,
            # such as add/remove Lora adapters.
            await engine.engine.stop_remote_worker_execution_loop_async()
            request_tracker = engine._request_tracker
            # Allow engine to be garbage collected while
            # waiting for new requests
            del engine
            await asyncio.sleep(0)
            if engine_ref() is None:
                return
            await request_tracker.wait_for_new_requests()
            engine = engine_ref()
            if not engine:
                return

```

```

        return

    logger.debug("Got new requests!")
    requests_in_progress = [
        asyncio.create_task(engine.engine_step(ve))
        for ve in range(pipeline_parallel_size)
    ]
    has_requests_in_progress = [True] * pipeline_parallel_size

    # Abort if iteration takes too long due to unrecoverable errors
    # (eg. NCCL timeouts).
    try:
        async with asyncio.timeout(ENGINE_ITERATION_TIMEOUT_S):
            done, _ = await asyncio.wait(
                requests_in_progress,
                return_when=asyncio.FIRST_COMPLETED)
            for _ in range(pipeline_parallel_size):
                await asyncio.sleep(0)
        for task in done:
            result = task.result()
            virtual_engine = requests_in_progress.index(task)
            has_unfinished_requests = (
                engine.engine.
                has_unfinished_requests_for_virtual_engine(
                    virtual_engine))
            if result or has_unfinished_requests:
                requests_in_progress[virtual_engine] = (
                    asyncio.create_task(
                        engine.engine_step(virtual_engine)))
                has_requests_in_progress[virtual_engine] = True
            else:
                has_requests_in_progress[virtual_engine] = False
    except asyncio.TimeoutError as exc:

```

```

        logger.error(
            "Engine iteration timed out. This should never happen!")
        engine.set_errored(exc)
        raise
    await asyncio.sleep(0)

# This method does not need to be async, but kept that way
# for backwards compatibility.
@overload
@deprecated("'inputs' will be renamed to 'prompt'")
def add_request(
    self,
    request_id: str,
    *,
    inputs: PromptType,
    params: Union[SamplingParams, PoolingParams],
    arrival_time: Optional[float] = None,
    lora_request: Optional[LoRARequest] = None,
    trace_headers: Optional[Mapping[str, str]] = None,
    prompt_adapter_request: Optional[PromptAdapterRequest] = None,
    priority: int = 0,
) -> Coroutine[None, None, AsyncGenerator[Union[
    RequestOutput, PoolingRequestOutput], None]]:
    ...

@overload
def add_request(
    self,
    request_id: str,
    prompt: PromptType,
    params: Union[SamplingParams, PoolingParams],
    arrival_time: Optional[float] = None,

```

```

        lora_request: Optional[LoRARequest] = None,
        trace_headers: Optional[Mapping[str, str]] = None,
        prompt_adapter_request: Optional[PromptAdapterRequest] = None,
        priority: int = 0,
    ) -> Coroutine[None, None, AsyncGenerator[Union[
        RequestOutput, PoolingRequestOutput], None]]:
        ...

    @deprecate_kwargs(
        "inputs",
        additional_message="Please use the 'prompt' parameter instead.",
    )
    async def add_request(
        self,
        request_id: str,
        prompt: Optional[PromptType] = None,
        params: Optional[Union[SamplingParams, PoolingParams]] = None,
        arrival_time: Optional[float] = None,
        lora_request: Optional[LoRARequest] = None,
        trace_headers: Optional[Mapping[str, str]] = None,
        prompt_adapter_request: Optional[PromptAdapterRequest] = None,
        priority: int = 0,
        *,
        inputs: Optional[PromptType] = None, # DEPRECATED
    ) -> AsyncGenerator[Union[RequestOutput, PoolingRequestOutput], None]:
        if inputs is not None:
            prompt = inputs
            assert prompt is not None and params is not None

        if not self.is_running:
            if self.start_engine_loop:
                self.start_background_loop()

```

```

        self.start_background_loop()
    else:
        raise AsyncEngineDeadError(
            "Background loop is not running. If it was running, "
            "inspect the output to find the stacktrace of the "
            "error that caused the background loop to stop "
            "(AsyncEngineDeadError).")

    if (priority != 0
        and not self.engine.scheduler_config.policy == "priority"):
        raise ValueError(f"Got priority {
            priority} but "
                "Priority scheduling is not enabled.")

    stream = self._request_tracker.add_request(
        request_id,
        verbose=self.log_requests,
        prompt=prompt,
        params=params,
        arrival_time=arrival_time or time.time(),
        lora_request=lora_request,
        trace_headers=trace_headers,
        prompt_adapter_request=prompt_adapter_request,
        priority=priority,
    )

    return stream.generator()

    async def generate(
        self,
        prompt: PromptType,
        sampling_params: SamplingParams,

```

```

request_id: str,
lora_request: Optional[LoRARequest] = None,
trace_headers: Optional[Mapping[str, str]] = None,
prompt_adapter_request: Optional[PromptAdapterRequest] = None,
priority: int = 0,
) -> AsyncGenerator[RequestOutput, None]:
    """Generate outputs for a request.

```

Generate outputs for a request. This method is a coroutine. It adds the request into the waiting queue of the LLMEngine and streams the outputs from the LLMEngine to the caller.

Args:

prompt: The prompt to the LLM. See :class:`~vllm.inputs.PromptType` for more details about the format of each input.

sampling_params: The sampling parameters of the request.

request_id: The unique id of the request.

lora_request: LoRA request to use for generation, if any.

trace_headers: OpenTelemetry trace headers.

prompt_adapter_request: Prompt Adapter request to use for generation, if any.

priority: The priority of the request.
Only applicable with priority scheduling.

Yields:

The output `RequestOutput` objects from the LLMEngine for the request.

Details:

- If the engine is not running, start the background loop, which iteratively invokes
:meth:`~vllm.engine.async\_llm\_engine.AsyncLLMEngine.engine\_step`

- to process the waiting requests.
- Add the request to the engine's `RequestTracker`.
On the next background loop, this request will be sent to the underlying engine.
Also, a corresponding `AsyncStream` will be created.
- Wait for the request outputs from `AsyncStream` and yield them.

Example:

```
>>> # Please refer to entrypoints/api_server.py for
>>> # the complete example.
>>>
>>> # initialize the engine and the example input
>>> # note that engine_args here is AsyncEngineArgs instance
>>> engine = AsyncLLMEngine.from_engine_args(engine_args)
>>> example_input = {
>>>     "prompt": "What is LLM?",
>>>     "stream": False, # assume the non-streaming case
>>>     "temperature": 0.0,
>>>     "request_id": 0,
>>> }
>>>
>>> # start the generation
>>> results_generator = engine.generate(
>>>     example_input["prompt"],
>>>     SamplingParams(temperature=example_input["temperature"]),
>>>     example_input["request_id"])
>>>
>>> # get the results
>>> final_output = None
>>> async for request_output in results_generator:
>>>     if await request.is_disconnected():
>>>         # Abort the request if the client disconnects
```

```

>>>         # Abort the request if the client disconnects.
>>>         await engine.abort(request_id)
>>>         # Return or raise an error
>>>         ...
>>>     final_output = request_output
>>>
>>> # Process and return the final output
>>> ...
"""
try:
    async for output in await self.add_request(
        request_id,
        prompt,
        sampling_params,
        lora_request=lora_request,
        trace_headers=trace_headers,
        prompt_adapter_request=prompt_adapter_request,
        priority=priority,
    ):
        yield LLMEngine.validate_output(output, RequestOutput)
except asyncio.CancelledError:
    await self.abort(request_id)
    raise

async def encode(
    self,
    prompt: PromptType,
    pooling_params: PoolingParams,
    request_id: str,
    lora_request: Optional[LoRARequest] = None,
    trace_headers: Optional[Mapping[str, str]] = None,
    priority: int = 0,

```



```
) -> AsyncGenerator[PoolingRequestOutput, None]:  
    """Generate outputs for a request from a pooling model.  
  
    Generate outputs for a request. This method is a coroutine. It adds the  
    request into the waiting queue of the LLMEngine and streams the outputs  
    from the LLMEngine to the caller.  
  
    Args:  
        prompt: The prompt to the LLM. See :class:`~vllm.inputs.PromptType`  
            for more details about the format of each input.  
        pooling_params: The pooling parameters of the request.  
        request_id: The unique id of the request.  
        lora_request: LoRA request to use for generation, if any.  
        trace_headers: OpenTelemetry trace headers.  
        priority: The priority of the request.  
            Only applicable with priority scheduling.  
  
    Yields:  
        The output `PoolingRequestOutput` objects from the LLMEn
```